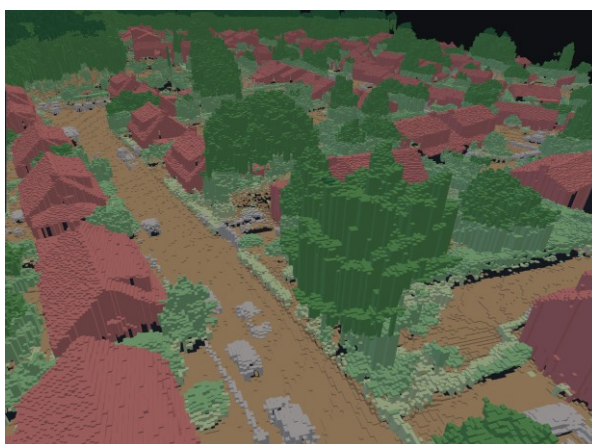


DANS LE CADRE DU PROJET IA.RBRE



Mémoire de Master

Voxelisation de données LiDAR urbaines : une représentation sémantique 3-D compressée par colonnes pour le projet IA.rbre



Auteur :

Nikolaos VYNIOS
Master 2 Informatique - Gamagora

Encadrants au laboratoire :

John SAMUEL - Gilles GESQUIÈRE

Encadrant académique :

Didier PUZENAT

Mai 2026 - Septembre 2026

Résumé

Les villes qui s'adaptent au changement climatique ont besoin de savoir où les arbres peuvent pousser, et cette question est volumétrique : les houppiers ont besoin d'air libre, les racines ont besoin de sol, et les deux entrent en concurrence avec les bâtiments et les infrastructures. Le projet IA.rbre construit une plateforme d'aide à la décision pour la végétalisation urbaine dans la Métropole de Lyon, et ce stage en fournit la couche volumétrique. Nous présentons la conception, l'implémentation et la validation d'un voxeliseur : un système qui lit les tuiles LiDAR HD classifiées de l'IGN et représente la ville par des voxels sémantiques, organisés en colonnes verticales indépendantes de leurs voisines suivant le cadre proposé par l'équipe encadrante. Chaque position (x, y) porte un profil vertical d'intervalles de matériau (sol, végétation, bâtiment), bien plus compact que les points qu'il résume. La structure est complétée par un décodeur d'espaces vides à trois états qui distingue l'air mesuré de l'intérieur de bâtiment non mesuré et du sous-sol. La validation sur 74 millions de points confirme la compacité de la représentation, sans perte pour les produits en vue en plan ; une simplification par majorité, testée, a été rejetée parce que la mesure a montré qu'elle écarte sensiblement l'empreinte de végétation en vue en plan de l'empreinte brute. Les exécutions de production vont d'une zone fusionnée de 751 millions de points à la Métropole entière, 2 842 tuiles et 51,9 milliards de points traités à trois résolutions par une chaîne out-of-core partitionnée, et la structure exporte des cartes, des statistiques, des visualiseurs 3-D en navigateur et du 3D Tiles. Une couche de lancer de rayons sensible aux classes traite la végétation comme partiellement transmissive, et une première expérience d'heures d'ensoleillement direct mesure ce que le modèle volumétrique apporte par rapport à un modèle opaque.

Table des matières

Résumé.....	2
1 Introduction.....	4
1.1 Le laboratoire LIRIS.....	4
1.2 Le projet VCity.....	4
1.3 Le projet IA.rbre : fonctionnement et financement.....	4
2 Problématique.....	5
2.1 Le sujet, les objectifs et les contraintes de conception.....	5
2.2 Données d'entrée.....	6
2.3 Données de sortie.....	6
2.4 Périmètre : cadrage initial et livraison.....	7
3 État de l'art.....	10
3.1 Structures de données par voxels.....	10
3.2 Méthodes de traitement sur grilles de voxels.....	11
3.3 Systèmes et outils.....	11
4 Proposition.....	14
4.1 Architecture.....	14
4.2 Conception.....	16
4.3 Implémentation.....	20
4.4 Requêtes par lancer de rayons : de l'occlusion à la transmittance sensible aux classes.....	23
5 Résultats.....	27
5.1 Tests.....	27
5.2 Validation.....	28
5.3 Démonstration.....	35
5.4 Discussion et limitations.....	40
5.5 Évaluation d'ensemble.....	41
6 Conclusion et perspectives.....	42
6.1 Au regard de la note de cadrage initiale.....	42
6.2 Perspectives.....	43
6.3 Remerciements.....	44
Bibliographie.....	45
Annexes.....	48
Annexe A - Algorithmes détaillés et pseudo-code.....	48
Annexe B - Référence CLI, modules et configuration.....	52
Annexe C - Données d'expérience et format de sérialisation.....	54
Annexe D - Figures supplémentaires.....	67
Annexe E - Codes de classification LiDAR HD de l'IGN.....	71
Annexe F - Glossaire.....	72

1 Introduction

Ce rapport présente un stage effectué au laboratoire LIRIS dans le cadre du projet IA.rbre, portant sur la conception et l'implémentation d'une représentation par voxels de l'espace urbain à partir de données LiDAR aériennes. Cette introduction présente le laboratoire d'accueil du stage (1.1), le projet de recherche auquel il appartient (1.2), puis le projet IA.rbre et son financement (1.3).

1.1 Le laboratoire LIRIS

Ce stage s'est déroulé au LIRIS (Laboratoire d'InfoRmatique en Image et Systèmes d'information), laboratoire d'informatique à Lyon, dont les tutelles sont le CNRS, l'INSA Lyon, l'Université Claude Bernard Lyon 1, l'Université Lumière Lyon 2 et l'École Centrale de Lyon. Le laboratoire compte environ 330 membres répartis en 12 équipes, couvrant des domaines de recherche allant de l'informatique graphique et de la géométrie à la science des données, à la vision et à la simulation. Pendant ce stage, j'ai intégré l'équipe Origami, dont la spécialité est l'informatique graphique, avec une expertise complémentaire en vision par ordinateur et en algorithmique. Le travail rapporté ici a été mené dans cet environnement sous l'encadrement de John Samuel et du Pr Gilles Gesquière, et avec l'accompagnement de Corentin Gautier et d'Arthur Villarroya-Palau.

1.2 Le projet VCity

Au sein du LIRIS, le projet VCity agrège des projets issus de plusieurs équipes, principalement autour de la modélisation géométrique et de la science des données appliquées à la ville : la modéliser, l'analyser et la visualiser. Le projet s'appuie sur la disponibilité croissante de données urbaines ouvertes produites par les collectivités territoriales, données à la fois tridimensionnelles, sémantiques et topologiques, et il contribue aux standards de l'Open Geospatial Consortium qui rendent ces données interopérables. L'écosystème logiciel de VCity (entre autres l'outillage py3DTilers et la ligne d'analyse solaire pySunlight [1], tous deux détaillés au chapitre 3) constitue le contexte technique immédiat de ce stage : la structure de voxels construite ici est conçue pour interopérer avec ces outils et pour alimenter les mêmes analyses à l'échelle de la ville.

1.3 Le projet IA.rbre : fonctionnement et financement

IA.rbre (Intelligence Artificielle pour l'arbre) est le projet dans lequel se déroule ce stage. Il co-développe une plateforme web pour le traitement et l'analyse de données volumineuses, qui aide les villes à décider où et comment planter des arbres, sur le territoire de la Métropole de Lyon et avec l'objectif explicite d'être répliquable dans d'autres villes françaises [2]. La plateforme traite deux questions : où la plantation est-elle faisable (réseaux souterrains, sol, infrastructures existantes, surplombs de bâtiments) et où elle est la plus nécessaire (ensoleillement, rafraîchissement urbain, qualité de l'air, biodiversité, équité) ?

Le consortium est mené par la coopérative TelesCoop, avec la Métropole de Lyon (appuyée par Exo-dev) et l'Université Lumière Lyon 2 via le LIRIS comme partenaires. Le projet se déroule sur 36 mois (2025 à 2027) et est financé dans le cadre du programme France 2030 par la Banque des Territoires au titre de l'appel DIAT, « Démonstrateurs d'IA frugale au service de la transition écologique des territoires » [3]. Deux de ses principes affichés concernent directement ce rapport : la frugalité (privilégier, quand c'est possible, des méthodes interprétables et peu gourmandes en ressources plutôt que de l'apprentissage lourd) et la science ouverte (données ouvertes, logiciel libre, reproductibilité complète), auxquels répondent les choix de conception du chapitre 4.

2 Problématique

2.1 Le sujet, les objectifs et les contraintes de conception

Le sujet, tel que formulé par la note de cadrage du laboratoire, s'énonce ainsi : structures de données pour représenter l'information urbaine volumétrique sous forme de voxels. Cette seule ligne repose sur un besoin opérationnel concret du projet IA.rbre. Décider où un arbre peut être planté exige de raisonner sur le volume, non sur la surface : la couronne a besoin d'air libre, les racines ont besoin de sol, et les deux entrent en concurrence avec les bâtiments, les routes et les infrastructures. Les modules existants du projet (détection de la végétation, désimperméabilisation, ensoleillement) raisonnent en 2,5 dimensions, sur des surfaces et des rasters ; la note de cadrage demandait à ce stade d'ajouter la dimension volumétrique, du point de vue de ce que le relevé LiDAR d'une ville peut réellement mesurer. Le problème est donc le suivant : étant donné la couverture LiDAR aérienne classifiée de la Métropole de Lyon, construire une représentation du volume urbain qui soit simultanément sémantique (chaque cellule occupée porte une classe de matériau), assez compacte pour que des zones métropolitaines tiennent sur une station de travail ordinaire, et interrogeable dans les termes de la végétalisation urbaine, qui sont avant tout verticaux : quel est le matériau le plus haut en ce point ? quelle est la hauteur de la canopée ? quel volume d'air libre s'étend au-dessus de ce trottoir ? la lumière traverse-t-elle cette colonne ?

Dès le départ, les encadrants ont proposé des lignes directrices et des limites de conception qui canalisent le travail vers son objectif. Premièrement, pas d'octrees : la note de cadrage écarte cette piste, l'intérêt scientifique du projet portant sur une structure de données géospatiales interopérable, et non sur une hiérarchie d'accès spatiale. La revue du chapitre 3 montre que cette famille de structures reste active, des systèmes à octree de 2023 et de 2024 figurant parmi les travaux récents les plus proches. Deuxièmement, la structure doit être indifférente au voisinage, traitant chaque position (x, y) comme un profil vertical indépendant, ce qui parallélise trivialement le traitement et permet aux zones de s'assembler par concaténation. Troisièmement, la dimension verticale doit être compressée par plages de matériau identique ; les encadrants ont esquissé ce codage en réunion avant qu'il ne soit jamais nommé codage par plages, et le chapitre 4 le formalise. Quatrièmement, les résultats doivent rester interopérables avec l'écosystème géospatial plus large dans lequel travaille le laboratoire, ce qui signifie en pratique une voie d'export vers 3D Tiles et les visualiseurs web. Cinquièmement, l'implémentation doit être reproductible par des non-spécialistes au sein de la plateforme IA.rbre : Python, dépendances minimales, aucune chaîne de compilation.

En suivant ces contraintes, le stage s'est fixé cinq objectifs mesurables :

- O1, voxeliser sans perte : les tuiles LiDAR HD classifiées isolées deviennent une structure sémantique en colonnes ; aucune information ne peut être écartée à la construction.
- O2, la compacité à fidélité connue : la représentation réduit les données de points autant que la fidélité l'autorise ; aucune étape ne perd d'information par effet de bord, toute simplification avec perte est un choix explicite et mesuré.
- O3, le passage à l'échelle : des zones entières de la Métropole se traitent sur une station de travail dotée de quelques dizaines de gigaoctets de mémoire, ce qui interdit toute étape conservant l'intégralité des données à la fois.
- O4, l'aptitude à l'analyse : la structure sert directement les cartes, les statistiques, l'inspection tridimensionnelle, les exports géospatiaux standards et les requêtes par lancer de rayons.
- O5, une sémantique rigoureuse des espaces vides : la structure distingue l'air mesuré de l'intérieur non mesuré et du sol, car la justesse du raisonnement physique en aval (l'ombrage, et à terme la plantabilité) est bornée par cette distinction.

Le chapitre 5 évalue chaque objectif au regard des mesures.

Trois hypothèses réduisent le périmètre. La classification sémantique ne fait pas partie du problème : l'IGN livre les nuages de points déjà étiquetés, et re-résoudre la classification dupliquerait une chaîne industrielle. Les données du sous-sol sont hors périmètre en tant qu'entrée : aucun jeu de données souterrain n'est consommé, et la représentation traite ce qui se trouve sous la surface du sol comme un état du décodeur, laissant l'intégration des données de réseaux et de sols aux travaux futurs. Enfin, les orthophotos ne sont pas une entrée : la chaîne de traitement peut draper ses cartes de classes sur une orthophoto à des fins de vérification visuelle, mais aucun calcul ne lit l'imagerie.

2.2 Données d'entrée

L'entrée est une source de données unique : les tuiles LiDAR HD classifiées couvrant la Métropole de Lyon, produites dans le cadre du programme national de l'IGN [4] et distribuées pour la Métropole via l'inventaire Grand Lyon 2023 [5]. Chaque tuile est un fichier LAZ (la forme compressée LASzip [6] du format ASPRS LAS [7]) couvrant 500 x 500 m au sol, nommé par les coordonnées de son coin sud-ouest. Le système de coordonnées de référence, lu dans les en-têtes de fichier, est RGF93/CC46 (EPSG:3946), la zone conique conforme couvrant Lyon ; le lecteur du code valide le système de coordonnées de chaque tuile au chargement et rejette toute tuile au repère absent ou étranger, précaution née des premières expériences menées sur l'acquisition de 2018 (tuiles de 1 km en Lambert-93, EPSG:2154). L'uniformité de millésime reste une propriété du corpus vers lequel une exécution est dirigée.

Une tuile contient environ 16 millions de points en valeur médiane. Un point du format livré porte ses trois coordonnées, une intensité, un numéro de retour, un angle de balayage, un horodatage GPS et un code de classification. Quatre de ces valeurs servent notre problème : les trois coordonnées, qui placent le point dans la grille, et le code de classification, qui porte la sémantique ; le reste n'alimente aucune requête visée et n'est pas consommé.

Les codes de classification portent la sémantique de l'entrée : sur l'ensemble des 2 842 fichiers, les codes réellement présents vont de 1 à 9 (table complète et précisions de format en annexe E). Le code 8 demande une clarification : dans les livraisons du Grand Lyon, il est utilisé comme un étage supérieur de canopée situé au-dessus de la classe 5, et il est substantiel : 3,32 milliards de points, 6,4 % du corpus, présent dans 2 691 des 2 842 tuiles. Le système le nomme et le restitue comme tel. Deux faits relatifs à cette entrée façonnent la conception : la classification est déjà faite, et les données sont livrées par tuile (la Métropole n'arrive jamais en un seul fichier, donc la représentation doit fusionner).

2.3 Données de sortie

La sortie comporte un produit principal et des produits dérivés. Le produit principal est le ColumnStore : la structure en colonnes compressée par plages du chapitre 4, présentée soit comme archive compressée pour l'échange, soit comme répertoire projetable en mémoire pour un usage en chaîne de traitement, pour un coût d'environ 16 octets par colonne occupée plus 13 octets par intervalle. C'est la source unique dont dérive tout autre produit, et elle est réversible : un voxeliseur inverse peut en reconstruire un nuage de points LAS/LAZ standard pour audit, par n'importe quel outil externe. Les sorties dérivées servent trois publics, l'analyse, l'inspection et l'intégration (Tableau 1). La Figure 1 résume les entrées et les sorties, et le Tableau 1 les détaille artefact par artefact ; les chapitres 4 et 5 les justifient et les évaluent respectivement.

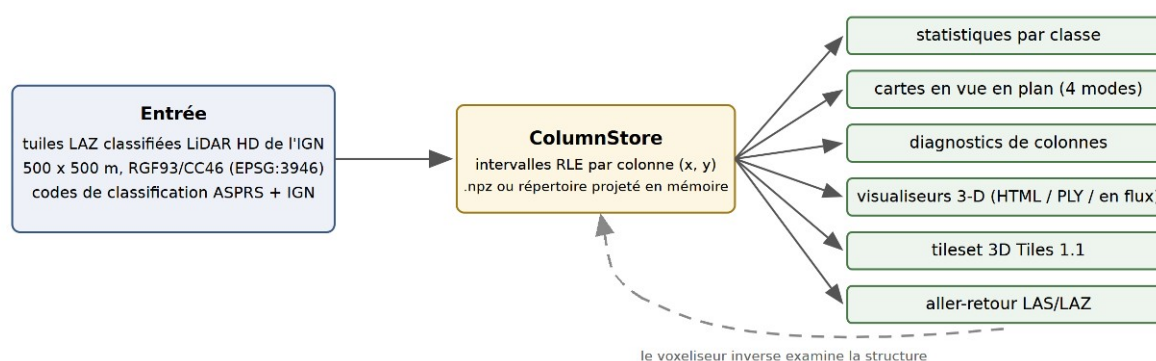


Figure 1 : Entrées et sorties du système : tuiles LiDAR HD en entrée, le ColumnStore au centre, produits dérivés en sortie.

Tableau 1 : Entrées et sorties du système : chaque artefact, son rôle et son format.

Artefact	Rôle	Format et détail
Tuiles LiDAR HD classifiées (entrée)	Source de données unique	LAZ, 500 × 500 m, EPSG:3946 ; 2 842 tuiles, ~16 M points par tuile en médiane ; consommés : x, y, z et classe
ColumnStore (produit principal)	Structure en colonnes compressée par plages, source de tout dérivé	Archive compressée ou répertoire projetable en mémoire ; ~16 octets par colonne occupée + 13 par intervalle
Statistiques et diagnostics	Analyse	Statistiques par classe, diagnostics par colonne
Cartes en vue en plan	Inspection	Quatre modes par tuile ou zone : hauteur maximale, classe dominante, classe sur orthophoto, nombre d'intervalles
Visualiseurs tridimensionnels	Inspection	Jusqu'à un visualiseur en flux à l'échelle de la ville
Nuage de points reconstruit	Audit et interopérabilité	LAS/LAZ standard, par le voxeliseur inverse, lisible par tout outil externe
Tileset 3D Tiles 1.1	Intégration géospatiale	Chargeable par les clients standards aux côtés des autres couches d'IA.rbre [9]

2.4 Périmètre : cadrage initial et livraison

Les cinq objectifs de la section 2.1 donnent une forme mesurable aux cinq buts de la note de cadrage du stage. Cette section énonce la correspondance, but par but : ce que chaque but demandait, ce que ce travail en livre, et les choix faits en chemin avec leurs raisons.

La note de cadrage visait à (i) concevoir un modèle de données voxelisé couvrant la végétation, le sol, les bâtiments et les structures du sous-sol ; (ii) construire ce modèle à partir de jeux de données existants incluant le LiDAR, les DSM et CityGML ; (iii) classer et catégoriser les voxels ; (iv) expérimenter des algorithmes de projection d'ombres ou de lancer de rayons volumétrique tenant

compte des interactions entre voxels de classes territoriales différentes ; et (v) comparer les résultats aux approches 2-D et 2.5-D existantes, telles que les modèles numériques de surface et les rasters de rayonnement solaire, afin d'évaluer les gains en précision, en réalisme et en interprétabilité.

Les cinq buts ont été menés. Trois font l'objet des chapitres 4 et 5 ; la projection d'ombres a été menée jusqu'à une expérience mesurée à l'échelle d'une zone, et la comparaison aux approches 2-D et 2.5-D jusqu'à des résultats mesurés sur dix kilomètres carrés stratifiés. Un choix d'entrée et une limite de mesure délimitent le périmètre, et chaque décision est consignée ici avec sa justification.

Le choix d'entrée est une source de données unique : le système n'ingère que du LiDAR aéroporté. Les rasters DSM et les modèles CityGML ne sont pas consommés. La raison est la valeur probante : une source uniformément classifiée couvrant toute la Métropole (2 842 tuiles, 51 912 212 206 points, un unique schéma de classification) permet de caractériser rigoureusement la représentation et ses encodages sur un territoire entier. Ajouter des sources hétérogènes aurait multiplié la surface d'ingestion et de réconciliation sans approfondir la question que le travail pose réellement, à savoir comment l'information territoriale volumétrique doit être représentée et compressée. L'objectif (ii) est donc rempli sur la source la plus probante, le LiDAR ; l'intégration des DSM et de CityGML est laissée aux travaux futurs.

La limite de mesure porte sur le sol et le sous-sol, que l'entrée disponible ne peut pas observer. Le LiDAR aéroporté mesure la première surface qu'il frappe ; sous le retour du sol, il ne porte aucune information. Le schéma de classification présent dans les tuiles livrées (codes 1 à 9) ne contient ni horizon pédologique ni classe de sous-sol : il n'y a donc rien à représenter dans l'entrée. Le modèle livré couvre par conséquent la végétation, les bâtiments, la surface du sol et l'eau. Représenter des horizons de sol ou des structures enfouies exigerait une tout autre modalité de mesure – radar à pénétration de sol, relevés de forage, prospection pédologique – et l'obstacle relève donc de l'acquisition de données. La structure de voxels n'est pas la limitation : `classes_config.py` porte déjà une table de codes taillée pour LAS 1.4 avec de la place pour de telles classes, et la section 4.2 montre le décodeur raisonnant sur le sous-sol comme sur un état. Ce qui manque, c'est la mesure.

La projection d'ombres (objectif iv) a été réalisée. Une primitive DDA de parcours de rayons sur la structure en colonnes (`voxelizer/ray_trace.py`, section 4.4) répond, directement sur la représentation compressée, à « que rencontre un rayon parti de ce point dans cette direction, et dans quel ordre ». L'expérience a été menée sur deux zones contrastées, avec des positions solaires validées contre l'astronomie sphérique et une occlusion sensible aux classes ; la section 4.4.6 en rend compte.

La comparaison aux approches 2-D et 2.5-D (objectif v) a été livrée. Elle est construite pour que la représentation soit la seule variable : trois modèles comparés sur le même LiDAR, répondant aux mêmes requêtes par le même code, à savoir la structure volumétrique en colonnes, un modèle de surface 2.5-D qui réduit chaque colonne à un corps unique entre son retour le plus bas et son retour le plus haut, et un jeu de rasters 2-D reproduisant le modèle de données du module de végétalisation voisin (section 3.3). Un quatrième modèle ne change que la hauteur du point d'échantillonnage, afin de distinguer celle-ci du modèle d'occlusion. Sur dix kilomètres carrés stratifiés et à deux grilles, le modèle de surface surestime nettement le volume de canopée (l'empreinte multipliée par la hauteur compte comme canopée toute la colonne située sous une couronne), ne peut pas représenter les colonnes portant deux corps séparés par de l'air mesuré, et surestime l'ensoleillement direct partout où une canopée surmonte le sol ; le coût va en sens inverse sur un point, la lecture scalaire par cellule restant plus rapide côté raster. Les chiffres sont rapportés au chapitre 5 (section 5.2.2), la conception et les vérifications en section 4.4.7, les tableaux en annexe C. Une comparaison avec un produit externe de provenance indépendante reste une étape ultérieure (discussion en 6.1).

« 2.5-D » désigne deux choses différentes, un modèle de données et une organisation de stockage, et seule la première est la limitation dont traite cette comparaison ; la section 4.4.1 énonce la distinction en entier et l'applique à cette structure. La divergence mesurée ci-dessus se situe là où cette distinction la

place : sous la canopée, sous les surplombs et dans la structure verticale d'une rue, là où le modèle numérique de surface n'a aucune information à offrir.

L'objectif (iii) – la classification et la catégorisation des voxels – est atteint intégralement ; l'objectif (i) – le modèle voxelisé lui-même – est mené intégralement sur tout ce que l'entrée mesure, la part sol et sous-sol relevant de la limite du LiDAR ci-dessus. Les deux font l'objet des chapitres 4 et 5.

3 État de l'art

La voxelisation, c'est-à-dire la conversion de données géométriques en une grille régulière d'éléments de volume, est un domaine mature dont l'espace de conception est bien travaillé. La revue d'Aleksandrov *et al.* [10] l'organise selon deux axes : l'algorithme utilisé pour décider quels voxels une primitive occupe (division entière pour les points, Bresenham 3-D pour les segments, tests conservatifs ou 6-séparants pour les triangles, tests de parité pour les solides), et la structure de données utilisée pour conserver le résultat. Pour notre problème, l'axe algorithmique est essentiellement réglé : le LiDAR aérien classifié arrive sous forme de points, et les points se voxelisent par quantification, une division entière de chaque coordonnée par la taille de cellule. La question ouverte, celle à laquelle ce stage devait répondre, est la suivante : quelle structure doit porter un modèle sémantiquement étiqueté d'une Métropole, et quelle structure répond le mieux aux questions verticales que la végétalisation urbaine lui pose ?

Ce chapitre passe en revue les structures de données par voxels (3.1), les méthodes de traitement construites sur elles (3.2) et les systèmes existants (3.3), chaque revue exposant ses limites pour notre usage, et conclut sur le besoin que ces limites laissent ouvert.

3.1 Structures de données par voxels

Le point de départ naïf est la grille dense en un unique tableau `grid[ix][iy][iz]` : accès en temps constant et implémentation triviale, mais un coût mémoire rédhibitoire. Une tuile de 500 m (médiane autour de 16 millions de points) à 1 m de résolution horizontale et 0,5 m de résolution verticale couvre de l'ordre de cinquante millions de cellules (détail en annexe C), dont l'écrasante majorité est de l'air ; à 0,5 m au cube, le compte croît encore de plusieurs facteurs. Même à un octet par cellule, cela dépasse la mémoire vive courante dès que quelques tuiles sont fusionnées, et presque chaque octet code du vide. Découper l'espace en blocs (chunks) répartit cette mémoire sans la réduire : la parcimonie doit venir de la structure elle-même. Les structures passées en revue ci-dessous existent précisément pour éviter ce coût.

La solution classique est l'octree, qui ne subdivise récursivement l'espace que là où il y a du contenu ; il peut aussi n'être qu'une simple décomposition de l'espace dont le plus petit élément est le voxel. Les espaces de voxels structurés en octree ont porté une grande part de la littérature de traitement LiDAR. Wang et Tseng segmentent des nuages de points aéroportés de façon incrémentale, directement dans un tel espace [11]. Le moteur d'intégration de voxels de Gorte *et al.* stocke une pyramide d'octree à cinq niveaux dans SQLite, avec quatre opérateurs génériques faisant office d'API de requête complète sur le terrain, les bâtiments, les routes et la végétation [12], et une ligne de travaux connexe étend le même cadre à l'analyse de très grands jeux de données de voxels [13].

Les DAG (graphes orientés acycliques) de voxels creux [14] compriment davantage en partageant les sous-arbres identiques, dans les proportions que reporte le Tableau 2.

Ces structures sont puissantes, mais deux considérations ont pesé contre elles ici. Elles optimisent l'uniformité spatiale (la même logique de subdivision sur les trois axes) alors que nos requêtes sont manifestement anisotropes : la plantabilité, la structure de la canopée et l'ombrage interrogent tous la colonne verticale au-dessus ou au-dessous d'une position (x, y) . Aucune de ces structures n'est nativement organisée en colonnes : une question verticale doit réassembler sa réponse à partir de nœuds dispersés. À cela s'ajoute une contrainte de cadrage : les encadrants ont exclu les octrees dès le départ, la note de cadrage cherchant une contribution structurelle différente (Tableau 2).

OpenVDB [15], standard de fait des effets visuels, empile une hiérarchie de profondeur fixe de nœuds denses sous une table de hachage, avec compression par tuiles des régions constantes et parcours de rayons DDA ; NanoVDB l'étend aux GPU et à l'out-of-core [16]. Pour une tuile de 500 m à 0,5 m, il code l'occupation en quelques dizaines de méga-octets là où la grille dense en exige des centaines.

Comme le note le Tableau 2, son écosystème est en C++ et son type de valeur est par voxel, non par plage : une requête en colonne s'assemble voxel par voxel.

Des solutions plates plus simples existent également : découper la grille dense en briques de taille fixe et hacher celles qui sont occupées se parallélise trivialement et se projette directement sur les formats de stockage par blocs, et une brique de 16^3 , soit 4 096 cellules, pèse quelques kilooctets face à un cache de données L1 typique de 32 KB. Les briques restent denses : un seul voxel occupé alloue encore les 4 095 autres.

À l'autre extrême, les représentations implicites ne stockent aucune grille : les champs de distance signée répondent à « à quelle distance est la surface la plus proche » depuis la fonction elle-même, sous forme fermée pour les primitives paramétriques que dérive Hart [17], et par une transformée de distance euclidienne exacte de l'image d'occupation pour les données échantillonnées [18], ce qui rend les requêtes de zone d'exclusion exactes mais la sémantique par classe malaisée. Chacune de ces options a été évaluée pendant la phase de conception, et le Tableau 2 résume cette évaluation ; la section 4.2.1 argumente ensuite la structure retenue face à la meilleure alternative interne au projet, sans rouvrir la revue.

Le plus proche de notre encodage par l'esprit est la littérature de la compression. Tzamarias *et al.* [19] compressent la géométrie de nuages de points voxelisés en organisant les voxels occupés en plages et en les codant en une seule passe directement dans le domaine des voxels, explicitement pour éviter les octrees et les matrices d'occupation denses. Cela confirme, depuis une communauté sans lien, que la structure par plages exploite directement la parcimonie des nuages de points. Le codage par plages est lui-même parmi les plus anciens résultats du domaine [20], et l'observation que le stockage par colonnes s'accorde aux requêtes par colonnes est l'argument fondateur des systèmes de bases de données en colonnes [21]. Ce que les travaux de compression n'apportent pas, c'est une structure prête pour l'analyse : leurs plages codent une occupation binaire destinée à la transmission, non des étiquettes sémantiques par plage, des densités de points et des opérateurs de requête.

3.2 Méthodes de traitement sur grilles de voxels

Un second ensemble de travaux utilise les voxels non comme produit final mais comme unité de calcul, et il nous importe surtout par ce qu'il implique sur les entrées.

Aljumaily *et al.* [22] classifient du LiDAR urbain aérien de Dublin, d'une densité environ quadruple de nos tuiles, par forêt aléatoire sur des descripteurs par voxel, réduisant de 99 % le nombre d'éléments à traiter par rapport à un traitement point par point ; Poux et Billen [23] séparent, sans supervision, les toits plans des canopées par la seule géométrie. Le Tableau 2 en donne les entrées, les sorties et les scores.

En France, ce problème de classification est de fait industrialisé : le programme LiDAR HD de l'IGN distribue des nuages dont chaque point porte déjà sa classe, attribuée par la chaîne myria3d fondée sur RandLA-Net [24] et validée sur des corpus tels que FRACTAL [25] (100 000 nuages annotés, 250 km², sept classes) [4]. Ce fait gouverne notre conception : la classification est déjà dans l'entrée. Le stage réorganise les classifications existantes sans en produire de nouvelles, transformant des dizaines de millions de points étiquetés par tuile en une structure qui répond sans perte et efficacement aux requêtes d'analyse urbaine, à l'échelle métropolitaine.

3.3 Systèmes et outils

Trois systèmes encadrent notre travail de la façon la plus directe. LiVoxGen [26] en est l'ancêtre direct : un outil C++ voxelisant du LiDAR en listes de voxels par colonne pour en dériver des métriques de canopée (hauteur et densité du couvert). Nous en avons retenu l'idée centrale, la colonne comme unité

d'organisation, et rejeté son implémentation en bloc, dont le kd-tree résout en $O(n \log n)$ une affectation à la grille qui relève de l'arithmétique entière en $O(n)$; le Tableau 2 en récapitule les autres obstacles.

Arches, développé dans ce laboratoire, en est l'ancêtre conceptuel : Peytavie *et al.* modélisent les terrains comme une grille 2-D de piles verticales de segments (matériau, épaisseur) représentant nativement les surplombs et les cavités qu'aucun champ de hauteurs n'exprime [27]. Notre système en est le pendant du côté de la mesure, dérivant le même modèle en colonnes empilées à partir d'observations LiDAR réelles.

Enfin, le cadre VoxCity [28] construit des maquettes 3-D sémantiques de villes à partir de rasters mondiaux librement accessibles, puis y exécute des simulations d'ensoleillement et de visibilité ; il partage nos ambitions de sortie mais consomme des rasters 2.5-D dérivés plutôt que du LiDAR brut classifié, d'où une fidélité verticale plus grossière (Tableau 2).

Au sein de l'écosystème VCity lui-même, les deux outils se partagent le travail : py3DTilers assemble des tilesets 3D Tiles (le format ouvert de l'OGC [9] qui diffuse de grands modèles 3-D en tuiles hiérarchiques) à partir de formats géométriques tels que OBJ, GeoJSON, IFC et CityGML, et pySunlight calcule l'ensoleillement sur ces tilesets, en les lisant via py3DTilers et en déléguant le calcul de lumière à la bibliothèque C++ Sunlight [1]. Tous deux travaillent sur des maillages, des représentations de ville plutôt que des structures volumétriques, et Jaillot *et al.* décrivent une approche générique pour calculer l'impact de l'ensoleillement et de l'ombrage sur de tels grands modèles de ville [29].

Au sein d'IA.rbre lui-même, un module voisin produit la ligne de produits 2-D et 2.5-D. Il dérive des GeoTIFF à 1 m à partir du même LiDAR du Grand Lyon et de l'orthophotographie correspondante (un modèle de terrain, un modèle de surface et leur différence) et y fusionne une segmentation de l'occupation du sol pixel par pixel produite par le modèle FLAIR-HUB [30], pour aboutir à un raster de végétation à trois classes indexé sur la hauteur au-dessus du terrain. C'est la forme concrète du modèle de données 2.5-D auquel la section 4.4.7 confronte cette structure : une valeur par cellule, et aucun moyen de demander ce qui existe dessous.

Au terme de cette revue, aucune méthode recensée ne combine les cinq propriétés qu'exige notre énoncé du problème : l'intégration native de la sémantique LiDAR HD de l'IGN ; une organisation en colonnes indifférente au voisinage, accordée aux requêtes verticales (contrainte de cadrage, motivée au chapitre 4) ; la compacité par plages avec étiquettes de classe et comptages de points par plage ; un fonctionnement out-of-core pour que des zones métropolitaines se traitent sur une station de travail ordinaire ; et une implémentation en Python pur, à faible dépendance, pour la reproductibilité au sein de la plateforme IA.rbre. Le Tableau 2 ci-dessous condense cette lecture méthode par méthode. La proposition du chapitre 4, une structure en colonnes compressée par plages dotée d'un décodeur d'espaces vides à trois états et d'une chaîne de partitionnement out-of-core, est conçue pour répondre à ce besoin.

Tableau 2 : Synthèse de l'état de l'art. La dernière ligne situe notre approche dans les mêmes colonnes.

Méthode	Structure de données	Entrées	Sorties	Remarques
Grille dense, y compris les variantes en briques adaptées au cache [10]	tableau 3-D, éventuellement découpé en briques denses	toute donnée voxelisable	grille adressable en $O(1)$	Simple et parallèle ; $\sim 5 \times 10^7$ cellules par tuile de 500 m, essentiellement de l'air ; le découpage en briques aide la localité mais conserve l'air à l'intérieur des briques occupées ; infaisable à l'échelle métropolitaine
Segmentation par octree [11]	espace de voxels structuré en octree	points LiDAR bruts	segments coplanaires (toits, murs, sol)	Mature ; subdivision isotrope inadaptée aux requêtes verticales ; octrees exclus par la note de cadrage
Moteur d'intégration de voxels [12], [13]	octree adossé à SQLite, pyramide à 5 niveaux	DEM, solides LOD2, routes, végétation	scène de voxels sémantique unifiée + 4 opérateurs	Système récent le plus proche ; dépendance à une base de données ; noyau octree
DAG de voxels creux [14]	octree dédoublé en DAG	occupation binaire	scène statique compressée	28 à 576 fois moins de nœuds que l'octree de même résolution sur les cinq scènes testées ; binaire seulement, pas de sémantique
OpenVDB / NanoVDB [15], [16]	hiérarchie à profondeur fixe par hachage	valeurs volumétriques	accès en $O(1)$, parcours DDA, out-of-core	Standard de l'industrie ; écosystème C++ ; valeurs par voxel, pas de sémantique par plage
Champ de distance signée [17], [18]	fonction de distance en forme fermée ou grille de distances échantillonnée	occupation ou primitives	distance à la surface la plus proche	Idéal pour les zones d'exclusion ; la sémantique par classe exige un champ par classe
Compression RLE de nuages de points [19]	plages dans le domaine des voxels	nuages de points voxelisés	géométrie compressée sans perte	Valide la structure par plages ; orientée transmission, sans sémantique ni API de requête
Classification RF par voxels [22]	grille régulière + 7 descripteurs/voxel	LiDAR aérien brut (297 M pts/km ²)	étiquettes sol/toit/mur/végétation, $F1 \approx 0,92$	Classification résolue en amont pour nous par l'IGN (myria3d [24] / FRACTAL [25])
Segmentation non supervisée par voxels [23]	grille de voxels dérivée d'un octree	nuages de points XYZ	étiquettes sémantiques via descripteurs de forme	Même conclusion : la géométrie sépare la canopée des bâtiments
LiVoxGen [26]	listes de voxels par colonne (C++)	LAS via liblas	métriques de canopée	Ancêtre direct ; dépendances abandonnées, kd-tree disproportionné, chaîne de compilation C++
Arches / SBRT [27]	grille 2-D de piles de (matériau, épaisseur)	règles procédurales	terrains avec surplombs, cavités	Héritage du laboratoire ; engendre ce que nous mesurons
VoxCity [28]	grille de voxels sémantique uniforme	rasters 2.5-D ouverts (hauteurs, occupation du sol)	modèle de ville + simulation solaire/de visibilité	Objectifs partagés ; dérivé de rasters, fidélité verticale plus grossière que le LiDAR
Notre approche	structure creuse en colonnes ; intervalles RLE par colonne en structure de tableaux numpy	LAZ classifié LiDAR HD de l'IGN (RGF93/CC46)	ColumnStore interrogeable ; cartes PNG ; statistiques ; visualiseurs 3-D ; 3D Tiles 1.1 ; aller-retour LAS/LAZ	indifférent au voisinage ; préserve la sémantique ; out-of-core à l'échelle métropolitaine ; Python pur

4 Proposition

Avant les mécanismes, la Figure 2 montre sur un transect réel de six mètres ce que la proposition produit et ce qu'elle sait répondre. Les algorithmes détaillés et le pseudo-code sont rassemblés en annexe A et référencés depuis le texte.

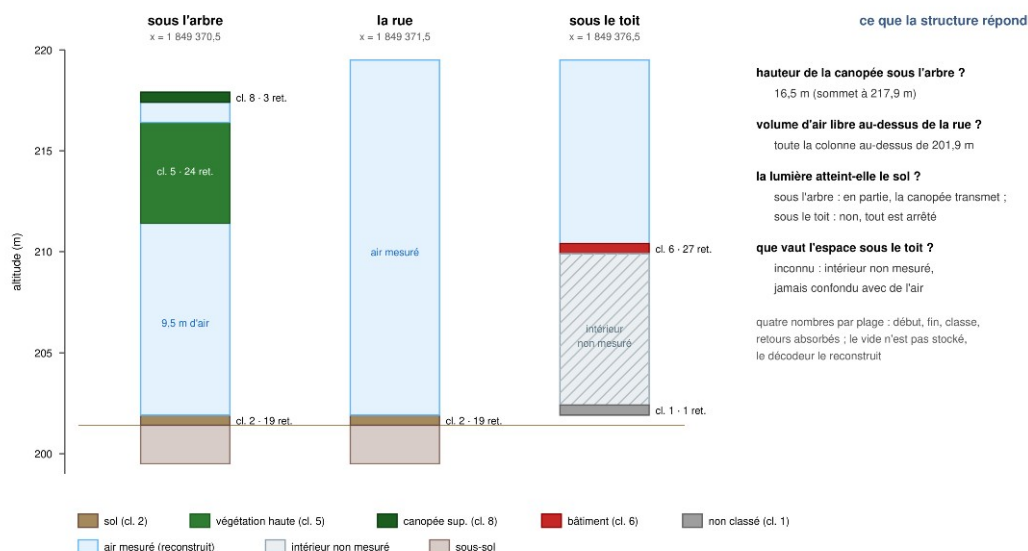


Figure 2 : Trois colonnes réelles d'un transect de 6 m (campus de Porte des Alpes, cellules de 0,5 m, y = 5 170 722) : sous l'arbre, la couronne (classes 5 et 8) surmonte 9,5 m d'air mesuré au-dessus du sol ; la rue est une unique plage de sol ; sous le toit (classe 6), l'espace sans retour de sol est un intérieur non mesuré. À droite, les réponses attendues de la structure.

4.1 Architecture

Le système est organisé en un unique paquet Python, voxelizer, dont les modules se regroupent en cinq niveaux de dépendance, du socle de services bas niveau au niveau supérieur des consommateurs CLI et GUI.

Au chargement, aucun module n'importe un module situé au-dessus de lui : la hiérarchie est sans cycle. Trois imports tardifs, exécutés seulement à l'appel d'une fonction, remontent la hiérarchie ; visibles dans la Figure 23, ils ne participent pas au chargement. Les chaînes d'import comptent jusqu'à dix liens de dépendance répartis sur onze niveaux (du niveau 0 au niveau 10), et la Figure 3 résume les cinq niveaux d'architecture avec les modules qui siègent dans chacun ; le graphe d'imports complet, généré depuis le code, est la Figure 23 en annexe D.

Cette organisation en niveaux présente trois bénéfices concrets. D'abord la testabilité : chaque niveau s'exerce contre les seuls niveaux inférieurs. Ensuite le confinement des défaillances : chaque étape de sortie (statistiques, cartes, diagnostics, export 3-D) ne dépend que de la structure persistante et s'exécute dans son propre processus, de sorte qu'un échec de rendu ne coûte jamais la voxelisation qui l'a précédé. Enfin la réutilisation : le même noyau sert cinq points d'entrée (CLI mono-tuile, CLI de zone, interface graphique, visualisation 3-D, export 3D Tiles) sans duplication. Chaque étape est ainsi isolée et rejouable séparément, en environnement local comme dans l'image Docker du projet, qui fige l'environnement d'exécution et rend les résultats reproductibles d'une machine à l'autre.

Le flux de données (Figure 4) est linéaire, de la lecture de la tuile LAZ au ColumnStore peuplé d'intervalles. Tout ce qui se trouve en aval ne consomme que la structure, seule interface entre la mesure

et l'analyse. Elle peut être rendue persistante (.npz compressé, ou répertoire brut projetable en mémoire) et rechargée sans re-voxeliser, si bien qu'une étape de sortie en échec ou améliorée n'exige jamais de retoucher les données d'entrée.

Deux dispositifs transversaux complètent l'architecture : un estimateur préalable, qui ne lit que les entêtes LAZ d'une zone et prédit la taille de la structure et le pic de mémoire avant qu'aucun point ne soit lu (la section 4.2.5 y revient), et un moniteur de diagnostics d'exécution qui enregistre en arrière-plan les traces système de n'importe quelle exécution, précieux pendant les investigations mémoire du chapitre 5. La reproductibilité est traitée à la couche la plus externe : répertoires d'exécution auto-incrémentés, scripts d'enveloppe et image Docker figeant l'environnement Python.

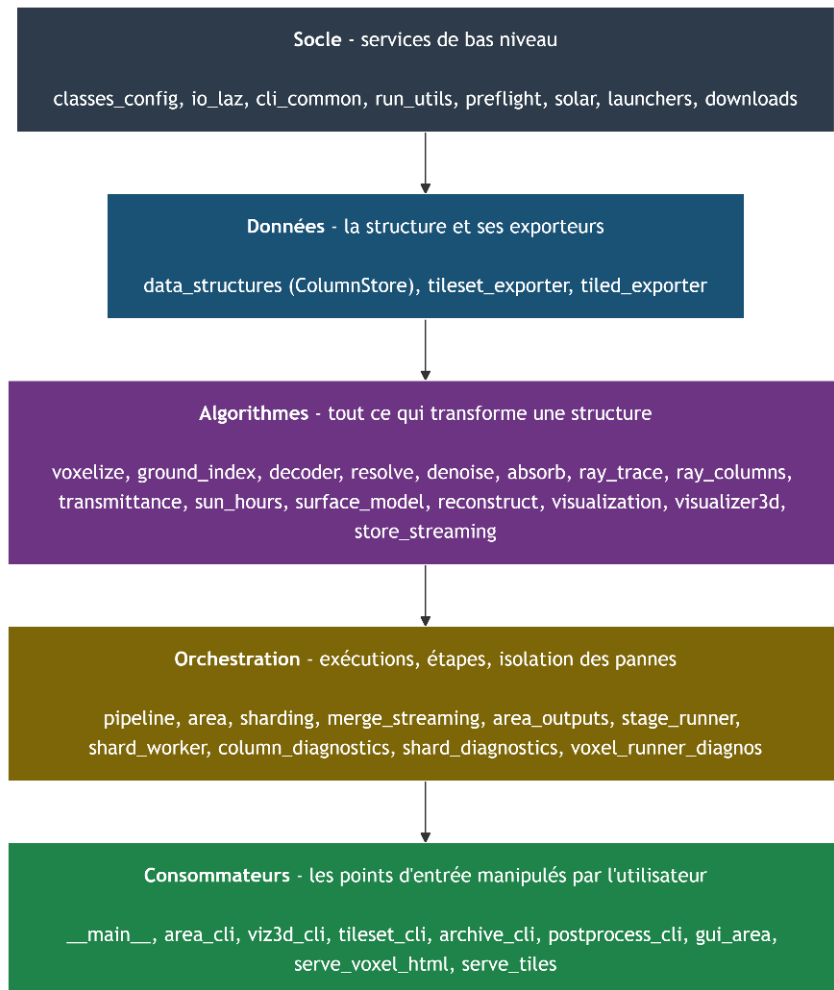


Figure 3 : Les cinq niveaux de dépendance du paquet voxelizer et les modules de chacun, générés depuis le graphe d'imports du code. Le graphe d'imports complet est la Figure 23 en annexe D.

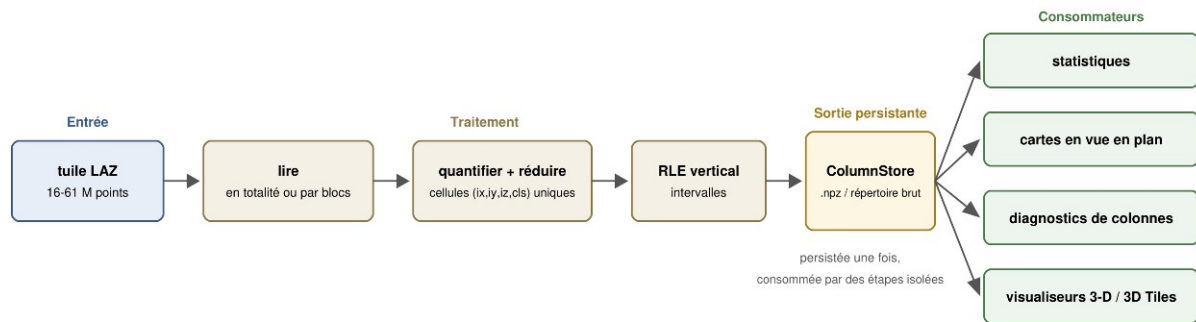


Figure 4 : Flux de données de la chaîne de traitement : de la tuile LAZ, via la quantification et le RLE vertical, jusqu'au ColumnStore persisté et à ses consommateurs.

4.2 Conception

4.2.1 La structure en colonnes et ses intervalles

Deux des contraintes de cadrage de la section 2.1 gouvernent la structure : indifférence au voisinage, chaque position (x, y) étant analysable sans ses voisins horizontales, et compression verticale par plages de matériau identique. La contribution du stage a été de formaliser, d'implémenter et de valider ce cadre. Le résultat est une grille creuse bidimensionnelle de profils verticaux : pour chaque colonne occupée (ix, iy) , une liste ordonnée d'intervalles $(z_start, z_end, class, count)$, où z_end est exclusif, $class$ est l'étiquette sémantique LiDAR HD de l'IGN, et $count$ est le nombre de retours LiDAR d'origine que l'intervalle absorbe. Un arbre de trente mètres devient à peu près deux intervalles, le sol en bas et une longue plage de végétation haute au-dessus, au lieu de milliers de points (Figure 5) ; le codage par plages exploite la continuité verticale que présente la matière urbaine.

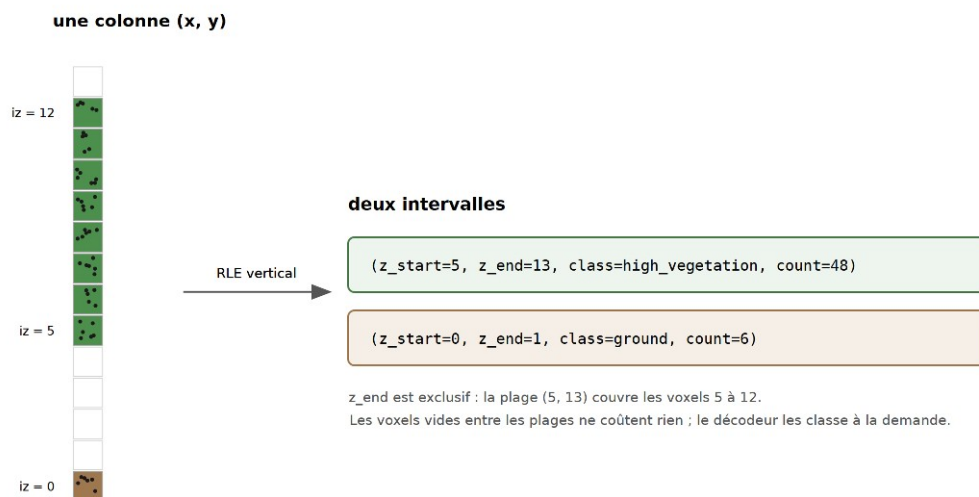


Figure 5 : Des points aux intervalles : une colonne et son codage par plages (z_end exclusif).

Deux propriétés de cet encodage méritent d'être signalées. Premièrement, la construction est conservatrice : si un voxel reçoit des retours de deux classes, les deux survivent comme cellules superposées. Le voxeliseur enregistre, il ne vote pas. Toute simplification avec perte (lissage, comblement de lacunes, réduction par majorité) est différée à des post-traitements explicites et validés séparément (section 4.2.4), de sorte que la structure brute préserve l'information des données du capteur. Deuxièmement, l'encodage est indifférent au voisinage par construction : rien dans un intervalle ne renvoie à une colonne adjacente, ce qui rend le calcul en aval trivialement parallèle, laisse les tuiles fusionner par concaténation et rend possible le mode *out-of-core* de la section 4.2.5. Le coût, des relations horizontales à recalculer au besoin, est acceptable ici : les requêtes du projet sont massivement verticales.

La compression ainsi obtenue est mesurée au chapitre 5 : sur la tuile la plus dense du corpus, la structure réduit les dizaines de millions de points à quelques millions d'intervalles, et l'annexe C en donne la comptabilité complète, classe par classe et grille par grille. Le nombre moyen d'intervalles par colonne est lui-même un diagnostic : les surfaces verticales, murs et troncs en particulier, fragmentent les colonnes en plages courtes, ce que les expériences d'encodeurs du chapitre 5 quantifient.

La façon de conserver ces intervalles en mémoire a été repensée une fois : la première implémentation, un dictionnaire d'objets Python conforme aux usages du langage, s'est révélée ruineuse à l'échelle. La disposition de production est une structure de tableaux numpy plats coûtant environ 16 octets par colonne plus 13 octets par intervalle, un gain mesuré d'un ordre de grandeur face à la forme objet. La mesure qui a tranché ce choix, et l'estimation erronée qu'elle a corrigée, sont consignées en annexe C.

Les bases de données en colonnes aboutissent à la même disposition pour la même raison [21] : quand chaque requête touche un champ de nombreux enregistrements, les tableaux supplantent les objets. Cette disposition rend aussi la persistance triviale (chaque tableau est un fichier dans la forme répertoire brut, projetable en mémoire pour la chaîne à étapes isolées) et maintient chaque algorithme en numpy vectorisé, au sens de NumPy : des opérations sur des tableaux entiers, sans aucune boucle Python par point ou par colonne sur le chemin de construction. La couche de requêtes de la section 4.4 est l'exception : le marcheur de rayons, l'algorithme qui avance le long d'un rayon voxel par voxel, progresse en Python, et la section 5.4 consigne ce que ce volume a coûté.

4.2.2 L'algorithme de voxelisation

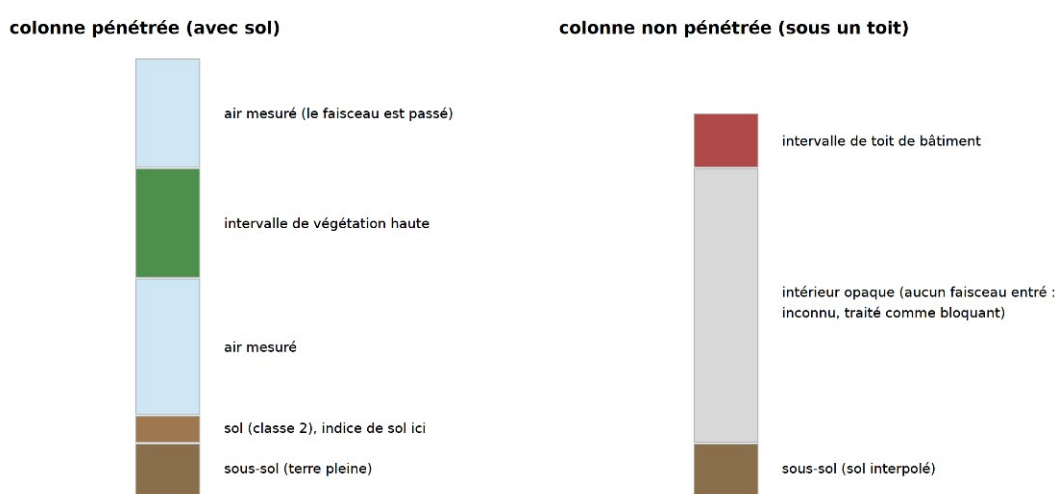
L'algorithme qui remplit la structure se déroule en deux étapes vectorisées, c'est-à-dire en opérations sur des tableaux entiers. L'étape une quantifie et réduit : les coordonnées de chaque point sont divisées par les tailles de cellule avec division entière (les deux tailles sont des paramètres ; chaque tableau du rapport indique la grille à laquelle il a été mesuré) pour donner des indices entiers, les tuples (ix, iy, iz, class) obtenus sont triés canoniquement, et les doublons se réduisent à des cellules uniques dont les comptages sont sommés. L'étape deux effectue le codage par plages : à l'intérieur de chaque colonne, des cellules consécutives à iz croissant et de même classe fusionnent en un intervalle ; une frontière d'intervalle survient exactement quand la colonne change, quand la classe change, ou quand iz saute. Les deux étapes sont en numpy pur (tri, différence, somme cumulée), si bien que le coût par tuile est dominé par la lecture et la décompression du LAZ lui-même.

Deux décisions d'implémentation façonnent le comportement à l'échelle : l'origine de la grille est fixée une fois par zone, de sorte que toutes les tuiles se quantifient contre la même origine et se concatènent sans rééchantillonnage, et la lecture existe en variante par blocs à mémoire bornée, vérifiée comme produisant des structures identiques à la voie fichier entier (suite de tests du chapitre 5) ; le détail est en annexe A.

4.2.3 Sémantique des espaces vides : l'indice de sol et le décodeur à trois états

Une structure de voxels dérivée de LiDAR aéroporté contient trois sortes de « vide », et les confondre corrompt les calculs physiques en aval. La Figure 6 les distingue : l'air mesuré, qu'un faisceau laser a réellement traversé ; l'intérieur non mesuré, typiquement sous un toit qui a arrêté tous les faisceaux, et qu'assimiler à de l'air laisserait la lumière simulée traverser les bâtiments ; et le sous-sol, la terre pleine déduite sous la surface, qui servira plus tard au volume d'enracinement.

Le décodeur attribue l'un de ces trois états à chaque voxel non occupé, ancré par une unique valeur par colonne : l'indice de sol, le sommet de l'intervalle de classe 2 le plus haut, les intervalles plus bas étant des erreurs de classification ou une surface pentue répartie sur deux voxels ; le cas du pont ne se présente pas dans cette livraison (annexe E). Les colonnes sans retour de sol reçoivent un intérieur opaque par défaut, l'« intérieur non mesuré » de la Figure 2, une zone inconnue n'étant pas assimilée à de l'air, et une interpolation optionnelle restaure une estimation du terrain sous les toits. Les cas limites et les règles précises sont en annexe A.



Décodeur à trois états : un vide ne devient de l'air mesuré que là où un retour de sol prouve qu'un faisceau a traversé la colonne.

Figure 6 : Le décodeur d'espaces vides à trois états, ancré par l'indice de sol de chaque colonne.

4.2.4 Raffinement sémantique : variantes d'encodeur et post-traitements

La structure brute est complète mais bruitée : voxels isolés mal classifiés, trous d'un voxel dans les canopées, colonnes déchiquetées par les retours sur les murs. Le raffinement est traité comme une famille de transformations explicites, chacune incarnant une hypothèse sur l'information qui compte, chacune mesurée contre la référence brute au chapitre 5 : la variante v1 (class smooth) lisse le grain isolé par fenêtre de majorité, v2 (gap fill) comble les lacunes courtes entre intervalles de même classe, sans perte pour les produits en vue en plan, v3 (minor class drop) supprime les intervalles marginaux, et v4 (majority above five metres) réduit tout ce qui dépasse cinq mètres à la classe dominante de la bande. La Figure 7 montre ce que chaque variante réécrit dans une colonne ; les mécaniques exactes sont en annexe A.

v4 est la formalisation littérale d'une suggestion de réunion d'encadrement (« trente points de bâtiment et cinq points de végétation : appelons cela du bâtiment »), incluse pour que son coût puisse être mesuré ; le chapitre 5 mesure ce coût, substantiel sur l'empreinte de végétation en vue en plan, ce pourquoi v4 est réservée à la visualisation et exclue de l'analyse écologique.

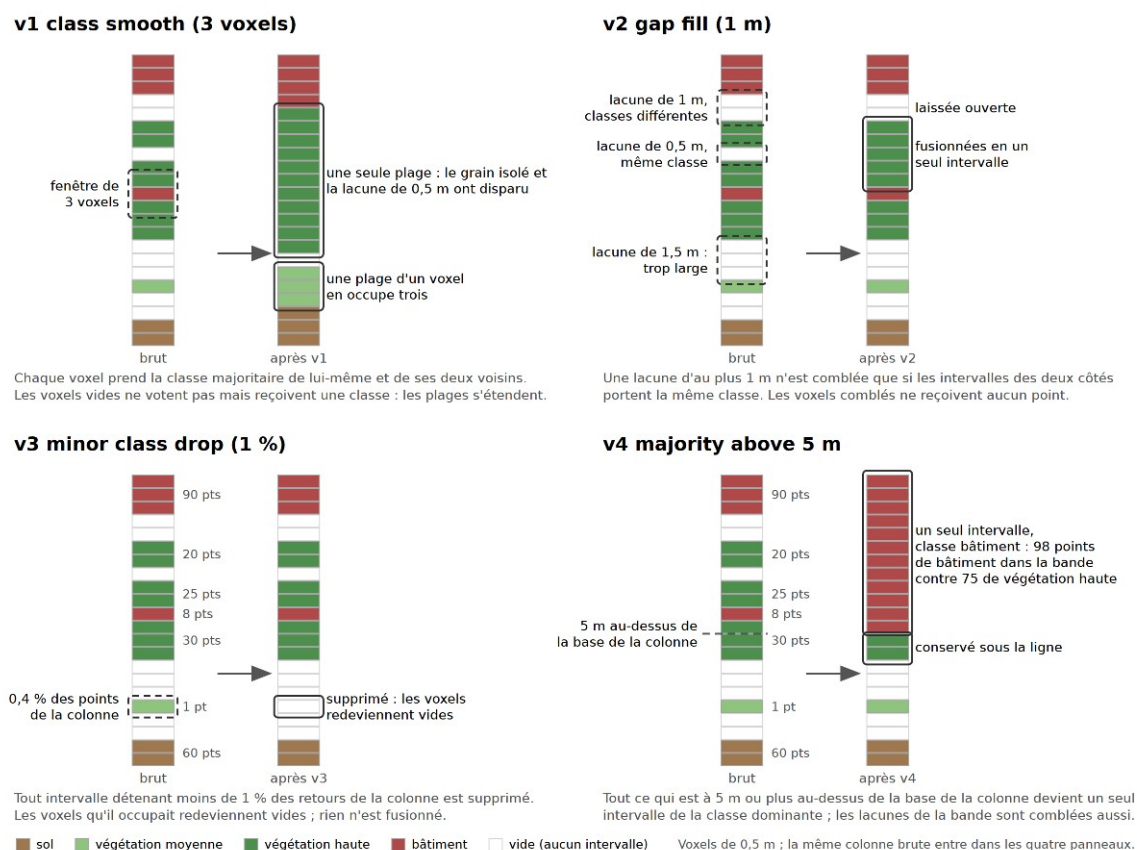


Figure 7 : Les quatre variantes de simplification de la section 4.2.4 : ce que chacune réécrit dans une colonne, montré sur la même pile d'intervalles en voxels de 0,5 m. Les contours en tirets marquent ce qui déclenche la règle, les contours pleins ce qu'elle a changé.

Trois post-traitements autonomes complètent ces variantes (un vote qui résout les recouvrements, une absorption de la végétation enfermée dans du bâti, un débruitage) : des opérations par lots qui consultent temporairement des voisinages puis rendent une structure de nouveau indifférente au voisinage ; leurs règles sont en annexe A et leurs coûts mesurés au chapitre 5 et en annexe C. Une autre idée des notes de conception, des rasters de sommet de surface par classe, a été envisagée puis écartée : un plus joli produit, rien pour l'objectif de plantabilité.

4.2.5 Conception du passage à l'échelle : des zones métropolitaines sur la mémoire courante

Le dernier axe de conception est la prise en compte de l'échelle, et son principe directeur est qu'aucune étape ne doit jamais exiger la ville tout entière en mémoire.

Le traitement par zone sélectionne les tuiles intersectant la boîte demandée en ne lisant que les en-têtes, voxelise chacune contre l'origine de grille commune et fusionne les structures par concaténation, peu coûteuse précisément parce que les colonnes sont indépendantes. Quand la structure fusionnée elle-même dépasserait la mémoire, le mode partitionné traite chaque tuile vers un fragment indépendant et agrège en flux : le pic de mémoire reste celui d'une seule tuile quelle que soit la zone. La fusion complète reste possible à cette échelle, car elle est elle-même out-of-core, assemblée sur le disque une bande de clés à la fois, avec un résultat démontré identique à la fusion en mémoire (section 5.1) ; une variante en flux, enfin, télécharge, voxelise et supprime chaque tuile pour les machines à faible espace disque.

L'estimateur préalable de la section 4.1 rend ces modes utilisables par des non-spécialistes : depuis les seuls en-têtes, il prédit si l'exécution demandée tient, en retenant la plus prudente de deux estimations (surface couverte, nombre de points des en-têtes), et propose les alternatives grossir-ou-partitionner avec des chiffres concrets avant que le moindre travail ne commence. Ses constantes sont calibrées sur la mesure et validées sur des cas tenus hors de l'ajustement, l'estimation restant prudente sans jamais sous-estimer ; la calibration détaillée est en annexe C, et l'autopsie du chapitre 5 montre ce qui a été repensé quand une première voie de visualisation a violé le principe directeur de cette section (Figure 8).

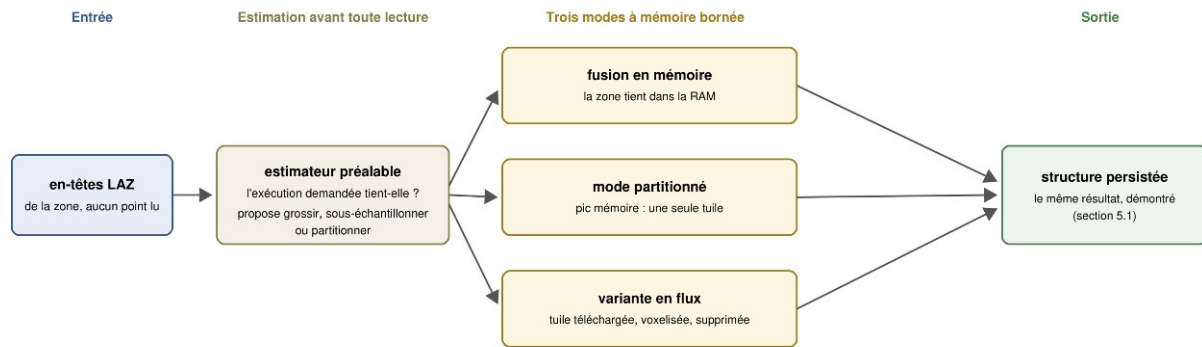


Figure 8 : Le passage à l'échelle en un schéma : l'estimateur préalable ne lit que les en-têtes et choisit, chiffres à l'appui, entre trois modes d'exécution à mémoire bornée ; tous aboutissent à la même structure persistée.

4.3 Implémentation

4.3.1 Langage et dépendances

Le langage d'implémentation a lui-même fait l'objet d'une décision argumentée. L'outil prédécesseur était en C++, et une réécriture en C++ aurait été le choix réflexe pour un voxeliseur sensible aux performances.

Nous avons choisi uniquement Python pour trois raisons. Premièrement, le chemin critique n'est pas du Python : chaque opération par point s'exécute à l'intérieur des noyaux compilés de numpy [31], et le profilage montre que le coût par tuile est dominé par la décompression LAZ, si bien qu'un portage C++ n'apporterait que peu là où cela compte. Deuxièmement, la reproductibilité : la plateforme IA.rbre privilégie les chaînes de traitement que des collègues peuvent installer avec pip et lire sans système de compilation ; l'empreinte en dépendances est une courte liste installable par pip (numpy, laspy [32] avec le moteur LASzip [6], matplotlib, Pillow, psutil, pyproj, pygltf, plus deux extras optionnels), dont l'annexe B détaille le rôle de chacune. Troisièmement, la continuité : tout composant IA.rbre en aval susceptible de consommer la structure est en Python.

Le seul endroit où ce choix nous coûte, les boucles par élément qui résistent à la vectorisation, a été supprimé par conception plutôt que par optimisation : les algorithmes de la section 4.2 sont formulés comme des transformations de tableaux précisément pour qu'aucune boucle de ce type n'existe.

4.3.2 Internes de la structure et persistance

La structure de tableaux de la section 4.2.1 est réalisée par six vecteurs numpy plats (les internes, empaquetage des clés et constantes de quantification, sont détaillés en annexe B). La persistance a deux formes aux rôles distincts : une archive .npz compressée pour l'échange, et un répertoire brut, un fichier

par tableau, pour la chaîne elle-même. La forme brute existe à cause de l'isolation des étapes : les processus fils projettent les tableaux en mémoire en lecture seule, s'exécutent sans copie, et une étape défaillante laisse la structure intacte sur le disque.

4.3.3 Étapes de sortie

La chaîne par tuile émet quatre cartes PNG en vue en plan, une par question fréquente : la hauteur maximale (la silhouette urbaine), la classe dominante, la classe sur fond d'orthophoto optionnel (affichage seulement, jamais une entrée du calcul), et le nombre d'intervalles par colonne (une carte de fragmentation, diagnostic de qualité des données). Une étape de statistiques écrit les comptages de points et d'intervalles par classe avec les taux de compression. Une étape de diagnostics de colonnes rend les colonnes les plus complexes de la structure en coupes verticales, un histogramme du nombre d'intervalles par colonne, et des panneaux par classe ; ces figures ont été le principal instrument de débogage d'un bout à l'autre, car un algorithme erroné se manifeste immédiatement par un profil de colonne invraisemblable. Les exécutions par zone produisent les mêmes livrables assemblés entre tuiles, sous le régime de mémoire bornée de la section 4.2.5.

4.3.4 Visualisation tridimensionnelle

La visualisation a été développée comme instrument de contrôle : voir la structure en trois dimensions est le moyen le plus direct de se convaincre qu'un résultat est plausible avant de le mesurer. Trois voies complémentaires la fournissent : un visualiseur HTML autonome qui s'ouvre dans un navigateur sans installation, des exports PLY et PyVista pour les outils d'inspection standards, et un visualiseur en flux qui diffuse la structure par tuiles autour de la caméra sous un budget GPU fixe, une zone à résolution fine portant des centaines de millions d'intervalles qu'aucun navigateur ne charge en une seule scène (la Figure 16, au chapitre 5, le montre en navigation). Un quatrième exportateur convertit la structure en tileset 3D Tiles 1.1 [9] pour les clients géospatiaux standards ; la sortie passe le validateur officiel sans erreur, sous les réserves consignées en section 5.5. Les mécanismes internes des quatre voies sont décrits en annexe B, et les Figures 18 à 20, en annexe D, en donnent les schémas.

4.3.5 Mécanisme de requête : le lanceur de rayons

L'aspect côté analyse implémenté au-dessus de la structure est un marcheur de rayons à la manière d'Amanatides et Woo [34] : un analyseur différentiel numérique tridimensionnel qui maintient la distance paramétrique jusqu'à la prochaine frontière de voxel sur chaque axe et avance celle qui est la plus proche, traversant la structure voxel par voxel suivant la direction du rayon. Le décodeur de la section 4.2.3 fournit la sémantique des voxels vides qu'il traverse, et un objet de configuration déclare quels états comptent comme solides. Le même marcheur sert donc la projection d'ombres (l'intérieur opaque et les voxels occupés bloquent), la visibilité (seuls les voxels occupés bloquent) et les futures requêtes de plantabilité. Le marcheur renvoie la première intersection solide avec sa position, sa classe et le paramètre du rayon. Il est gardé simple parce qu'il est l'implémentation de référence : la section 4.4 bâtit sur lui la machinerie de projection d'ombres – une borne exacte de terminaison anticipée, une transmittance sensible aux classes et la géométrie solaire – et teste chaque optimisation contre ses résultats.

4.3.6 Interfaces et reproductibilité

Le système expose trois niveaux d'interface, reflétant trois publics. Pour les collègues novices en technique, une interface Tkinter offre un mode fichier unique et un mode zone par coordonnées, précédés du dialogue préalable de la section 4.1 (tuiles et coût mémoire prédits ; poursuivre, sous-échantillonner ou partitionner). Pour l'utilisateur qui scripte ses traitements, la CLI expose les points d'entrée tuile unique, zone, visualisation, tileset et reconstruction, avec des options factorisées pour que

les outils ne dérivent pas l'un de l'autre ; la même chaîne s'exécute sous Windows, sous Linux et dans l'image Docker du projet.

Pour l'auditeur muni d'outils standards de nuages de points, un voxeliseur inverse reconstruit un nuage de points LAS/LAZ depuis une structure, soit un point par retour d'origine en utilisant les comptages stockés, soit un point par voxel, ce qui permet à n'importe quel outil standard de nuages de points d'examiner ce que la structure contient réellement et referme la boucle pour la validation (le chapitre 5 l'utilise). Chaque exécution écrit dans un répertoire d'exécution auto-incrémenté, et une image Docker fige l'environnement ; l'objectif de conception est qu'un successeur puisse reproduire n'importe quelle figure de ce rapport à partir du dépôt et d'une liste de tuiles.

4.3.7 Interopérabilité : la structure comme LAS/LAZ ordinaire

Un ColumnStore sur disque n'est lisible que par ce dépôt, ce qui est acceptable pour un produit intermédiaire et inacceptable pour un livrable. La voie de reconstruction a donc été complétée en un inverse complet : voxelizer/reconstruct.py réécrit une structure en LAS ou LAZ, le format même dans lequel arrivent les mesures, et relit un tel fichier vers une structure ; tout outil qui lit du LAS/LAZ lit donc directement le contenu de la structure, et une exécution partitionnée entière se convertit dans les deux sens pour l'archivage. Trois modes d'export existent parce que trois choses méritent d'être préservées : la densité de points (density), la géométrie des voxels (one_per_voxel), ou les deux (exact). L'exactitude du mode exact est une affirmation littérale, épinglée par test : re-voxeliser un export exact reproduit la structure source octet pour octet, vérifié par égalité de SHA-256, et cinquante tests couvrent la garantie et ses protections, y compris les cas négatifs.

Deux décisions de conception soutiennent la garantie : la grille voyage à l'intérieur du fichier (un enregistrement propriétaire de 52 octets portant l'origine, les tailles de cellule et le code EPSG, car une origine devinée depuis le nuage tombe à une demi-cellule près), et seules les structures brutes sont archivées, le regroupement pouvant rendre un intervalle inconvertible sans que rien ne le distingue. Le coût a été mesuré sur trois tuiles réelles à trois résolutions (Tableau 10) : aux résolutions fines, l'archive est plus petite que la structure qu'elle remplace ; à 0,5 m les deux sont proches de la parité ; à 1 m le cas doit être mesuré exécution par exécution. Le détail des modes, des protections et de l'étude de coût est en annexe C.

Une limitation borne tout cela. Un LAZ exact est un conteneur sans perte de la structure sur la grille où elle a été construite, non un remplacement des tuiles sources : chaque point qu'il contient se tient à un centre de voxel, si bien qu'il se re-voxelise sans perte vers tout multiple entier plus grossier et jamais vers une grille plus fine, et les champs d'intensité, de couleur, de retour et de datation ont été écartés avant que le premier voxel n'existe. Conserver l'archive au lieu des tuiles de l'IGN est un pari sur le fait que la géométrie de la grille reste fixe.

4.3.8 Obstacles importants à consigner

Trois obstacles d'implémentation ont façonné le code plus que n'importe quelle demande de fonctionnalité. La mémoire, d'abord et toujours : le collecteur de géométrie naïf matérialisait des tuples Python par colonne, ce qui est invisible sur une seule tuile et fatal sur cent ; le correctif, relaté au chapitre 5, a consisté à maintenir chaque voie de traitement massif à l'intérieur de numpy de bout en bout. Deuxièmement, la fragilité de l'environnement sur la plateforme cible : du bytecode compilé périmé a produit un jour une erreur qu'aucune lecture des sources ne pouvait expliquer ; le chapitre 5 consigne l'épisode, et la leçon est de faire moins confiance à l'environnement de compilation qu'au code. Troisièmement, le millésime du jeu de données : les premières expériences tournaient sur l'acquisition de 2018, livrée en tuiles de 1 km en Lambert-93 (EPSG:2154), et des commentaires écrits alors ont survécu au passage à la livraison Grand Lyon 2023 de tuiles de 500 x 500 m nommées par hectomètres en RGF93/CC46 (EPSG:3946) ; le résidu a été éliminé au regard de la source faisant autorité, les en-têtes LAZ eux-mêmes, et le rapport n'énonce que des faits vérifiés sur en-têtes.

4.4 Requêtes par lancer de rayons : de l'occlusion à la transmittance sensible aux classes

4.4.1 Ce que la structure rend « bon marché », et ce qu'elle a rendu coûteux

La structure en colonnes répond directement aux questions verticales, celles dont une requête par lancer de rayons a besoin. Un rayon entrant dans une colonne rencontre une liste ordonnée de matériaux compressée par plages, si bien que la question « que rencontre ce rayon, et dans quel ordre » trouve réponse sur la représentation compressée sans la développer.

De cette référence (voxelizer/ray_trace.py, section 4.3.5), un point de vocabulaire et un problème de coût la séparaient encore de l'objectif initial (iv).

« 2.5-D » désigne deux choses aux conséquences opposées : un modèle de données (un champ de hauteurs, une valeur par cellule, incapable de représenter un surplomb ou une canopée au-dessus d'un sol dégagé) et un simple schéma d'accès sur une structure tridimensionnelle, qui n'est qu'une optimisation. La structure en colonnes est un modèle tridimensionnel, les intervalles multiples par colonne étant précisément les surplombs qu'un champ de hauteurs ne peut exprimer ; la section 4.4.7 s'appuie sur cette distinction.

Le coût, lui, variait avec la grille plutôt qu'avec la géométrie : un rayon d'échappement marche voxel par voxel à travers l'air vide longtemps après avoir tout dépassé, au point de rendre l'expérience d'ombrage infaisable aux résolutions fines. Cette limitation, et non une idée manquante, explique qu'elle n'avait pas été menée (ordres de grandeur au Tableau 11, annexe C).

Une réserve de mesure accompagne ces requêtes : la règle d'air du décodeur suppose un faisceau nadiral, alors que les angles de balayage de la livraison couvrent ± 30 degrés (Figure 9). L'argument selon lequel le recouvrement des bandes rendrait les retours de sol quasi nadiraux a été éprouvé sur les données et ne tient pas ; l'atténuation implémentée corrige la requête sans toucher à la structure, et le chapitre 5 consigne l'erreur résiduelle comme une limitation (mesures complètes en annexe A).

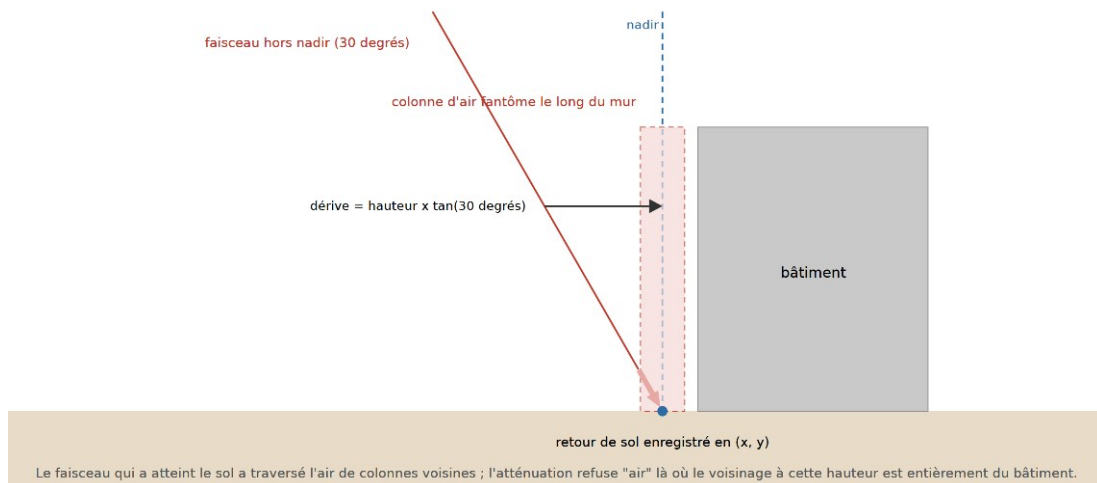


Figure 9 : La réserve du faisceau nadiral : les faisceaux hors nadir attribuent l'air traversé à une colonne voisine ; la vérification de voisinage du décodeur supprime la colonne fantôme.

4.4.2 Une borne de plafond exacte

Le correctif n'approxime pas : au-dessus du voxel occupé le plus haut d'une colonne et de ses quatre voisines, tout vide est démontrablement de l'air, et un rayon montant qui franchit ce plafond peut être

terminé immédiatement. La preuve est un théorème sur le décodeur, l'implémentation (CeilingDDA) laisse le marcheur de référence intact, et l'effet est décisif, ramenant l'arc solaire journalier de l'ordre de la quinzaine de jours à l'ordre de l'heure (preuve et tests en annexe A, coûts au Tableau 12 en annexe C, Figure 21 en annexe D) ; un rayon oblique, qui quitte sa colonne, se contente du plafond global, plus faible, dont la section 4.4.6 donne le coût.

4.4.3 Transmittance sensible aux classes

L'objectif (iv) implique un lancer de rayons qui tienne compte des « interactions entre voxels de classes territoriales différentes ». Une requête de première intersection ne peut pas exprimer cela, parce qu'elle est binaire : le marcheur de référence s'arrête au premier voxel occupé quelle que soit sa classe, si bien qu'une feuille arrête un rayon comme le fait un mur de béton. La question est de savoir si une canopée laisse passer un peu de lumière, et un booléen ne peut pas y répondre.

voxelizer/transmittance.py remplace le booléen par une transmittance dans $[0, 1]$ accumulée le long du rayon par la loi de Beer-Lambert [35], dans la tradition de l'extinction lumineuse dans les couverts de Monsi et Saeki [36] et de Campbell et Norman [37]. Deux familles plus riches ont été envisagées puis rejetées : les modèles complets de transfert radiatif tels que SAIL [38] et les modèles géométriques-optiques [39] exigent des distributions d'angles foliaires et des géométries explicites que le LiDAR ne fournit pas. La longueur de trajet est comptée en mètres vrais et non en voxels, afin que la réponse dépende de la géométrie et non de la grille ; cette invariance en résolution est vérifiée par test (détail en annexe A).

$$\tau = \exp(-\sum_c k_c L_c)$$

où L_c est la longueur de trajet en mètres à travers la classe c et k_c est le coefficient d'extinction de cette classe, en m^{-1} . Les matériaux opaques (bâtiment, sol, eau, ainsi que les états intérieur opaque et sous-sol du décodeur) prennent un coefficient infini et terminent la marche à zéro.

Les coefficients ne sont pas fournis par défaut : un coefficient inventé ferait de chaque chiffre en aval une hypothèse accompagnée d'une décimale. Leur dérivation depuis le corpus est l'objet de la section 4.4.5, et tout chiffre produit avec un coefficient provisoire est étiqueté comme tel.

Le comportement mesuré de cette machinerie, et ce qu'il faut ou non y lire, est rapporté au chapitre 5 avec la Figure 10.

4.4.4 Géométrie solaire, et pourquoi cet algorithme

Projeter une ombre demande d'avoir la direction du soleil. L'implémentation (voxelizer/solar.py) suit la formulation du NOAA Solar Calculator [41], l'algorithme à faible précision de Meeus [42] : sa précision dépasse déjà largement la quantification de la grille, les solutions plus précises (NREL SPA [43]) ou plus simples (PSA [44], Michalsky [45]) ont été écartées, et aucune bibliothèque solaire n'a été ajoutée à l'environnement (comparatif en annexe C). La validation se fait contre des faits indépendants du code, l'astronomie sphérique et les extrêmes publiés de l'équation du temps (Tableau 14, annexe C). Un piège réel mérite d'être signalé : l'azimut solaire se mesure depuis le nord alors que les axes de la grille sont projetés ; sun_vector_grid convertit explicitement la convergence des méridiens, et une inversion des branches d'azimut, invisible pour tout test d'élévation, a été attrapée par la seule vérification est/ouest.

4.4.5 Coefficients d'extinction dérivés du corpus

Les coefficients laissés non fournis par la section 4.4.3 sont dérivés du corpus lui-même : le LiDAR mesure directement la pénétration de la canopée, et pour chaque colonne portant de la végétation au-dessus d'un retour de sol, la fraction p d'impulsions atteignant le sol et l'épaisseur de canopée H donnent

$k = -\ln(p) / H$. L'ordre obtenu est physiquement sensé sans avoir été imposé (valeurs et décroissance monotone au Tableau 3).

Tableau 3 : Coefficient d'extinction par classe de végétation dominante, 0,5 m au cube, à partir de la fraction de sol par impulsion.

Classe	<i>n</i>	<i>k</i> médian (1/m)	p10	p90
3 végétation basse	7 147	0,497	0,178	1,198
4 végétation moyenne	103 330	0,429	0,212	0,942
5 végétation haute	396 145	0,326	0,176	0,656
8 canopée supérieure	233 294	0,250	0,141	0,501
toutes	739 916	0,311	0,163	0,666

Quatre limitations, détaillées en annexe C, empêchent d'en faire un résultat abouti : *k* est une distribution et non une propriété par colonne ; l'estimation dépend de la résolution ; *p*, d'abord estimée par comptages de retours, a dû être reconstituée impulsion par impulsion ; et le coefficient, dérivé d'un relevé proche du nadir, est appliqué sans correction angulaire aux rayons obliques. La Figure 22 (annexe D) montre les distributions, et la dérivation est reproductible depuis le dépôt.

4.4.6 Heures d'ensoleillement direct : l'expérience

Pour chaque colonne et chaque position solaire au-dessus de l'horizon, un rayon part de 1,5 m au-dessus du sommet de l'intervalle le plus bas de la colonne, la hauteur des yeux là où un sol est mesuré et le niveau du toit là où il ne l'est pas, en direction du soleil, et la transmittance s'accumule en heures d'ensoleillement. Le calcul est mené deux fois, tous matériaux opaques puis avec les coefficients de la section 4.4.5 : la différence isole ce qu'apporte le modèle volumétrique des matériaux. Les règles de bord exactes sont en annexe C, et les résultats mesurés au chapitre 5.

Ces exécutions portent la correction du plafond oblique, qui a abaissé toutes les valeurs absolues sans en inverser aucune (détail et coûts en annexe C).

Périmètre de l'expérience : le faisceau direct seulement, sans composante diffuse du ciel, réflexion, atténuation atmosphérique ni nuage ; un énoncé géométrique sur la ville, non un énoncé radiométrique sur l'énergie, dont la conversion exigerait des termes que la géométrie LiDAR ne fournit pas. Les coefficients ne sont pas réglés non plus : le biais par comptages de retours a été corrigé, la dépendance à la résolution demeure, et ce calcul, mené à 1,0 m avec les coefficients les plus faibles, rapporterait un gain plus petit sur une structure à 0,5 m, d'un montant non mesuré (chiffres en annexe C).

Au-delà des vérifications solaires, la géométrie des ombres est validée analytiquement contre la forme fermée de l'ombre d'une tour sur terrain plat : la frontière calculée s'accorde à 0,36 m près, moins d'une cellule horizontale (détail en annexe A).

Une défaillance native (violation d'accès Windows) a frappé la première exécution complète ; le diagnostic et les correctifs, qui ont à peu près divisé le temps d'exécution par deux à résultats prouvés identiques, sont consignés en section 5.4 et en annexe C.

4.4.7 Comparaison aux approches 2-D et 2.5-D

La comparaison aux approches 2-D et 2.5-D (objectif v) a été menée, sans aucun jeu de données externe : quatre modèles comparés sont construits à partir d'une même structure en colonnes, de sorte que la représentation soit la seule variable. Le modèle V est la structure elle-même, lue en opaque puis sous la transmittance de la section 4.4.3 ; le modèle S, un modèle de surface 2.5-D qui réduit chaque colonne à un corps unique ; le modèle A, un jeu de rasters 2-D reproduisant le modèle de données du module de végétalisation voisin (section 3.3) ; le modèle S2, un contrôle qui ne change que la hauteur du point d'échantillonnage, séparant celle-ci du modèle d'occlusion. Le matériau est de dix kilomètres carrés stratifiés du Grand Lyon à deux grilles, et les modèles sont vérifiés sur quatre scènes de synthèse, trois où ils doivent s'accorder et une où ils doivent diverger (détail en annexe C).

Les résultats mesurés de cette comparaison, et la part de la divergence qui relève de la grille plutôt que de la représentation, sont rapportés au chapitre 5.

Deux mesures restent dues et sont énoncées comme telles : une figure de l'occlusion en fonction de la résolution avec l'origine des rayons définie relativement à la surface du sol, et une passe d'heures d'ensoleillement à l'échelle de la Métropole sous l'isolation de processus par bloc de la chaîne de traitement.

5 Résultats

Ce chapitre suit le plan des encadrants pour les résultats : ce qui a été testé (5.1), comment la conception a été validée quantitativement (5.2), ce que le système produit (5.3). Ensuite seront ouvertes une discussion des limitations et des échecs (5.4), et une évaluation d'ensemble au regard des objectifs du chapitre 2 (5.5).

5.1 Tests

La suite de tests a grandi avec le système et couvre chaque module algorithmique isolément ainsi que les propriétés qui tiennent la chaîne de traitement ensemble. Des tests par module exercent successivement le décodeur, l'indice de sol, le résolveur de recouvrements, la passe d'absorption, les filtres de débruitage, le lanceur de rayons, le marcheur de rayons borné par le plafond, l'accumulateur de transmittance, la géométrie solaire, le codec de grille, l'exportateur de tileset, l'isolation des étapes et la persistance des zones.

Trois propriétés d'équivalence sont plus porteuses que tout test unitaire isolé, car elles autorisent la souplesse de l'architecture.

D'abord, la lecture par blocs égale la lecture du fichier entier : voxeliser une tuile diffusée en blocs à mémoire bornée produit une structure identique à celle obtenue en une seule passe, ce qui permet à la chaîne de choisir sa stratégie de lecture sur les seules considérations de mémoire.

Ensuite, le traitement par zone égale l'union découpée : traiter une zone tuile par tuile contre l'origine commune produit la même structure que voxeliser l'union des entrées découpées, ce qui rend les modes fusionné et partitionné interchangeables quant au sens.

Cette égalité a été confrontée à l'échelle de production. Une boîte d'un kilomètre carré, quatre tuiles et 52 705 942 points, a été voxelisée deux fois à 0,5 m au cube, une fois en une seule zone en mémoire et une fois en quatre fragments fusionnés hors mémoire : les deux voies donnent 3 995 579 colonnes et 5 593 880 intervalles, des clés de colonne égales élément par élément, et aucune colonne divergente. La conservation des comptages de points, vérifiée sur chacune des quarante structures de l'annexe C, tient partout : la somme des comptages des seize fragments égale, à l'unité près, le total de la structure fusionnée.

La comparaison a aussi révélé un défaut de forme depuis corrigé : la voie directe écrivait les comptages en int64 contre l'int32 que la classe documente, avec des valeurs toujours identiques, si bien que le chargement ramenait déjà le tableau au type déclaré. Le défaut corrigé, la voie directe écrit 14,1 % de tableau brut en moins, et la voxelisation directe reprise sur les mêmes tuiles produit une archive dont les sept membres sont identiques octet pour octet (SHA-256) à ceux de la voie fusionnée.

Enfin, les exécutions interrompues échouent proprement : une exécution arrêtée par le chien de garde mémoire laisse des sorties partielles dans un état bien défini ; les fichiers ne sont pas corrompus.

Les instantanés de campagne enregistrés pendant le développement (30 sur 30, 32 sur 32, 41 sur 41, puis 14 sur 14 et 19 sur 19 vérifications CLI de bout en bout) sont consignés en annexe C ; le balayage en résolution de la section 5.2 fait office de campagne de non-régression finale, huit exécutions complètes à cinq étapes de sortie isolées, les quarante exécutions d'étapes réussies.

Le paquet est couvert par une suite de tests automatisés étendue, collectée par pytest sous tests/, aux côtés de quatre suites en forme de scripts exécutées séparément et d'un module conditionné à la plateforme, ignoré sous Windows. Les sous-ensembles cités ailleurs dans ce rapport sont à l'intérieur de cette suite : dix tests pour la borne de plafond, quinze pour la transmittance, quatorze pour la géométrie solaire, et cinquante répartis sur les deux fichiers d'archive et d'aller-retour.

Le voxeliseur inverse referme la boucle depuis l'extérieur : reconstruire un nuage de points LAS/LAZ depuis une structure et l'auditer avec des outils standards confirme que les positions et les classes survivent à la conversion, à la quantification près. Cette boucle a ensuite été refermée exactement : le mode d'export exact de la section 4.3.7 convertit une structure en LAS/LAZ et retour avec égalité de SHA-256 de la structure ré-enregistrée, vérifiée sur neuf combinaisons réelles de tuile et de résolution et épinglée par les tests d'archive et d'aller-retour couvrant la garantie et chacune des protections qui l'entourent.

5.2 Validation

La campagne de validation quantitative traite l'affirmation centrale de la conception : que des colonnes compressées par plages compriment fortement le LiDAR urbain tout en préservant la sémantique par classe. La zone de test comprend trois tuiles de 500 m dans l'est lyonnais (Villeurbanne et le périphérique : les tuiles 18410_51755, 18410_51760 et 18415_51760, ensemble 74,3 millions de points couvrant le sol, les trois étages de végétation, le bâtiment, l'étage supérieur de canopée de classe 8, les retours non classés et l'eau, cette dernière présente dans deux des trois tuiles). La référence brute et les quatre variantes d'encodeur de la section 4.2.4 ont été appliquées à deux résolutions, et chaque résultat a été évalué selon quatre critères : taille de la charge utile des intervalles sérialisés, octets par point d'origine, taux de désaccord sur la classe supérieure entre la structure brute et la structure encodée (le pourcentage de colonnes dont la classe la plus haute a changé), et recouvrement de Jaccard [46] par classe des empreintes en plan (classe dominante par colonne) pour le bâtiment, la végétation et le sol (1,000 signifie que la structure encodée occupe exactement les voxels bruts de cette classe). Le Tableau 4, ci-dessous, donne les résultats aux deux résolutions.

Quatre conclusions en découlent, et chacune correspond à une recommandation.

Premièrement, le codage par plages brut à lui seul est une forte réduction, mais le facteur dépend entièrement de la référence, et les deux sont donc donnés. Les 74 265 365 points d'entrée se sérialisent en 37,54 MB à la résolution grossière, soit 0,51 octet par point. Face à des enregistrements de points LAS 1.2 non compressés (28 octets chacun, soit 2 079 MB), c'est une réduction d'un facteur 55,4 ; face aux tuiles telles qu'effectivement livrées par l'IGN, déjà compressées en LAZ et totalisant 312,2 MB pour ces trois tuiles, c'est un facteur 8,3. Le second est le chiffre défendable pour un argument de stockage, puisque personne ne stocke du LAS non compressé ; le premier reste le bon pour un argument de contenu informationnel.

Deuxièmement, gap fill (v2) supprime environ 30 % des intervalles à la résolution grossière et la campagne n'enregistre aucun changement sémantique (0,000 % de désaccord sur la classe supérieure, tous les Jaccard à 1,000). Les deux métriques y sont structurellement aveugles : v2 ne fusionne que des intervalles de même classe. Dès lors, la classe supérieure d'une colonne ne peut pas changer, et les empreintes en vue en plan ne peuvent pas bouger. Ce résultat nul est une vérification de cohérence, non une mesure d'absence de perte. L'affirmation repose plutôt sur la définition de la fusion, qui n'invente jamais un changement de classe, ainsi que sur la réserve relative aux comptages de points de l'annexe C ; sur cette base, elle devrait toujours être appliquée pour les produits en vue en plan.

Troisièmement, class smooth (v1) achète la plus forte réduction pour un coût réel mais faible : Jaccard de la végétation de 0,951 à la grille grossière et de 0,953 à la fine, face à des réductions d'intervalles de 32,2 et 22,5 %. C'est le réglage par défaut recommandé pour les produits d'analyse.

Quatrièmement, la variante par majorité (v4), l'idée de réunion d'encadrement implémentée pour être mesurée, est quantifiablement nuisible : à la résolution grossière, son empreinte de végétation en vue en plan s'écarte de l'empreinte brute sur 27 % de leur union (J_veg 0,731), et sur 18 % à 0,5 m (J_veg 0,820). Un moins un indice de Jaccard est une différence symétrique et non une fraction détruite, et la métrique couvre l'empreinte de végétation entière, tandis que la règle ne réécrit que la bande au-dessus de 5 m ; ce qu'elle établit est que les canopées brute et encodée se séparent sur plus d'un quart de leur

union à la grille grossière, sur la canopée que le projet veut protéger. Elle reste utile pour une visualisation légère et est exclue de l'analyse.

La suppression des classes minoritaires (v3) est la seule variante dont l'effet ne survit pas au changement de résolution, supprimant 8,9 % des intervalles à 1,0 m à l'horizontale et 0,1 % à 0,5 m : des cellules horizontales plus fines portent moins de points par colonne, si bien que moins d'intervalles tombent sous un seuil fixé en part des retours de la colonne. Cette lecture est cohérente avec les deux tableaux et n'a pas été mesurée séparément.

La comparaison des résolutions porte elle-même une recommandation : la grille grossière est 3,3 fois plus petite que la fine (37,5 contre 124,6 MB en brut), et les reconstructions en vue en plan montrent que chaque emprise de bâtiment et chaque houppier de la zone survivent au grossissement ; des cellules horizontales de 1,0 m sont donc le choix par défaut raisonnable pour les produits d'analyse, 0,5 m étant réservé aux études à l'échelle du quartier. La valeur par défaut du logiciel lui-même reste inchangée à 0,5 m au cube (annexe B) : la recommandation porte sur les produits, et elle n'a pas été propagée à la valeur par défaut livrée.

La validation à l'échelle d'une zone provient d'un balayage de production systématique, non du banc d'essai expérimental : la même zone de 3 x 3 km (36 tuiles, 751,6 millions de points, en flux télécharger-voxéliser-supprimer, chaîne de sortie complète y compris le visualiseur en flux) a été traitée à quatre résolutions, chacune avec et sans regroupement d'intervalles, l'option de production qui fusionne par colonne les intervalles consécutifs de même classe avant que les sorties ne soient dessinées.

Le regroupement et le comblement de lacunes (v2) sont une seule opération exécutée à des tolérances différentes sur des grilles différentes : les -29,5 % du Tableau 4 et les -7,01 % de la section 5.2.1 mesurent la même fusion sur des entrées différentes et ne sont pas comparables. De même, l'octet par point repose sur trois bases distinctes selon le tableau ; les définitions exactes sont en annexe C, et le Tableau 5 ci-dessous résume les huit exécutions archivées.

Tableau 4 : Validation des encodeurs aux résolutions horizontales 1,0 m et 0,5 m (résolution verticale 0,5 m ; 74,3 M points en entrée).

Encodeur	Intervalles	Charge utile	vs brut (interv.)	o/pt	Désaccord %	J_bld	J_veg	J_gnd
Résolution horizontale 1,0 m								
v0 brut	2 195 849	37,54 MB	référence	0,51	0,000	1,000	1,000	1,000
v1 class smooth (3 voxels)	1 488 853	28,35 MB	-32,2 %	0,38	1,659	0,997	0,951	0,986
v2 gap fill (1 m)	1 547 692	29,11 MB	-29,5 %	0,39	0,000	1,000	1,000	1,000
v3 minor class drop (1 %)	1 999 664	34,99 MB	-8,9 %	0,47	0,486	0,999	0,977	0,998
v4 majority above 5 m	1 718 806	31,34 MB	-21,7 %	0,42	5,578	0,992	0,731	0,997
Résolution horizontale 0,5 m								
v0 brut	6 833 550	124,58 MB	référence	1,68	0,000	1,000	1,000	1,000
v1 class smooth (3 voxels)	5 298 833	104,63 MB	-22,5 %	1,41	1,527	0,998	0,953	0,988
v2 gap fill (1 m)	5 411 809	106,10 MB	-20,8 %	1,43	0,000	1,000	1,000	1,000
v3 minor class drop (1 %)	6 828 027	124,51 MB	-0,1 %	1,68	0,0003	1,000	1,000	1,000
v4 majority above 5 m	5 329 452	105,03 MB	-22,0 %	1,41	3,261	0,997	0,820	0,999

Tableau 5 : Le balayage en résolution et en regroupement : une zone de 3 x 3 km, 751,6 M points, huit exécutions complètes de la chaîne.

Exécution	cellule xy / z (m)	Regroupement	Intervalles	Moy/col	Structure (.npz)	Octets/pt	.bin en flux
1	1,0 / 1,0	regroupé	17 958 311	2,46	70,5 MB	0,09	575 MB
2	1,0 / 1,0	brut	20 719 785	2,83	78,2 MB	0,10	663 MB
3	0,5 / 0,5	regroupé	57 979 172	1,98	241,7 MB	0,32	1,86 GB
4	0,5 / 0,5	brut	82 213 507	2,81	305,3 MB	0,41	2,63 GB
5	0,25 / 0,25	regroupé	187 211 562	1,61	817,5 MB	1,09	5,99 GB
6	0,25 / 0,25	brut	270 144 263	2,32	1,04 GB	1,39	8,64 GB
7	0,25 / 0,1	regroupé	187 346 004	1,61	920,2 MB	1,22	6,00 GB
8	0,25 / 0,1	brut	317 402 458	2,72	1,32 GB	1,76	10,16 GB

5.2.1 Série résolution et regroupement sur la Métropole entière

Au-delà du banc d'essai à trois tuiles, le corpus Grand Lyon 2023 complet a été traité de bout en bout : soit 2 842 tuiles, 51 912 212 206 points, une origine commune (1831000,0 ; 5151500,0 ; 130,75), bbox x [1831000, 1861000] et y [5151500, 5195500], 30 km sur 44 km. À 500 m de côté, 2 842 tuiles font 710,5 km² d'emprise : la livraison couvre le territoire administratif entier et le déborde ; un chiffre rapporté à la surface administrative serait faux d'un tiers. L'origine verticale, 130,75 m, est le minimum sur les boîtes englobantes des en-têtes de tuiles, une origine d'indexation, qui ne revendique aucune surface : la classe 7 recueille des retours situés sous le sol, et les plages de certains en-têtes descendent jusque-là. Chaque exécution ci-dessous porte sur ce même corpus ; seuls la taille de cellule et le réglage de regroupement d'intervalles changent. Les exécutions de numéro impair sont regroupées, celles de numéro pair sont le témoin non regroupé à la même résolution. Le Tableau 6 ci-dessous donne le détail par exécution : colonnes, intervalles et trois tailles sur le disque.

Tableau 6 : Géométrie de la structure sur la Métropole entière, par résolution et regroupement (51 912 212 206 points).

Exécution	cellule xy / z (m)	Regroupement	Colonnes	Intervalles	iv/col	Charge utile	area.npz	Stream .bin
Run1	2,0 / 2,0	regroupé	162 581 998	422 234 263	2,60	8,1 GB	1,56 GB	13,51 GB
Run2	2,0 / 2,0	non regroupé	162 581 998	426 927 675	2,63	8,2 GB	1,58 GB	13,66 GB
Run3	1,0 / 1,0	regroupé	648 546 151	1 351 550 067	2,08	27,9 GB	5,23 GB	43,25 GB
Run4	1,0 / 1,0	non regroupé	648 546 151	1 453 375 280	2,24	29,3 GB	5,56 GB	46,51 GB
Run5	0,5 / 0,5	regroupé	2 584 348 757	4 591 625 358	1,78	101,0 GB	18,21 GB	146,93 GB
Run6	0,5 / 0,5	non regroupé	2 584 348 757	5 418 440 422	2,10	111,8 GB	20,41 GB	173,39 GB

La charge utile est la taille sérialisée exacte (16 octets par colonne plus 13 octets par intervalle) ; area.npz est la structure compressée et area_stream.bin la charge utile du visualiseur en flux, toutes deux telles qu'écrites sur le disque. Les trois colonnes de taille sont en gigaoctets décimaux. Le regroupement a fusionné les intervalles de même classe par-dessus des lacunes d'une cellule à 2,0 m, de deux cellules à 1,0 m et de trois cellules à 0,5 m. Deux recoupements se tiennent à l'intérieur du tableau : le journal de fusion de Run3 enregistre son comptage d'avant regroupement à 1 453 375 280 intervalles, exactement le total de Run4, si bien que les exécutions regroupée et non regroupée à 1,0 m s'accordent sur la structure dont elles partent ; et la charge utile en flux de Run4 divisée par son nombre d'intervalles vaut exactement 32 octets, l'enregistrement fixe de la section 4.3.4. Les deux exécutions à 0,5 m ont elles aussi été diffusées en flux ; chaque charge utile vaut son nombre d'intervalles multiplié par l'enregistrement de 32 octets.

Deux constats en découlent.

Premièrement, la structure passe à l'échelle comme attendu, mais son coût n'est pas linéaire : diviser par quatre la surface de cellule, de 2,0 m à 1,0 m, multiplie les colonnes par 4,0 et les intervalles par 3,2, et de nouveau de 1,0 m à 0,5 m par 4,0 et 3,4, en comparant partout les exécutions regroupées entre elles. Le nombre d'intervalles par colonne reste proche de deux sur un changement d'un facteur seize de la surface de cellule, ce qui est l'affirmation centrale de la conception compressée par colonnes tenant à l'échelle de la Métropole : une colonne coûte environ deux intervalles quelle que soit la résolution, d'où une structure qui ne dégénère jamais vers une grille dense.

Deuxièmement, le bénéfice du regroupement d'intervalles dépend fortement de la résolution, et il va à l'encontre de l'intuition.

Le regroupement ne supprime presque rien à 2,0 m et deux cinquièmes des intervalles à 0,25 m, et la série entre les deux est monotone. Le mécanisme est physique, non algorithmique : avec des cellules grossières, une colonne s'effondre déjà en quelques longues plages, si bien qu'il ne reste presque plus de lacunes de même classe à refermer ; avec des cellules fines, les retours LiDAR éparpillés fragmentent un

même objet physique en de nombreuses plages courtes séparées par des voxels vides, et le regroupement récupère cette fragmentation.

Résolution	Non regroupé	Regroupé	Réduction
2,0 m au cube (Run2 -> Run1)	426 927 675	422 234 263	-1,10 %
1,0 m au cube (Run4 -> Run3)	1 453 375 280	1 351 550 067	-7,01 %
0,5 m au cube (Run6 -> Run5)	5 418 440 422	4 591 625 358	-15,26 %
0,25 xy / 0,1 z (Run8 -> Run7, zone de 3 km)	317 402 458	187 346 004	-40,98 %

L'implication pratique est que --group-intervals doit être considéré comme obligatoire à 0,5 m et en dessous et est proche d'une opération nulle au-dessus de 1 m. À noter : la dernière ligne porte sur une zone de 3 km, elle établit donc la tendance et non un chiffre à l'échelle métropolitaine ; une exécution sur le corpus complet à 0,25 m serait nécessaire pour clore la série (travaux futurs, section 6.2). Les exécutions 5 à 8 du Tableau 5 font le même constat par l'autre bout : raffiner la cellule verticale de 0,25 m à 0,1 m porte le nombre d'intervalles non regroupés de 270,1 à 317,4 millions tandis que le nombre regroupé bouge de 0,07 %, de 187 211 562 à 187 346 004. Ce que la grille plus fine ajoute est par conséquent une fragmentation que le regroupement retire aussitôt.

La plus grande exécution de production fusionnée à ce jour couvre une boîte englobante de 3 x 3 km de 36 tuiles du côté ouest de l'inventaire (bbox 1834000 5170000 à 1837000 5173000, dont le centre se situe 10,5 km à l'ouest et 2,0 km au sud du centre du corpus) : 751 592 610 points sont devenus 82 213 507 intervalles répartis sur 29 242 275 colonnes, soit une moyenne de 2,81 intervalles par colonne à des cellules de 0,5 m (exécution archivée, consignée dans outputs/README.md). À l'échelle d'une zone, la colonne moyenne coûte moins de trois intervalles, ce qui est le régime pour lequel la conception a été bâtie.

La comparaison avec la tuile dense isolée 18410_51825 demande de la prudence, car la résolution y diffère autant que le caractère urbain ; ramenée à une grille commune, une tuile riche en murs coûte environ deux fois plus d'intervalles par colonne qu'une zone mixte, et non cinq (détail en annexe C). Au-delà de la capacité, l'exécution démontre les affirmations de l'architecture en production : traitement tuile par tuile contre une origine commune, statistiques fusionnées et cartes de zone entière assemblées.

5.2.2 Résultats du lancer de rayons

La couche de requêtes de la section 4.4 livre trois résultats mesurés : le comportement de la sonde de transmittance, l'expérience d'heures d'ensoleillement direct, et la comparaison aux approches 2-D et 2.5-D.

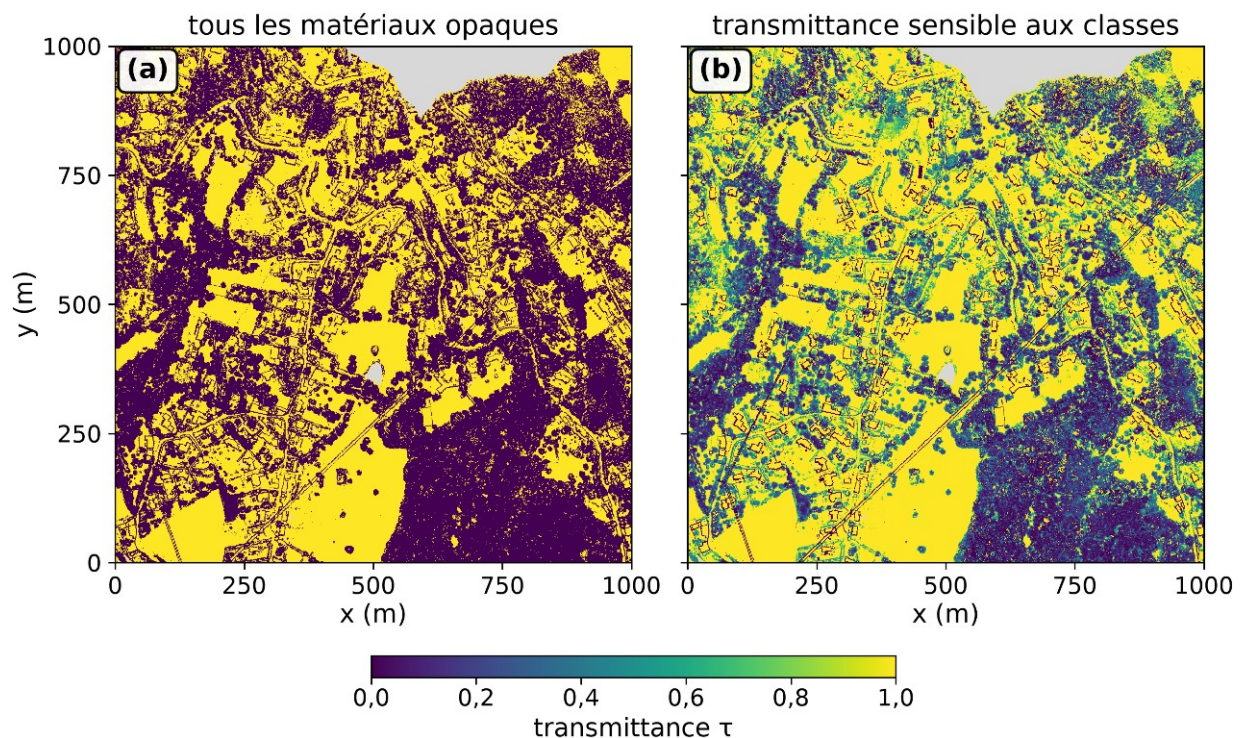
Trois choses sont à lire dans le Tableau 13, reporté en annexe C, et une à n'y pas lire.

Premièrement, le modèle se comporte comme la physique l'exige : pour une structure fixée, augmenter le coefficient abaisse la transmittance moyenne de façon monotone, tandis que la longueur moyenne de trajet en canopée reste pratiquement constante, puisque la géométrie n'a pas changé. Cette séparation de la géométrie et du matériau est ce que le marcheur booléen ne pouvait pas exprimer.

Deuxièmement, l'écart entre la ligne opaque et les lignes à coefficient fini est la grandeur que demande l'objectif (iv). À $0,25 \text{ xy} / 0,1 \text{ z}$ avec $k = 0,1$, la transmittance moyenne monte de 0,650 à 0,815 et la fraction de colonnes recevant plus de la moitié de la lumière incidente monte de 0,65 à 0,84. Cette différence est entièrement imputable au fait de traiter la végétation comme partiellement transmissive plutôt que solide.

Troisièmement, le coût est désormais dominé par le nombre de rayons plutôt que par la résolution verticale, ce qui rend faisable une simulation sur une journée entière.

Ce qu'il ne faut pas en lire : la forte variation de la référence opaque avec la résolution (0,000 ; 0,440 ; 0,650) n'est pas un résultat physique. C'est en partie un effet réel, en ce que des voxels plus grossiers sont occupés dès qu'un retour tombe n'importe où à l'intérieur et occluent donc davantage, et en partie un artefact de cette sonde, qui fait partir chaque rayon à 0,5 m fixes au-dessus de la base de l'intervalle le plus bas, hauteur qui se trouve à l'intérieur du voxel de sol pour des cellules de 1,0 m et au-dessus de lui pour des cellules de 0,1 m. La sonde est une démonstration que la machinerie fonctionne et se comporte de façon monotone, non une mesure de l'occlusion en fonction de la résolution. Une mesure défendable de cet effet demande une origine de rayon définie relativement à la surface du sol (travaux futurs, section 6.2). La Figure 10, ci-dessous, place les deux modèles de matériaux côte à côte sur une zone entière à une origine de ce type : un rayon vertical vers le ciel par colonne de la zone B04, depuis 1,5 m au-dessus du sommet de l'intervalle le plus bas de la colonne, avec tous les matériaux opaques dans la première vue et les coefficients sensibles aux classes de la section 4.4.5 dans la seconde.



Zone B04, 1 km sur 1 km, coin sud-ouest (1838000, 5182000), EPSG:3946, grille de 1,0 m au cube.

967 432 colonnes, un pixel par colonne, 1,0 m par pixel ; gris = aucune colonne.

Rayons verticaux vers le ciel depuis 1,5 m au-dessus du sommet de l'intervalle le plus bas de chaque colonne.

$k = 0,163$ (3), $0,238$ (4), $0,208$ (5), $0,163$ (8) par mètre ; $\tau = 0$ bloqué, $\tau = 1$ ciel dégagé.

Figure 10 : Transmittance verticale vers le ciel sur la zone B04, les mêmes rayons dans les deux vues : (a) tous les matériaux opaques, (b) transmittance sensible aux classes aux coefficients de la section 4.4.5. La transmittance moyenne passe de 0,461 à 0,635 et la fraction bloquée tombe de 0,539 à 0,086.

Le passage du modèle opaque au modèle sensible aux classes gagne du soleil direct dans les deux zones ; le Tableau 7 donne les moyennes mesurées.

Tableau 7 : Heures d'ensoleillement direct, solstice d'été 2026-06-21, échantillonnage à 15 minutes (63 positions solaires), 12 000 colonnes échantillonnées par zone à 1,0 m au cube.

B04 et B10 sont des boîtes de 1 km sur 1 km dont les coins sud-ouest sont en (1838000, 5182000) et (1853000, 5171000), tirées par une règle de quintiles déterministe sur les 2 657 boîtes de 1 km complètes du corpus (Experiments/run_stratified_n10.py) ; les pourcentages de composition sont des parts des points LAZ bruts de la boîte. Les coefficients sont l'ajustement à 1,0 m de la section 4.4.5, appliqués par mètre : 0,163 pour la classe 3, 0,238 pour la classe 4, 0,208 pour la classe 5 et 0,163 pour la classe 8. Les moyennes sont arrondies au millième d'heure ; les différences sont calculées sur les valeurs non arrondies.

Zone	Caractère	Moyenne opaque	Moyenne sensible aux classes	Différence	Colonnes sans soleil (opaque -> sensible aux classes)
B04	58,6 % de végétation, 3,7 % de bâtiment	4,398 h	6,233 h	+1,834 h (+41,7 %)	47,9 % -> 7,5 %
B10	7,9 % de végétation, 25,9 % de bâtiment	10,357 h	11,064 h	+0,707 h (+6,8 %)	15,1 % -> 9,3 %

Le résultat se comporte comme la physique l'exige, et la comparaison entre les deux zones montre qu'il mesure bien ce qu'il prétend. L'effet est six fois plus grand en termes relatifs dans la zone végétalisée que dans la zone bâtie (+41,7 % de la moyenne opaque contre +6,8 %), parce que la différence entre les deux modèles est le traitement de la canopée : les bâtiments sont opaques dans les deux cas. Les deux zones diffèrent aussi fortement par leur étendue verticale – la structure végétalisée couvre 234 niveaux de voxels occupés, autant par le relief du terrain que par la canopée, contre 39 dans la boîte bâtie – et cette étendue gouverne la quantité de géométrie située plus loin sur la route qu'un rayon montant doit franchir. Sous le modèle opaque, 47,9 % des colonnes de la zone végétalisée ne reçoivent aucun soleil direct ; traiter la canopée comme partiellement transmissive ramène cela à 7,5 %. Cet écart – quelque deux cinquièmes de la zone – est la conséquence pratique du choix de représentation, et il n'est accessible à aucun modèle de surface, lequel n'a aucun moyen d'exprimer « le houpplier au-dessus de ce point laisse passer un peu de lumière ».

Comparé naïvement, le modèle de surface lit près de cinq heures de plus par jour que la structure opaque sur la zone végétalisée (annexe C). L'essentiel n'est pas le modèle d'occlusion : le modèle S2, qui ne change que la hauteur de départ du rayon, se situe environ un tiers d'heure sous la structure, et ce coût va dans l'autre sens. La comparaison naïve flatte le modèle volumétrique ; la comparaison contrôlée montre encore la divergence et la localise. Là où une colonne porte un corps unique, les deux réponses s'accordent à l'heure sur 79 à 92 % des colonnes ; là où une canopée surmonte le sol, elles s'écartent de 9,7 à 11,7 heures par jour. Un champ de hauteurs n'a pas de dessous, et c'est cela, non la fusion de la colonne, que mesure la divergence. L'erreur va dans les deux sens : face au modèle sensible aux classes, le modèle de surface est plus bas sur 45,3 % des colonnes végétalisées, la structure laissant passer une part du faisceau à travers une couronne voisine. Sur les dix kilomètres carrés stratifiés, le modèle de surface surestime le volume de canopée de 13 à 54 % à 1,0 m, davantage à 0,5 m, et ne peut

représenter les 8,5 à 40 % de colonnes portant deux corps séparés par de l'air mesuré. La résolution ne déplace qu'un seul côté : le modèle S est presque invariant par la grille, tandis que les modèles volumétriques montent d'environ une heure quand la cellule est divisée par deux (un voxel de 1 m portant un seul retour de feuillage est entièrement opaque, contre un sous-voxel sur huit à 0,5 m). Le chiffre que ce rapport retient est donc celui de 0,5 m : 3,755 heures sur la zone végétalisée et 1,299 sur la zone bâtie face à la structure opaque, contre 5,291 et 2,334 à 1,0 m. Le modèle A étant fixé en résolution par sa conception, la comparaison fine oppose des modèles volumétriques à 0,5 m à un raster à 1 m, et cette asymétrie est elle-même un résultat. Les quatre autres requêtes, le détail par zone et les trois coûts mesurés sont en annexe C.

5.3 Démonstration

La Figure 11 montre la validation des encodeurs en vue en plan, la perte de végétation de la variante par majorité visible à l'œil nu.

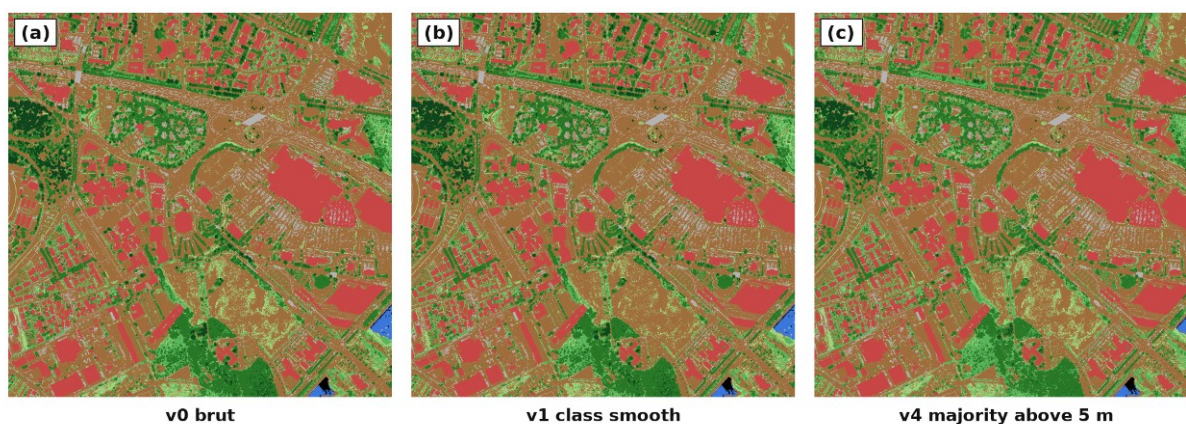


Figure 11 : Comparaison des encodeurs sur une zone urbaine de 1,5 km de côté (Bron, autour du campus Porte des Alpes ; cellules de 1,0 m à l'horizontale par 0,5 m à la verticale) : (a) brut, (b) class smooth et (c) majority above 5 m ; les variantes ne divergent du brut que sur 2,7 % et 3,4 % des colonnes ; les différences se concentrent sur les lisières de canopée et les toits mixtes. Les taches sombres près des plans d'eau sont des zones sans retour laser (absorption par l'eau).

La Figure 12 donne la distribution du nombre d'intervalles par colonne pour l'exécution brute la plus fine du balayage de la section 5.2 (0,25 m à l'horizontale, 0,1 m à la verticale, sans regroupement), le régime où la fragmentation est à son pire et où les diagnostics sont les plus informatifs.

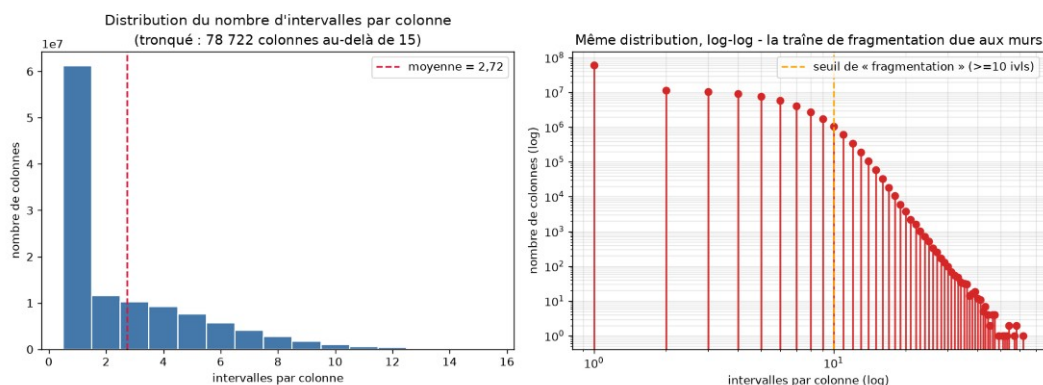


Figure 12 : Distribution du nombre d'intervalles par colonne pour la structure brute la plus fine : la longue traîne est la fragmentation due aux murs.

La Figure 13 présente les quatre modes de carte en vue en plan de l'exécution sur la zone de 3 x 3 km.

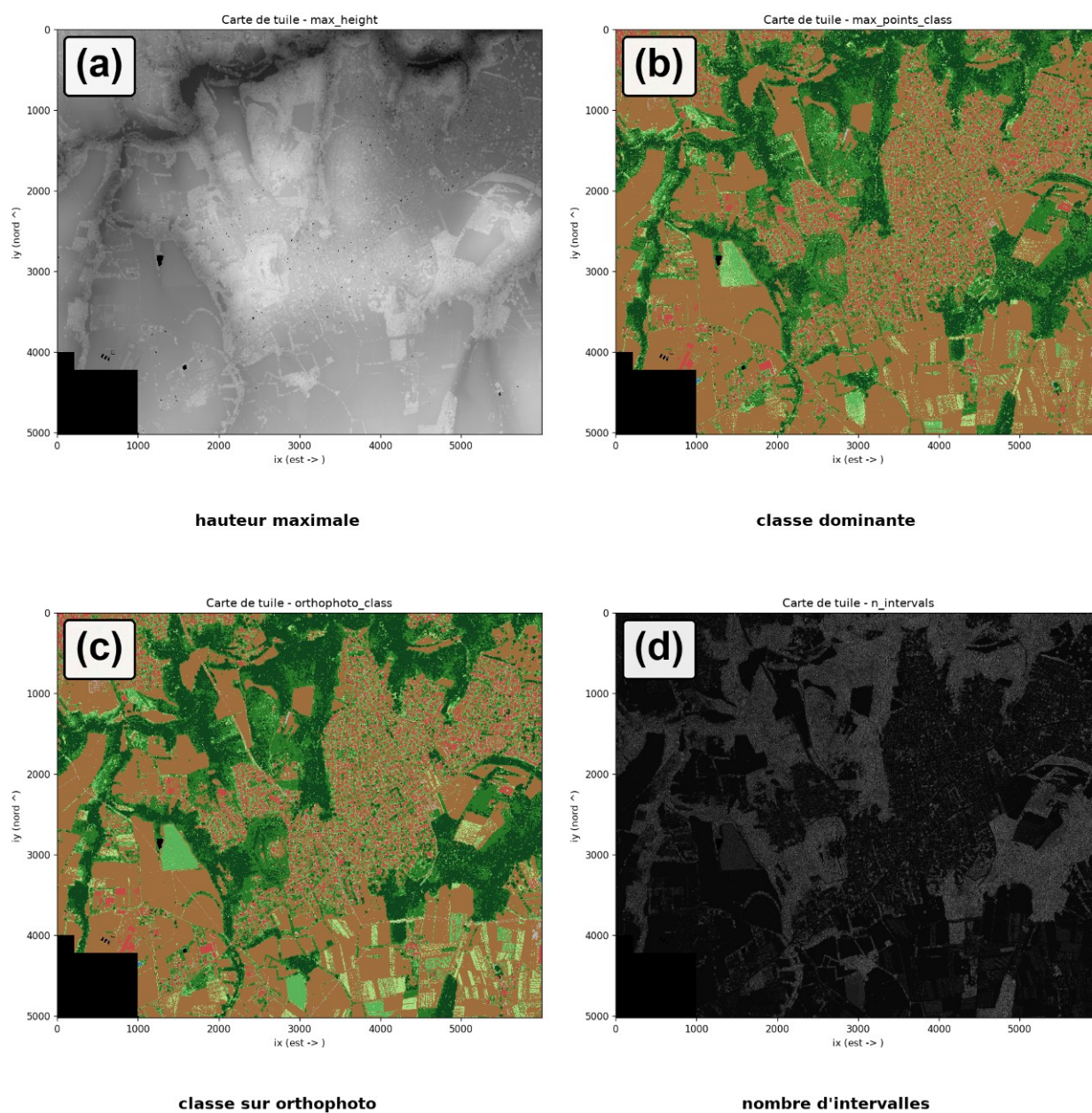


Figure 13 : Les quatre modes de carte en vue en plan de l'exécution sur la zone de 3 x 3 km :
(a) hauteur maximale, (b) classe dominante, (c) classe sur orthophoto, (d) nombre d'intervalles.

Les Figures 14 et 15 montrent les diagnostics de colonnes, la forme visuelle directe de la structure de données : colonnes témoins en piles d'intervalles, puis panneaux par classe de la même exécution brute la plus fine.

Colonnes témoins de 18350_51730 (cell_xy=0,25 m, cell_z=0,1 m)

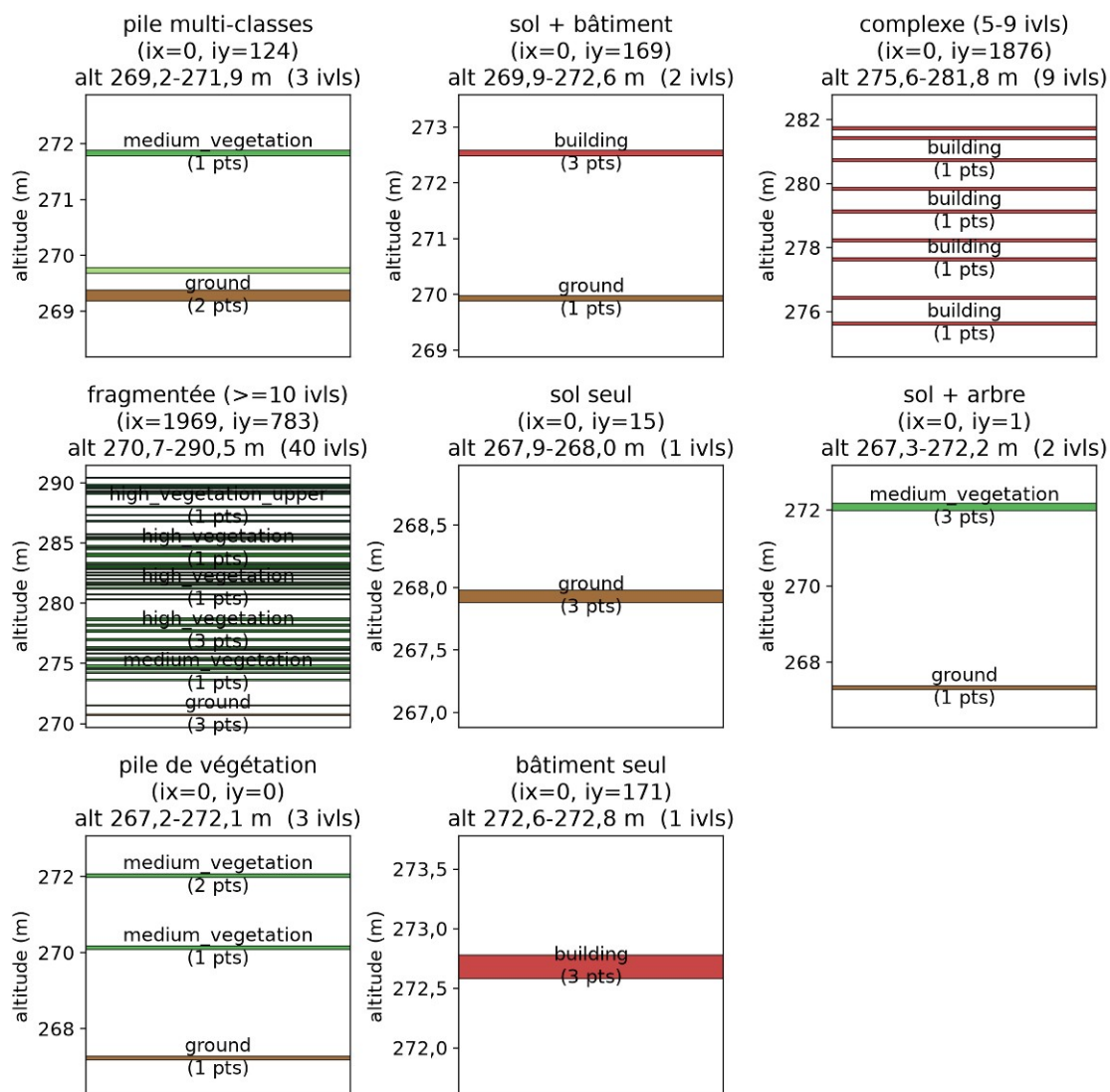


Figure 14 : Diagnostics de colonnes de la tuile 18350_51730, une colonne témoin par catégorie rendue sous forme de pile d'intervalles.

Une colonne représentative par classe (cell_xy=0,25 m, cell_z=0,1 m)

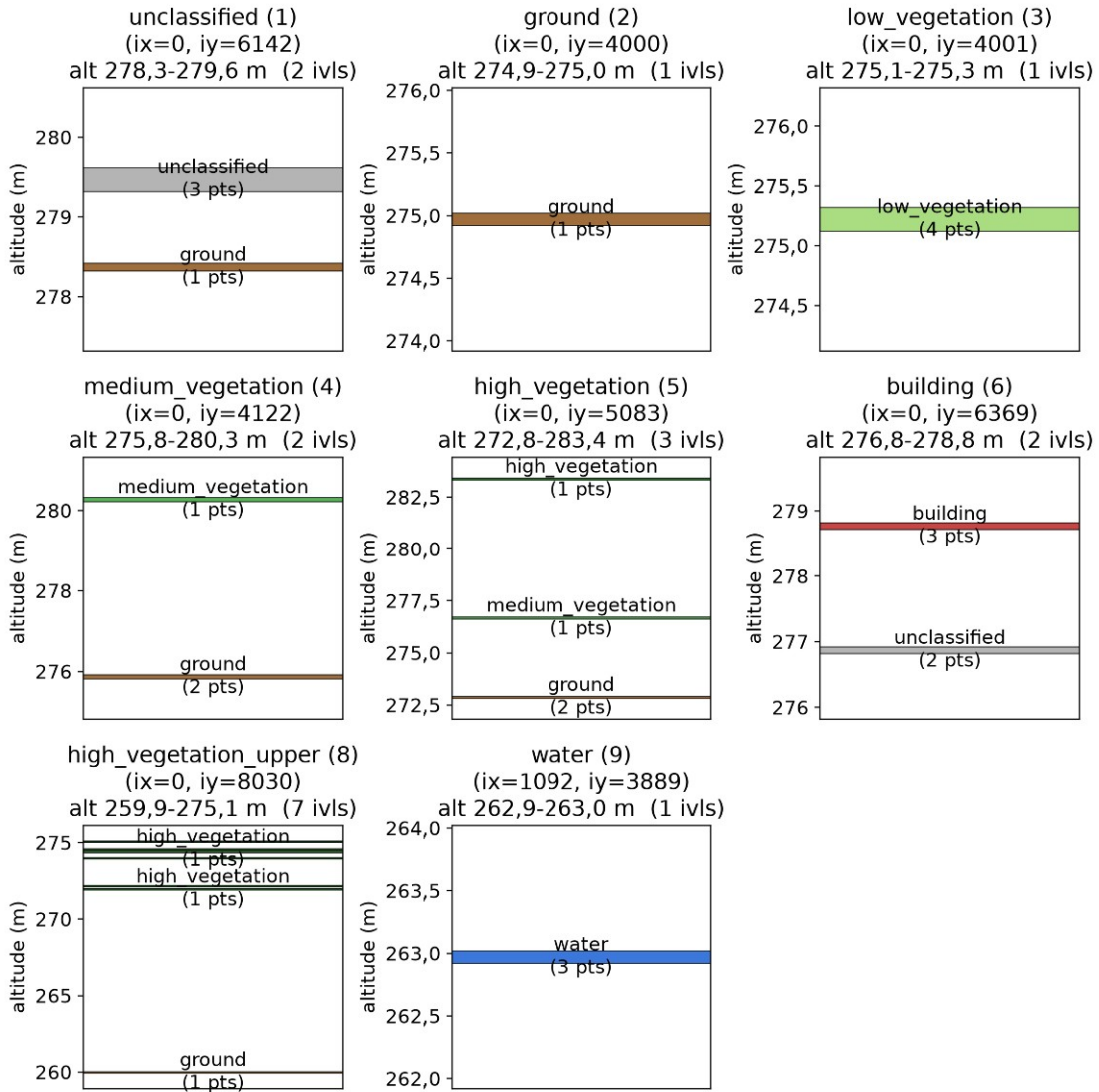


Figure 15 : Panneaux de colonnes par classe de la structure brute la plus fine.

La Figure 16 montre le visualiseur en flux sur une zone d'étude de 2 x 2 km à des cellules de 0,25 m à l'horizontale et 0,1 m à la verticale, parcourue dans un navigateur : la charge utile porte un enregistrement de 32 octets par boîte d'intervalle au détail complet, 96,5 millions pour cette zone, et le budget par image décide des tuiles résidentes sur le GPU, ici 46 tuiles sur 1 024 et environ 4,5 millions d'instances dessinées à 35 images par seconde. La Figure 17 place la même zone en regard dans Google Earth, sur une imagerie de 2023 comme la campagne LiDAR : le bâti et les couronnes d'arbres s'y reconnaissent directement.

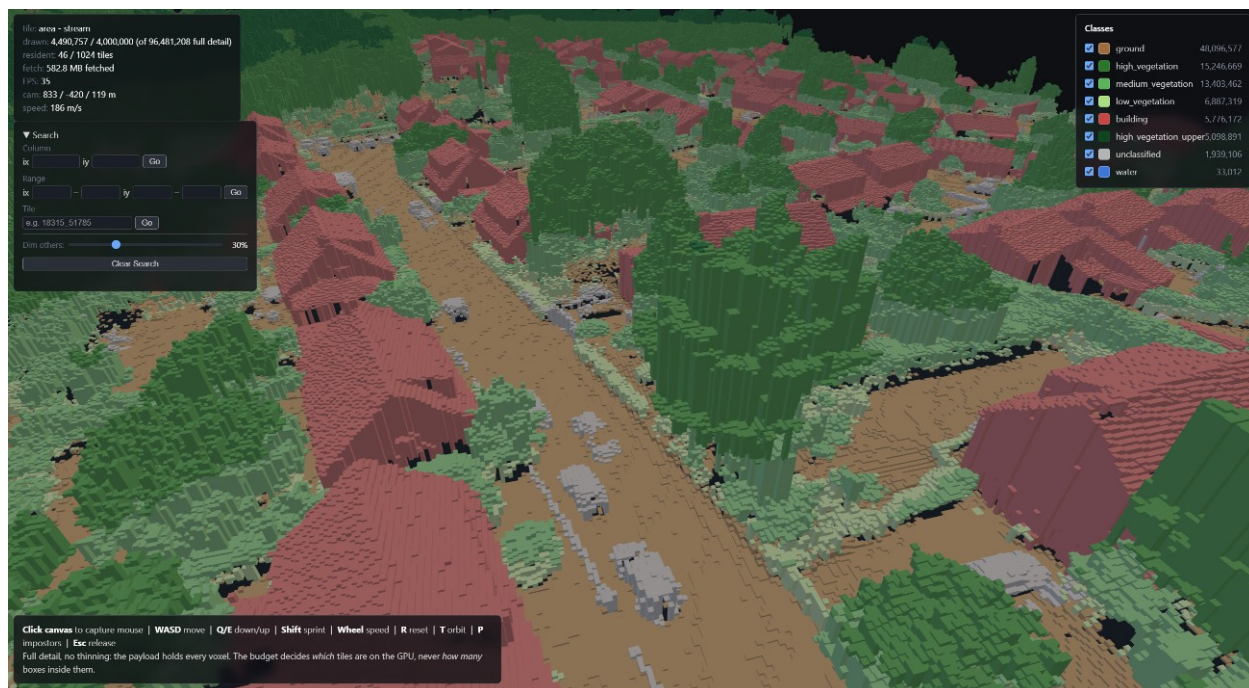


Figure 16 : Le visualiseur en flux sur une zone d'étude de 2 x 2 km à des cellules de 0,25 m à l'horizontale et 0,1 m à la verticale : navigation au niveau de la rue avec comptages par classe et bascules de visibilité ; 46 tuiles résidentes sur 1 024, environ 4,5 millions de boîtes d'intervalles dessinées sur 96,5 millions au détail complet, à 35 images par seconde.



Figure 17 : Vue Google Earth de la même zone d'étude (imagerie 3-D de 2023, © Google) : les bâtiments et les couronnes d'arbres de la Figure 16 s'y reconnaissent.

Au-delà des visualiseurs de la chaîne d'outils elle-même, le tileset exporté démontre l'interopérabilité en pratique : le même tileset.json, servi comme fichiers statiques, s'affiche dans CesiumJS, dans iTowns de l'IGN et dans Unreal Engine via Cesium for Unreal, à ses véritables coordonnées lyonnaises, le détail étant diffusé par la pyramide ; le répertoire tilesexport du dépôt documente le chemin complet.

5.4 Discussion et limitations

Les limitations se divisent en ce que l'entrée impose, ce que la conception accepte, et ce que l'implémentation nous a appris avec difficultés. Héritée de l'entrée, la sémantique de la structure ne vaut pas mieux que la classification de l'IGN. Les retours sur les murs fragmentent les colonnes, mais l'ampleur en est modérée : à grille commune, environ deux fois plus d'intervalles par colonne qu'une zone mixte, non les cinq fois d'une comparaison naïve entre résolutions différentes (section 5.2.1). L'étage supérieur de canopée de classe 8 est une convention de production de fait, non une classe documentée, et les retours de sol mal étiquetés doivent être traités par des heuristiques (la règle du sol le plus haut de la section 4.2.3). Rien de tout cela n'est corrigé à notre stade sans re-résoudre la classification, ce qui a été exclu du périmètre de notre étude.

Acceptée par conception, l'hypothèse de faisceau nadiral laisse une erreur résiduelle de quelques colonnes suspectes à côté des bâtiments hauts, même après atténuation (section 4.4.1), dont la largeur a été mesurée sur un kilomètre carré, non sur la livraison entière : un chiffre dérivé d'un champ de vue supposé de plus ou moins 15 degrés a été démenti par la mesure sur les 2 842 tuiles (l'intervalle réel est de moins 30 à plus 30 degrés, ce qui double la dérive latérale à 0,58 m par mètre de hauteur, soit 28,9 m à 50 m). Le remplacement est une fraction de la hauteur du bâtiment et non un nombre fixe de colonnes, et il repose sur quatre tuiles dont le bâtiment le plus haut atteint 14,00 m, sans éprouver les hauteurs de tour ; la construction indifférente au voisinage implique que les analyses horizontales paient un coût de recalcul ; et le sous-sol reste un état du décodeur plutôt qu'un contenu mesuré, d'où une plantabilité sous le trottoir bornée par ce que la surface révèle, tant que des sources de données souterraines ne sont pas intégrées.

La défaillance mémoire du visualiseur en flux est l'incident déterminant, celui dont le chapitre 4 argumente : une structure de zone de 317 millions d'intervalles sur 116,5 millions de colonnes a fait tomber l'étape de visualisation sur une station de travail de 32 GB, l'autopsie trouvant le collecteur de géométrie en train de matérialiser chaque clé de colonne en tuple Python avant qu'une seule boîte ne soit dessinée ; le détail chiffré est en annexe C.

Les correctifs ont ramené le tout à la lettre du principe même de la conception : les clés restent dans leur tableau numpy empaqueté (0,9 GB pour la même structure), l'empaquetage est devenu incrémental par blocs, et la prise de conscience qu'aucun navigateur ne devrait télécharger 10,16 GB par chargement de page a produit le visualiseur à ensemble de travail de la section 4.3.4. Le balayage de la section 5.2 clôt l'histoire : la même structure, régénérée comme l'exécution brute la plus fine du balayage, traverse désormais l'exportateur repensé de bout en bout avec un pic d'étape inférieur à cinq gigaoctets.

Deux incidents plus petits complètent le tableau : une tuile contenant des coordonnées NaN a corrompu l'origine de grille commune sans qu'aucune erreur ne soit signalée, jusqu'à ce qu'une validation des entrées soit ajoutée avant la quantification, et un après-midi a été perdu sur une erreur qu'aucune lecture des sources ne pouvait expliquer, causée par du bytecode compilé périmé, rappel que l'environnement mérite autant de suspicion que le code. Le résidu de millésime de jeu de données consigné à la section 4.3.8 (taille de tuile et système de coordonnées hérités de l'acquisition de 2018) appartient à la même famille : chaque nombre de ce rapport a été re-dérivé des en-têtes de fichiers ou des sorties d'exécution, jamais repris de confiance depuis des commentaires.

Un incident de requêtes par rayons, une violation d'accès Windows survenue sur la structure la plus fine d'une zone d'étude (15,4 millions de colonnes à des cellules de 0,25 m sur 0,1 m), a demandé deux diagnostics successifs avant de tenir ; le récit complet et ses correctifs, à résultats prouvés identiques et temps d'exécution divisé par deux, sont consignés en annexe C.

Des diagnostics matériels sur la station de travail de développement ont ensuite enregistré des erreurs de contrôle machine du processeur, corrigées mais récurrentes, dégénérant en fautes au niveau du noyau. La machine elle-même est donc un contributeur plausible – peut-être la totalité – de ce non-déterminisme ; l'atténuation tient sous l'une comme sous l'autre explication. Le résidu : une faute native non déterministe ne peut pas être prouvée impossible, si bien qu'une passe de détection des ombres à

l'échelle de la Métropole devrait en outre s'exécuter sous l'isolation de processus par bloc déjà présente dans la chaîne de traitement – le mécanisme même qui contient déjà les défaillances natives du décodeur LiDAR – ce qui transforme toute faute survivante en un bloc perdu qui reprend ; l'exécution elle-même n'est pas perdue. Tous les chiffres rapportés proviennent d'exécutions qui se sont achevées proprement.

5.5 Évaluation d'ensemble

Au regard des cinq objectifs du chapitre 2, le bilan se lit comme suit.

La voxelisation d'une tuile isolée (O1) est atteinte et testée : la construction n'écarte rien, les propriétés d'équivalence tiennent, et l'aller-retour vers LAS/LAZ s'audite proprement. La compacité à fidélité connue (O2) est atteinte et quantifiée : un facteur 55,4 face aux enregistrements de points LAS 1.2 non compressés et un facteur 8,3 face aux tuiles LAZ livrées, par la seule structure par plages, une économie supplémentaire due à des encodeurs dont le coût sémantique est mesuré par classe, fortement dépendante de la résolution (Tableau 4), avec un encodeur rejeté pour l'usage analytique parce que la mesure en a montré le coût.

Le passage à l'échelle (O3) est atteint à la taille de la Métropole entière : la chaîne partitionnée a traité l'inventaire Grand Lyon complet, 2 842 tuiles et 51,9 milliards de points, de bout en bout à trois résolutions (2,0, 1,0 et 0,5 m), avec une mémoire bornée par tuile, des fragments reprenables, et une isolation des défaillances par étape ; l'unique défaut d'entrée réellement rencontré, une tuile portant des coordonnées NaN, a été attrapé par la validation ajoutée avant la quantification (section 5.4) ; la classe fusionnée de trois kilomètres (751,6 millions de points) et la classe de visualisation à des centaines de millions d'intervalles ont été exercées en chemin.

L'aptitude à l'analyse (O4) est atteinte : cartes, statistiques, diagnostics, trois voies de visualisation, un export 3D Tiles qui passe le validateur officiel sans aucune erreur, et une couche de requêtes par rayons sensible aux classes avec une expérience mesurée d'heures d'ensoleillement direct (section 4.4.6) consomment toutes la même structure. Deux réserves accompagnent ce résultat, détaillées dans le tableau ci-dessous.

Tableau 8 : Les deux réserves du validateur officiel sur le tileset livré, leur nature et leur conséquence.

Réserve du validateur	Nature	Conséquence
Extension EXT_mesh_gpu_instancing ignorée	Le validateur ne reconnaît pas l'extension.	Elle porte toute la géométrie, qui n'est donc pas vérifiée par lui.
Inclusion des volumes englobants non vérifiée	Angle mort du schéma, non une violation ; un défaut d'élagage hiérarchique.	556 paires parent-enfant hors boîte dans le tileset livré (« parent » désigne le volume englobant et « enfant » les objets englobés), dont 29 significatives (au-delà de 1 mm), corrigées par une union ascendante des volumes et épinglées par un test de non-régression.

La sémantique du vide (O5) est implémentée avec un modèle d'erreur énoncé : le décodeur à trois états distingue l'air de l'intérieur et du sous-sol, l'atténuation nadirale supprime l'artefact principal, et le résidu est caractérisé. L'affirmation que ce chapitre soutient est précise : pour des requêtes urbaines verticales sur du LiDAR aérien classifié, la structure en colonnes compressée par plages fait le travail pour lequel elle a été conçue, aux échelles dont le projet a actuellement besoin, avec ses compromis mesurés.

6 Conclusion et perspectives

L'objectif final étant l'étude de la plantabilité, ce stage s'était donné pour but de doter le projet IA.rbre d'une représentation volumétrique de la ville à partir des données LiDAR, en explorant une voie au-delà de l'approche octree classique : des colonnes indépendantes de leurs voisines, et un encodage vertical compressé par plages.

En définitive, le projet IA.rbre dispose d'une base mesurée pour déterminer informatiquement où planter des arbres.

De manière pratique, le résultat obtenu est une structure en colonnes compressée par plages pour du LiDAR aérien classifié, construite par un algorithme à deux étapes entièrement vectorisé et tenue dans une disposition en structure de tableaux coûtant environ 16 octets par colonne et 13 octets par intervalle. Autour d'elle, les mécanismes qui rendent une structure de données utilisable en pratique : un décodeur d'espaces vides à trois états ancré sur un indice de sol par colonne, une famille mesurée d'encodeurs de simplification sémantique, un traitement de zone out-of-core avec estimation préalable de la mémoire, des cartes et des diagnostics, trois voies de visualisation, un export 3D Tiles 1.1, et une couche de requêtes par rayons sensible aux classes avec une géométrie solaire validée et une première expérience d'heures d'ensoleillement direct.

La campagne de validation du chapitre 5 a quantifié les compromis les plus importants : une compression d'un facteur 55,4 face aux enregistrements LAS non compressés et d'un facteur 8,3 face aux LAZ livrés, par la seule structure par plages, trente pour cent supplémentaires par le comblement des lacunes, sans perte pour les produits en vue en plan. Une idée de simplification née d'une réunion d'encadrement a été implémentée, mesurée et rejetée pour l'usage analytique précisément parce que la mesure a montré que son empreinte de végétation en vue en plan se sépare de l'empreinte brute sur vingt-sept pour cent de leur union à la grille de 1,0 m, et dix-huit pour cent à 0,5 m, sur la canopée que le projet existe pour protéger. Ce rapport consigne les échecs à côté des résultats : l'autopsie mémoire qui a imposé la conception du visualiseur en flux fait désormais partie de l'argument de conception.

6.1 Au regard de la note de cadrage initiale

La section 2.4 a exposé comment les cinq objectifs mesurables de ce rapport correspondent aux cinq buts de la note de cadrage du stage. Cette section ferme la boucle. Le Tableau 9 récapitule ces cinq buts ; les paragraphes qui suivent détaillent chaque ligne.

Tableau 9 : Bilan des cinq buts de la note de cadrage : résultat obtenu et statut de chaque but.

#	But de la note de cadrage	Résultat obtenu	Statut
1	Modèle voxel sémantique de la ville (végétation, sol, bâti, sous-sol)	Structure en colonnes compressée par plages (chapitre 4), évaluée sur la Métropole entière (chapitre 5) ; un état dédié réserve le sous-sol	Partiellement atteint : le sous-sol reste sans données, limite du capteur
2	Construction depuis les données existantes	2 842 tuiles, 51,9 milliards de points, trois résolutions	Complètement atteint sur le LiDAR seul ; DSM et CityGML écartés par choix argumenté (section 2.4)
3	Classification des voxels	Classes IGN conservées ; classe 8 nommée étage supérieur de canopée ; trois états d'espace vide	Complètement atteint
4	Ombrage par lancer de rayons volumique	Transmittance par classe validée, contribution du volumique isolée (section 4.4.6)	Complètement atteint, à l'échelle d'une zone d'étude ; la Métropole en perspective
5	Comparaison aux approches 2-D et 2.5-D	Surestimation 2.5-D de 9,7 à 11,7 h/j sous canopée ; volume de canopée surestimé de 13 à 54 % à 1,0 m (section 4.4.7)	Complètement atteint ; une comparaison externe indépendante reste en perspective

Les buts 1 à 3 (modèle, construction, classification) font l'objet des chapitres 4 et 5 ; deux choix méritent d'être rappelés. Une source unique uniformément classifiée, plutôt que le LiDAR joint aux DSM et CityGML, couvre 2 842 tuiles et 51,9 milliards de points et caractérise la représentation sur un territoire entier, ce que des sources partiellement recouvrantes ne permettraient pas. Le sous-sol reste une limite de mesure : le capteur ne relève que la première surface frappée, la classification livrée ne porte aucune classe de sous-sol, et aucune modélisation ne fournit ce que le capteur n'a pas mesuré.

Les deux buts applicatifs ont été menés à l'échelle d'une zone (but 4, section 4.4.6) et sur dix kilomètres carrés stratifiés (but 5, section 4.4.7), leurs périmètres énoncés : des heures géométriques d'ensoleillement plutôt qu'un éclairage radiométrique, et un contrôle séparant la hauteur d'échantillonnage du modèle d'occlusion ; les valeurs figurent dans le tableau ci-dessus et à la section 4.4.7. Une comparaison à un produit externe de provenance indépendante reste à mener, les deux modèles de référence étant des réimplémentations de ce modèle sur nos propres données ; la comparaison interne des encodeurs du chapitre 5 la complète sans la remplacer.

6.2 Perspectives

Physique : passer le calcul des heures d'ensoleillement direct de la section 4.4.6 de zones échantillonnées à la Métropole entière sous l'isolation de processus par bloc, l'étendre à la composante diffuse du ciel qui sépare les heures d'ensoleillement géométriques de l'éclairage, valider l'atténuation nadirale contre un lanceur de rayons de référence, reprendre la sonde de transmittance avec une origine de rayon définie relativement à la surface du sol, ce qui rendrait défendable la mesure de l'occlusion en fonction de la résolution (section 5.2.2), et relier le résultat aux cas d'usage de la ligne pySunlight. La structure peut servir ses propres vues 2-D : matérialiser les rasters qu'elle dérive répond aux requêtes de

type champ de hauteurs à la vitesse du raster, si bien que l'écart de lecture de la section 4.4.7 est un coût d'implémentation et non une propriété de la représentation.

Échelle : première étape faite, l'inventaire complet du Grand Lyon (2 842 tuiles, 51,9 milliards de points) a été traité de bout en bout par la chaîne partitionnée à trois résolutions ; ce qui reste est de clore la série résolution-regroupement par une exécution du corpus complet à 0,25 m (section 5.2.1), puis de promouvoir ces exécutions ponctuelles en une chaîne exploitée et rafraîchie régulièrement, dont les structures fusionnées, les cartes assemblées et les visualiseurs en flux sont servis à la plateforme et non produits à la demande.

Donnée : intégrer des sources du sous-sol (réseaux, cartes pédologiques) remplacerait l'état de sous-sol du décodeur par du contenu mesuré et débloquerait en profondeur le volet faisabilité de la question de plantabilité ; la modélisation des matériaux à la manière de SBRT [27] étudiée pendant la phase de conception (extensions verticales exactes, attributs physiques par matériau) est la représentation qu'appelleraient de telles données.

Temporelle : si une nouvelle campagne de mesure du LiDAR HD de l'IGN est organisée, deux structures du même territoire à des dates différentes se soustrairont naturellement, colonne par colonne (sous réserve d'un même alignement) en une carte de changement de la croissance de la canopée, des constructions et des démolitions, passant ainsi d'une comparaison d'images à une comparaison de structures.

Intégration : déjà en partie faite, l'export 3D Tiles produit désormais des tilesets validés avec une pyramide de niveaux de détail qui s'affichent dans CesiumJS, iTowns et Unreal Engine via Cesium for Unreal, aux véritables coordonnées lyonnaises. Ce qui reste est d'enrichir les tuiles de métadonnées par instance pour un stylage par clic et requête, et d'exposer les tilesets à travers la plateforme IA.rbre ; le décalage orthométrique du géoïde (+49,70 m près de Lyon, d'après la grille RAF de l'IGN) est désormais appliqué automatiquement à l'export.

L'objectif principal reste la plantabilité des arbres. Une fois les zones plantables identifiées, leur faisabilité restera à confirmer face aux réseaux souterrains, égouts ou câbles électriques, une vérification que les données LiDAR fournies par la Métropole de Lyon ne peuvent pas porter seules.

6.3 Remerciements

Je remercie mes encadrants au laboratoire, John Samuel, Gilles Gesquière, Corentin Gautier et Arthur Villarroja-Palau, pour un sujet d'une réelle consistance et pour leur accompagnement constant ; Didier Puzenat, mon encadrant académique, pour le principe selon lequel un rapport doit argumenter ses choix ; l'équipe VCity et les collègues du LIRIS ; et les partenaires d'IA.rbre, dont la plateforme donne à ce travail sa raison d'être.

Bibliographie

- [1] VCity Team, "pySunlight: Light pre-calculation based on real data (urban data and sun position) with 3D Tiles," GitHub, 2023. [Online]. Available: <https://github.com/VCityTeam/pySunlight>. [Accessed: Aug. 2026].
- [2] TelesCoop and IA.rbre Consortium, "IA.rbre: Intelligence artificielle pour l'arbre - Plateforme web Big Data d'aide à la décision pour la végétalisation urbaine," France 2030 project, 2025. [Online]. Available: <https://iarbre.fr>. [Accessed: Jul. 2026].
- [3] Banque des Territoires, "Démonstrateurs d'IA frugale au service de la transition écologique des territoires (DIAT) - Appel à projets France 2030," Caisse des Dépôts et Consignations, Paris, France, 2023. [Online]. Available: <https://www.banquedesterritoires.fr>. [Accessed: Jul. 2026].
- [4] Institut national de l'information géographique et forestière (IGN), "LiDAR HD: Vers une nouvelle cartographie 3D du territoire," IGN, Saint-Mandé, France, 2023. [Online]. Available: <https://geoservices.ign.fr/lidarhd>. [Accessed: Jul. 2026].
- [5] Métropole de Lyon, "Data Grand Lyon - Open data portal," 2023. [Online]. Available: <https://data.grandlyon.com>. [Accessed: Jul. 2026].
- [6] M. Isenburg, "LASzip: Lossless compression of lidar data," *Photogramm. Eng. Remote Sens.*, vol. 79, no. 2, pp. 209-217, 2013.
- [7] American Society for Photogrammetry and Remote Sensing (ASPRS), LAS Specification 1.4-R15, ASPRS, Baton Rouge, LA, USA, 2019.
- [8] American Society for Photogrammetry and Remote Sensing (ASPRS), LAS Specification Version 1.2, approved by the ASPRS Board 2 Sep. 2008. [Online]. Available: https://www.asprs.org/wp-content/uploads/2010/12/asprs_las_format_v12.pdf. [Accessed: Aug. 2026].
- [9] Open Geospatial Consortium, 3D Tiles Specification 1.1, OGC Document 22-025r4, 2023. [Online]. Available: <https://docs.ogc.org/cs/22-025r4/22-025r4.html>. [Accessed: Jul. 2026].
- [10] M. Aleksandrov, S. Zlatanova, and D. J. Heslop, "Voxelisation algorithms and data structures: A review," *Sensors*, vol. 21, no. 24, p. 8241, Dec. 2021, doi: 10.3390/s21248241.
- [11] M. Wang and Y.-H. Tseng, "Incremental segmentation of lidar point clouds with an octree-structured voxel space," *The Photogrammetric Record*, vol. 26, no. 133, pp. 32-57, Mar. 2011, doi: 10.1111/j.1477-9730.2011.00624.x.
- [12] B. Gorte, S. Zlatanova, M. Pilouk, A. Diakité, and J. Barton, "3D data integration in the voxel domain," *ISPRS Ann. Photogramm. Remote Sens. Spatial Inf. Sci.*, vol. X-4-2024, pp. 133-140, Oct. 2024, doi: 10.5194/isprs-annals-X-4-2024-133-2024.
- [13] B. Gorte, "Analysis of very large voxel data sets," *Int. J. Appl. Earth Obs. Geoinf.*, vol. 119, p. 103316, May 2023, doi: 10.1016/j.jag.2023.103316.
- [14] V. Kämpe, E. Sintorn, and U. Assarsson, "High resolution sparse voxel DAGs," *ACM Trans. Graph.*, vol. 32, no. 4, p. 101, Jul. 2013, doi: 10.1145/2461912.2462024.
- [15] K. Museth, "VDB: High-resolution sparse volumes with dynamic topology," *ACM Trans. Graph.*, vol. 32, no. 3, pp. 1-22, Jun. 2013, doi: 10.1145/2487228.2487235.
- [16] K. Museth, "NanoVDB: A GPU-friendly and portable VDB data structure for real-time rendering and simulation," in *ACM SIGGRAPH 2021 Talks*, New York, NY, USA, 2021, pp. 1-2, doi: 10.1145/3450623.3464653.
- [17] J. C. Hart, "Sphere tracing: A geometric method for the antialiased ray tracing of implicit surfaces," *Vis. Comput.*, vol. 12, no. 10, pp. 527-545, 1996, doi: 10.1007/s003710050084.
- [18] T. Saito and J.-I. Toriwaki, "New algorithms for euclidean distance transformation of an n-dimensional digitized picture with applications," *Pattern Recognit.*, vol. 27, no. 11, pp. 1551-1565, Nov. 1994, doi: 10.1016/0031-3203(94)90133-3.
- [19] D. E. O. Tzamaras, K. Chow, I. Blanes, and J. Serra-Sagristà, "Fast run-length compression of point cloud geometry," *IEEE Trans. Image Process.*, vol. 31, pp. 4490-4501, 2022, doi: 10.1109/TIP.2022.3185541.

- [20] S. W. Golomb, "Run-length encodings," *IEEE Trans. Inf. Theory*, vol. 12, no. 3, pp. 399-401, Jul. 1966, doi: 10.1109/TIT.1966.1053907.
- [21] D. Abadi, P. Boncz, S. Harizopoulos, S. Idreos, and S. Madden, "The design and implementation of modern column-oriented database systems," *Found. Trends Databases*, vol. 5, no. 3, pp. 197-280, 2013, doi: 10.1561/19000000024.
- [22] H. Aljumaily, D. F. Laefer, D. Cuadra, and M. Velasco, "Point cloud voxel classification of aerial urban LiDAR using voxel attributes and random forest approach," *Int. J. Appl. Earth Obs. Geoinf.*, vol. 118, p. 103208, Apr. 2023, doi: 10.1016/j.jag.2023.103208.
- [23] F. Poux and R. Billen, "Voxel-based 3D point cloud semantic segmentation: Unsupervised geometric and relationship featuring vs deep learning methods," *ISPRS Int. J. Geo-Inf.*, vol. 8, no. 5, p. 213, May 2019, doi: 10.3390/ijgi8050213.
- [24] C. Gaydon, "Myria3D: Deep learning for the semantic segmentation of aerial lidar point clouds," GitHub, Institut national de l'information géographique et forestière (IGN), 2022. [Online]. Available: <https://github.com/IGNF/myria3d>. [Accessed: Aug. 2026]. Le modèle de segmentation est une implémentation PyTorch Geometric de RandLA-Net (Q. Hu *et al.*, CVPR 2020).
- [25] C. Gaydon, M. Daab, and F. Roche, "FRACTAL: An ultra-large-scale aerial lidar dataset for 3D semantic segmentation of diverse landscapes," 2024, arXiv:2405.04634.
- [26] M. Kress, "LiVoxGen: LiDAR voxelization and generation of canopy metrics," GitHub, Michigan State University, 2015. [Online]. Available: <https://github.com/MeganKress/LiVoxGen>. [Accessed: Jul. 2026].
- [27] A. Peytavie, E. Galin, J. Grosjean, and S. Mérillou, "Arches: A framework for modeling complex terrains," *Comput. Graph. Forum*, vol. 28, no. 2, pp. 457-467, 2009, doi: 10.1111/j.1467-8659.2009.01385.x.
- [28] K. Fujiwara, R. Tsurumi, T. Kiyono, Z. Fan, X. Liang, B. Lei, W. Yap, K. Ito, and F. Biljecki, "VoxCity: A seamless framework for open geospatial data integration, grid-based semantic 3D city model generation, and urban environment simulation," *Comput. Environ. Urban Syst.*, vol. 123, Art. 102366, 2026 (online 2025; volume year 2026), doi: 10.1016/j.compenvurbysys.2025.102366 (arXiv:2504.13934).
- [29] V. Jaillot, F. Pedrinis, S. Servigne, and G. Gesquière, "A generic approach for sunlight and shadow impact computation on large city models," in *Proc. 25th Int. Conf. in Central Europe on Computer Graphics, Visualization and Computer Vision (WSCG 2017)*, Plzeň, Czech Republic, 2017. [Online]. Available: <https://hal.science/hal-01559175>. [Accessed: Aug. 2026].
- [30] Institut national de l'information géographique et forestière (IGN), "FLAIR-HUB: Multi-source semantic segmentation model for land cover mapping," GitHub, 2023. [Online]. Available: <https://github.com/IGNF/FLAIR-HUB>. [Accessed: Aug. 2026].
- [31] C. R. Harris *et al.*, "Array programming with NumPy," *Nature*, vol. 585, no. 7825, pp. 357-362, Sep. 2020, doi: 10.1038/s41586-020-2649-2.
- [32] laspy Development Team, *laspy: A Python library for reading, modifying, and writing LAS files*, 2023. [Online]. Available: <https://github.com/laspy/laspy>. [Accessed: Jul. 2026].
- [33] G. M. Morton, "A computer oriented geodetic data base and a new technique in file sequencing," IBM Ltd., Ottawa, ON, Canada, Tech. Rep., 1966.
- [34] J. Amanatides and A. Woo, "A fast voxel traversal algorithm for ray tracing," in *Proc. Eurographics*, Amsterdam, Netherlands, 1987, vol. 87, no. 3, pp. 3-10.
- [35] A. Beer, "Bestimmung der Absorption des rothen Lichts in farbigen Flüssigkeiten," *Annalen der Physik und Chemie*, vol. 86, pp. 78-88, 1852.
- [36] M. Monsi and T. Saeki, "Über den Lichtfaktor in den Pflanzengesellschaften und seine Bedeutung für die Stoffproduktion," *Japanese Journal of Botany*, vol. 14, pp. 22-52, 1953. English translation: *Annals of Botany*, vol. 95, no. 3, pp. 549-567, 2005, doi: 10.1093/aob/mci052.
- [37] G. S. Campbell and J. M. Norman, *An Introduction to Environmental Biophysics*, 2nd ed. New York, NY, USA: Springer, 1998, doi: 10.1007/978-1-4612-1626-1.

- [38] W. Verhoef, "Light scattering by leaf layers with application to canopy reflectance modelling: the SAIL model," *Remote Sensing of Environment*, vol. 16, no. 2, pp. 125-141, 1984, doi: 10.1016/0034-4257(84)90057-9.
- [39] X. Li and A. H. Strahler, "Geometric-optical modeling of a conifer forest canopy," *IEEE Trans. Geosci. Remote Sens.*, vol. GE-23, no. 5, pp. 705-721, 1985, doi: 10.1109/TGRS.1985.289389.
- [40] T. Nilson, "A theoretical analysis of the frequency of gaps in plant stands," *Agricultural Meteorology*, vol. 8, pp. 25-38, 1971, doi: 10.1016/0002-1571(71)90092-6.
- [41] National Oceanic and Atmospheric Administration, Global Monitoring Laboratory, "NOAA Solar Calculator," 2024. [Online]. Available: <https://gml.noaa.gov/grad/solcalc/>. [Accessed: Jul. 2026].
- [42] J. Meeus, *Astronomical Algorithms*, 2nd ed. Richmond, VA, USA: Willmann-Bell, 1998.
- [43] I. Reda and A. Andreas, "Solar position algorithm for solar radiation applications," *Solar Energy*, vol. 76, no. 5, pp. 577-589, 2004, doi: 10.1016/j.solener.2003.12.003.
- [44] M. Blanco-Muriel, D. C. Alarcón-Padilla, T. López-Moratalla and M. Lara-Coira, "Computing the solar vector," *Solar Energy*, vol. 70, no. 5, pp. 431-441, 2001, doi: 10.1016/S0038-092X(00)00156-0.
- [45] J. J. Michalsky, "The Astronomical Almanac's algorithm for approximate solar position (1950-2050)," *Solar Energy*, vol. 40, no. 3, pp. 227-235, 1988, doi: 10.1016/0038-092X(88)90045-X.
- [46] P. Jaccard, "Étude comparative de la distribution florale dans une portion des Alpes et du Jura," *Bulletin de la Société Vaudoise des Sciences Naturelles*, vol. 37, pp. 547-579, 1901. English statement of the same coefficient: "The distribution of the flora in the alpine zone," *New Phytologist*, vol. 11, no. 2, pp. 37-50, 1912, doi: 10.1111/j.1469-8137.1912.tb05611.x.

Annexes

Ces annexes portent le matériau qui étaye le corps du texte sans l'interrompre : les algorithmes détaillés du chapitre 4 (A), la référence des lignes de commande et des modules (B), les données d'expérience et le format de sérialisation du chapitre 5 (C), des figures supplémentaires (D), la table de classification des entrées (E), et le glossaire (F).

Annexe A - Algorithmes détaillés et pseudo-code

Décisions d'implémentation de la voxelisation (section 4.2.2), validation analytique des ombres (section 4.4.6) et mécanique de la variante v2 (section 4.2.4), déplacées du corps du texte :

Deux décisions de niveau implémentation façonnent le comportement de l'algorithme à l'échelle. L'origine de la grille est fixée une fois par zone, au coin aligné sur la grille de la boîte englobante demandée, avec l'origine verticale au minimum global des en-têtes de tuiles, de sorte que toutes les tuiles se quantifient contre la même origine et que leurs structures se concatènent sans rééchantillonnage. Et la lecture est disponible en une variante par blocs qui diffuse le LAZ en blocs à mémoire bornée et voxelise de façon incrémentale ; il est vérifié que la voie par blocs et la voie fichier entier produisent des structures identiques (cette équivalence fait partie de la suite de tests décrite au chapitre 5), ce qui autorise la chaîne de zone à choisir sa stratégie de lecture sur des considérations purement mémoire.

Validation. Au-delà des vérifications solaires ci-dessus, la géométrie des ombres est elle-même validée analytiquement : une tour de 20 m sur terrain plat projette une ombre de hauteur / $\tan(\text{élévation})$ le long de l'azimut antisolaire. Mesurée contre cette forme fermée à des élévations de 20, 30 et 45 degrés et à deux azimuts, la frontière d'ombre calculée s'accordait à 0,36 m près, moins d'une cellule horizontale.

Mécanique v2 : le comblement fusionne « arbre, branche manquée, arbre » en une seule plage (deux cellules à 0,5 m de résolution verticale), n'invente jamais un changement de classe, et est la même fusion que l'option de regroupement d'intervalles en production (store.grouped, section 5.2.1), exécutée à une autre tolérance de lacune.

Borne de plafond exacte (section 4.4.2) : preuve, implémentation et vérification.

La preuve découle de la règle propre au décodeur (decoder.classify_gap). Un voxel de vide n'est signalé comme intérieur opaque que lorsque chaque voisine pénétrée est du bâtiment à cette hauteur exacte. Au-dessus du voxel occupé le plus haut d'une voisine, cette voisine est elle-même un vide, donc elle n'est pas du bâtiment, donc la condition échoue et le voxel est de l'air mesuré. L'état de sous-sol ne peut pas s'appliquer non plus, puisqu'il exige que le voxel se trouve sous l'indice de sol. La borne est donc un théorème sur le décodeur, non une tolérance.

Elle est implémentée comme voxelizer/ray_columns.CeilingDDA, une sous-classe qui laisse le marcheur de référence intact. Deux cas sont distingués, car un seul est valide en général : pour un rayon strictement vertical, la colonne ne change jamais et le plafond local prédomine ; pour un rayon oblique, la colonne change à mesure que le rayon avance et seul le maximum sur toute la structure est sûr. Le raccourci se désactive de lui-même chaque fois que la configuration changerait le sens de « rien au-dessus » (air traité comme solide, ou colonnes absentes traitées comme solides).

La dernière ligne du Tableau 12, en annexe C, est la boucle interne de la projection d'ombres. Une position solaire sur un million de colonnes passe d'environ 8,2 heures à un peu plus de 2 minutes, et un arc journalier de quarante positions d'environ une quinzaine de jours à environ 1,5 heure. Les deux projections valent pour des rayons verticaux, qui conservent le plafond local par colonne ; un rayon oblique quitte sa colonne d'origine et doit utiliser le plafond plus faible valable sur toute la structure, et la section 4.4.6 donne le coût mesuré dans ce régime. La ligne « depuis le sol » à 1,0 m ne montre aucun gain parce que la référence se terminait déjà sur le premier voxel solide ; la borne ne peut aider

qu'un rayon qui traverserait sinon de l'air. La Figure 21, en annexe D, trace les deux marcheurs en fonction de la taille de cellule verticale, sur la même exécution de mesure.

Vérification. `tests/test_ray_columns_equivalence.py` vérifie des résultats identiques à ceux du marcheur de référence pour 2 700 rayons tirés au hasard, 900 à chacune de trois résolutions verticales, chacun borné à une marche de 40 m, plus des cas écrits pour dépasser la borne plutôt que pour la confirmer : rayons partant exactement au plafond, juste en dessous et juste au-dessus ; configurations qui doivent la désactiver ; colonnes isolées sans voisines ; et rayons descendants, qui ne doivent jamais l'emprunter. Les dix tests passent avec succès.

Transmittance sensible aux classes (section 4.4.3) : paramétrage de la loi et invariance en résolution.

`voxelizer/transmittance.py` remplace le booléen par une transmittance dans $[0, 1]$ accumulée le long du rayon par la loi de Beer-Lambert [35]. Son application à une canopée végétale suit Monsi et Saeki [36], dont la formulation de l'extinction lumineuse dans les couverts reste la référence. La forme implémentée ici est celle de l'extinction volumique : le coefficient porte des unités d'inverse de mètre et multiplie une longueur de trajet géométrique, là où le coefficient de Monsi et Saeki est sans dimension et multiplie un indice de surface foliaire ; [36] soutient la forme de la loi plutôt que ce paramétrage. La dépendance du coefficient d'extinction à la structure du couvert est traitée comme chez Campbell et Norman [37]. Deux familles plus riches ont été envisagées puis rejetées : les modèles complets de transfert radiatif tels que SAIL [38] exigent des distributions d'angles foliaires et des propriétés optiques que la géométrie LiDAR ne fournit pas, et les modèles géométriques-optiques [39] exigent une géométrie explicite de la canopée que la structure en colonnes ne conserve pas par conception. Le raisonnement sur la fraction de trouées qui rend le coefficient estimable à partir de la transmission observée est celui de Nilson [40], et c'est ce que fait la section 4.4.5 avec les retours de sol. La loi elle-même s'écrit

Longueur de trajet en mètres. Compter les voxels ferait dépendre la réponse de la grille au lieu de la géométrie : un rayon oblique traverse plus de voxels qu'un rayon vertical pour la même épaisseur de canopée, et diviser par deux la taille de cellule verticale double le compte sans changer la physique. L'implémentation normalise donc la direction en longueur unité et accumule la véritable longueur de trajet par classe. La distance paramétrique est ainsi en mètres. Un rayon à 45 degrés accumule correctement $\sqrt{2}$ mètre par mètre de hauteur.

Cette propriété est vérifiée par un test automatisé : `test_resolution_invariance` construit la même scène physique (2 m de sol, 10 m de canopée de 4 m à 14 m) à des tailles de cellule verticale de 1,0, 0,5, 0,25 et 0,1 m et exige que les transmittances obtenues s'accordent à 0,02 près. Il s'exécute à $k = 0,1$ et $k = 0,3$, qui placent la transmittance à travers ces 10 m près de 0,37 et de 0,05, si bien que la tolérance est serrée face à la première et lâche face à la seconde. Une implémentation comptant les voxels passe tous les autres tests du fichier et échoue à celui-là, ce pourquoi il est là. Quinze tests au total, dont une vérification en forme fermée que τ vaut $\exp(-k L)$ pour une épaisseur connue, et une vérification d'équivalence montrant qu'une configuration purement opaque reproduit exactement l'intersection ou l'absence d'intersection du marcheur booléen.

Cette annexe donne les algorithmes du chapitre 4 en pseudo-code compact. Toute boucle écrite ici pour la clarté est implémentée comme une passe numpy vectorisée ; aucune boucle Python par point ou par colonne n'existe sur le chemin de construction. Le marcheur de rayons de A.3 est l'exception, en Python par voxel par construction. A.1 est le voxeliseur principal (section 4.2.2), A.2 l'indice de sol et le décodeur d'espaces vides (sections 4.2.3 et 4.2.4), A.3 le marcheur de rayons sur colonnes (section 4.3.5), A.4 les règles d'absorption et A.5 le résolveur de recouvrements (section 4.2.4), et A.6 la construction en flux de la pyramide de LOD (section 4.3.4).

A.1 Voxelisation en deux étapes

```
fonction VOXELIZE(points, cell_xy, cell_z, origin):  
# étape 1: quantification et réduction (vectorisé: tri + réduction ;  
l'implémentation dérive les indices des coordonnées entières brutes du LAS et  
des facteurs d'échelle de l'en-tête, la forme flottante ci-dessous étant la  
définition mathématique)  
ix = floor((x - x0) / cell_xy); iy = floor((y - y0) / cell_xy); iz = floor((z  
- z0) / cell_z)  
trier les tuples (ix, iy, iz, cls) lexicographiquement  
réduire les tuples égaux en cellules uniques avec sommation des comptages de  
points  
# étape 2: codage par plages vertical  
pour chaque plage maximale de cellules de (ix, iy, cls) égaux et d'iz  
consécutifs:  
émettre l'intervalle (iz_first, iz_last + 1, cls, somme des comptages)  
retourner le ColumnStore construit à partir de la liste d'intervalles
```

A.2 Indice de sol et classification des vides à trois états

```
fonction GROUND_INDEX(store): # une seule passe vectorisée en pratique  
pour chaque colonne c:  
g = intervalle le plus haut de c dont la classe est GROUND  
ground[c] = g.z_end si g existe sinon NODATA  
  
fonction CLASSIFY_GAP(colonne c, hauteur iz):  
si ground[c] == NODATA: # colonne à toit seul ou à canopée seule  
retourner MEASURED_AIR au-dessus du retour le plus haut de c, sinon  
OPAQUE_INTERIOR  
si iz < ground[c] - SUBSURFACE_BAND: retourner SUBSURFACE # SUBSURFACE_BAND =  
2 voxels ; la bande juste sous l'ancrage retombe sur les règles ci-dessous  
si c a au moins une voisine orthogonale pénétrée et que toute voisine pénétrée  
est du bâtiment à iz:  
retourner OPAQUE_INTERIOR # atténuation du faisceau nadiral  
retourner MEASURED_AIR
```

A.3 Parcours de rayons sur la grille de colonnes

```
fonction TRACE(origin, direction, config): # DDA 3-D d'Amanatides-Woo  
(ix, iy, iz) = voxel contenant origin  
initialiser les pas par axe et les distances à la prochaine frontière tMaxX,  
tMaxY, tMaxZ  
tant que t <= config.max_t:  
state = classe de (ix, iy, iz): intervalle occupé s'il y en a un,  
sinon CLASSIFY_GAP / sous-sol / air mesuré depuis le décodeur  
si config déclare state solide: retourner hit(position, classe, t)  
avancer l'axe (x, y ou z) dont la prochaine frontière est la plus proche  
retourner no_hit
```

A.4 Règles d'absorption de la végétation

```
pour chaque plage de végétation r dans la colonne c: # passe d'absorption  
si r n'est pas prise en sandwich verticalement par du bâtiment: conserver #  
vrai arbre, arbuste  
sinon si r est un bruit court (au plus 2 voxels): absorber dans le bâtiment
```

```
sinon si moins de 75 % des 8 voisines pénétrées sont du bâtiment aux hauteurs
de r, ou si aucune n'est pénétrée: conserver
sinon: absorber, sauf si au moins 2 directions cardinales n'ont pas de
bâtiment à ces hauteurs # échappatoire arbre de balcon
```

A.5 Résolution des recouvrements

```
pour chaque voxel portant des intervalles superposés de classes différentes: #
résolveur
votes = comptage de points par classe
si une classe l'emporte au-delà des seuils d'égalité: l'attribuer
sinon: déclasser les classes incertaines, préférer la structure la plus haute,
puis le plus petit code
chaque segment survivant garde la part du comptage de sa propre plage que
représente sa hauteur;
les points de la classe perdante sont écartés, non transférés au gagnant
```

A.6 Construction en flux de la pyramide de LOD

```
fonction BUILD_PYRAMID(leaves):
trier les feuilles par MORTON(i, j) # clé à bits entrelacés: les descendants
de tout nœud du quadtree
# forment une plage contiguë à chaque niveau de l'arbre
pour chaque feuille dans cet ordre:
pour le niveau L = 1 .. sommet: si le nœud ouvert en L a changé: FLUSH(L)
écrire le GLB de la feuille; la grossir d'un facteur 2 dans le nœud ouvert de
niveau 1
pour le niveau L = 1 .. sommet: FLUSH(L) # vidange: les enfants strictement
avant les parents
fonction FLUSH(L):
concaténer les parties du nœud ouvert, triées par identité d'enfant # contenu
indépendant du parcours
écrire le GLB du nœud; enregistrer son volume englobant
grossir le nœud d'un facteur 2 dans son parent en L + 1; le libérer
# un nœud ouvert par niveau: le pic de mémoire suit la profondeur de l'arbre,
non l'étendue de la zone
```

Réserve du faisceau nadiral (section 4.4.1): mesures d'angles et de bande suspecte à l'appui de l'atténuation.

Les angles de balayage enregistrés couvrent ± 30 degrés, chiffre établi à partir des mesures du champ d'angle de balayage sur les 2 842 tuiles de la livraison, et non d'après une fiche technique de scanner (la mesure est la section 6 de tests/diagtests/genstats/LAZ_STATS.md: le plus petit minimum par tuile vaut -30,0 et le plus grand maximum par tuile +30,0, soit l'enveloppe complète et non un centile de la distribution); un faisceau qui renvoie du sol en (x, y) a traversé de l'air situé jusqu'à $\tan(30 \text{ degrés}) \approx 0,58$ m latéralement par mètre de hauteur à l'écart de cette colonne, ce qui représente plusieurs colonnes aux hauteurs d'arbres typiques et des dizaines aux hauteurs de tours. Le mode de défaillance visible est un « puits de lumière » d'air déclaré fantôme immédiatement à côté des bâtiments hauts (Figure 9), qui ferait fuir de la lumière dans les zones d'ombre et biaiserait les facteurs de vue du ciel le long des façades.

L'argument confortable serait que le recouvrement des bandes et la géométrie à deux scanners fassent provenir les retours de sol en terrain plat très majoritairement de faisceaux proches du nadir. L'essentiel de l'air traversé serait alors attribué à la colonne réellement balayée; une passe par point sur quatre tuiles adjacentes a depuis éprouvé cet argument sur les données et ne le soutient pas, les retours de sol situés d'une à six cellules d'un bâtiment ayant un angle absolu moyen de 12,87 degrés contre 13,17 degrés au-delà de quarante cellules, et 74 % de ceux qui bordent immédiatement un bâtiment arrivant à

plus de 5 degrés du nadir. La largeur de la bande suspecte bordant l'arête d'un bâtiment a depuis été mesurée sur un kilomètre carré, en joignant les angles de balayage des retours proches des arêtes aux hauteurs de bâtiment de la même structure : elle vaut 1,00 m à la médiane et 2,46 m au neuvième décile, soit environ 0,45 fois la hauteur du bâtiment. La règle empirique de trois à quatre colonnes que portaient les notes de conception tient à ces hauteurs et échoue en tant que nombre fixe de colonnes.

Post-traitements de la section 4.2.4 : règles détaillées.

Trois post-traitements autonomes complètent les variantes par colonne, et tous trois sont des exceptions bornées à la règle d'indifférence au voisinage : ce sont des opérations par lots qui consultent temporairement des voisinages, puis rendent une structure aussi libre de voisinage qu'auparavant. Le résolveur de recouvrements réduit les intervalles superposés de classes différentes (les voxels mixtes de la construction conservatrice) à une étiquette par voxel, par vote majoritaire tenant compte des quasi-égalités : chaque segment survivant conserve la part du comptage de sa propre plage que représente sa hauteur, et les points de la classe perdante sont écartés et ne sont pas transférés à la classe retenue, ce qui coûte 6,33 % des comptages de points sur la structure de référence (chiffres en annexe C, règles en annexe A.5).

La passe d'absorption reclassifie les plages de végétation prises en sandwich verticalement et enfermées horizontalement par du bâtiment ; de la verdure poussant apparemment à l'intérieur d'un toit est presque toujours faite de retours de façade ou d'erreurs de classification. Une clause d'échappement s'applique lorsque deux directions cardinales ou plus sont ouvertes, de sorte que cours intérieures et toitures végétalisées survivent. Enfin, deux filtres de débruitage (un seuil sur le nombre de points et un test morphologique de voisin de même classe) éliminent le bruit d'intervalles isolés.

L'ordre dans lequel ces post-traitements s'exécutent n'est pas neutre : deux séquences qui ne diffèrent que par leur ordre ne convergent pas, et la passe d'absorption élargit leur désaccord dans les quatre cas mesurés, de +5,8 % à +23,5 % de colonnes divergentes, sur deux zones et deux résolutions, les points étant conservés exactement de part et d'autre (annexe C). Toute grandeur dérivée d'une structure post-traitée doit donc porter l'ordre dont elle provient.

Annexe B - Référence CLI, modules et configuration

Rôles des dépendances (section 4.3.1) : numpy ; laspy [32] avec le moteur LASzip [6] pour les entrées/sorties LAZ et lazrs comme décodeur de second avis derrière la reprise isolée des défaillances ; matplotlib et Pillow pour le rendu 2-D ; psutil pour le chien de garde mémoire et les diagnostics ; pyproj pour la validation du système de coordonnées et le géoréférencement 3D Tiles ; pygltflib pour l'export 3D Tiles ; PyVista et tkinterdnd2 en extras optionnels pour la 3-D interactive et le glisser-déposer.

Internes de la structure et persistance (section 4.3.2), tels qu'implémentés :

La disposition en structure de tableaux de la section 4.2.1 est réalisée par six vecteurs numpy plats, les clés de colonnes étant empaquetées en un seul uint64 par colonne afin que le tri et la fusion opèrent sur un unique tableau de type primitif. Les facteurs d'échelle sont incorporés aux constantes de quantification une fois par tuile, de sorte que les indices de voxels sont calculés directement à partir des coordonnées entières brutes stockées dans l'enregistrement LAS, ce qui évite un aller-retour (représentation flottante) sur des dizaines de millions de points. La persistance a deux formes aux rôles distincts : une archive .npz compressée pour l'échange et l'archivage, et un répertoire brut contenant chaque tableau dans un fichier séparé pour la chaîne de traitement elle-même. La forme brute existe à cause de l'isolation des étapes : les processus fils projettent les tableaux en mémoire en lecture seule. Les étapes de sortie peuvent s'exécuter sur une même structure sans en faire de copies, et une étape défaillante laisse la structure intacte sur le disque.

Mécanismes des voies de visualisation 3-D et de l'export 3D Tiles (section 4.3.4) :

La première est un visualiseur HTML autonome : les boîtes d'intervalles sont exportées comme géométrie instanciée dans une scène Three.js, un enregistrement compact par boîte, si bien qu'un fichier unique s'ouvre dans n'importe quel navigateur sans serveur ni installation ; un amincissement automatique maintient le nombre de boîtes dans ce qu'un navigateur tolère. La charge utile de voxels voyage à l'intérieur de la page, mais les deux modules Three.js (et une solution de repli de décompression pour les navigateurs dépourvus de DecompressionStream) sont récupérés depuis un CDN, si bien que la première ouverture demande une connexion réseau.

La deuxième est un export PLY pour CloudCompare, MeshLab ou Blender, et une fenêtre PyVista optionnelle pour l'inspection interactive pendant le développement ; les trois moteurs de rendu partagent une même passe de collecte de géométrie, si bien qu'une correction dans le collecteur bénéficie à toutes les voies.

La troisième, le visualiseur en flux, existe parce que la première voie a une limite : à résolution fine, une structure de zone peut contenir des centaines de millions d'intervalles, et aucun navigateur ne charge cela en une seule scène.

L'exportateur en flux parcourt donc la structure une bande de rangée de tuiles à la fois (la mémoire reste bornée quelle que soit la taille de la structure) et écrit une charge utile binaire d'enregistrements d'instances de 32 octets fixes à côté d'un index spatial ; le visualiseur traite le budget d'instances configuré comme un ensemble de travail GPU, en gardant les tuiles proches de la caméra résidentes en pleine finesse pendant que les tuiles lointaines s'effondrent en silhouettes simplifiées.

Le chapitre 5 relate la défaillance mémoire qui a imposé cette conception, et le traitement vectorisé des clés qui est sorti de l'autopsie. Un quatrième exportateur vise l'interopérabilité plutôt que la visualisation : il convertit une structure ou une charge utile en flux existante en un tileset 3D Tiles 1.1 [9], avec translation et échelle par instance portées par l'instanciation GPU, de sorte que la ville de voxels puisse être chargée par tout client géospatial conforme aux standards à côté des autres couches d'IA.rbre. Les altitudes des tuiles sont orthométriques alors que la transformation du tileset demande des hauteurs ellipsoïdales : l'exportateur applique donc par défaut l'ondulation du géoïde issue de la grille de l'IGN, environ +49,70 m près de Lyon ; si la grille ne peut pas être récupérée, l'export émet un avertissement et le modèle se place une cinquantaine de mètres trop bas. L'exportateur construit ainsi une pyramide en quadtree de niveaux de détail : les tuiles feuilles de 100 m portent le détail complet, les nœuds internes portent une géométrie réellement sous-échantillonnée à taille double de voxels, et la cascade d'erreurs déclarée permet à tout client de récupérer un aperçu d'environ 10 MB d'une zone de 3 x 3 km avant de ne diffuser du détail que là où la caméra regarde.

Par construction, de petites superpositions peuvent apparaître aux niveaux les plus bas.

La pyramide est construite en flux : les feuilles sont parcourues dans l'ordre de Morton [33], dans lequel les descendants de tout nœud du quadtree sont contigus à chaque niveau. Chaque nœud interne est écrit dès l'arrivée de son dernier « enfant » puis libéré – le pic de mémoire suit la profondeur de l'arbre plutôt que l'étendue de la zone, et diviser par deux la taille des feuilles le laisse inchangé. La sortie passe le validateur officiel 3D Tiles sans aucune erreur, sous réserve des limites que consigne la section 5.5 : le validateur ignore l'extension EXT_mesh_gpu_instancing, laquelle porte toute la géométrie, et il ne vérifie pas du tout l'inclusion des volumes englobants. Les Figures 18 à 20, reportées en annexe D, représentent les quatre voies telles que le code les implémente : la passe de géométrie que les deux premières partagent, la charge utile en flux et le visualiseur qui la charge tuile par tuile, et la cascade 3D Tiles.

Le paquet expose ses fonctionnalités par les points d'entrée ci-dessous ; les groupes d'options partagés (tailles de cellule, options de géométrie 3-D) sont factorisés dans un seul module afin que les outils restent cohérents. La taille de cellule par défaut est de 0,5 m au cube partout – la fonction de bibliothèque, les deux outils en ligne de commande et l'interface graphique – tandis que 1,0 m à l'horizontale par 0,5 m à la verticale est ce que recommande la section 5.2 pour les produits d'analyse. Les cinq niveaux de dépendance de la section 4.1 et leurs modules sont documentés dans le README

du dépôt ; les quarante-huit modules vont de la configuration des classes et du lecteur LAZ au socle jusqu'aux consommateurs CLI et GUI au sommet.

Point d'entrée	Objet
python -m voxelizer.single TILE.laz	Voxeliser une tuile : quatre cartes PNG, statistiques, diagnostics de colonnes (3-D optionnelle)
python -m voxelizer.area_cli area --xmin ... --ymax ...	Voxeliser toutes les tuiles intersectant une boîte englobante en une grille fusionnée unique, avec toutes les sorties
... area ... --shard	Même chaîne de zone, out-of-core : pic de mémoire borné par une seule tuile
... area ... --download / --stream --json INVENTORY	Récupérer d'abord les tuiles intersectantes, ou télécharger-voxeliser-supprimer chaque tuile à la volée
... area ... --preflight-only	Prédire la taille de la structure et le pic de mémoire vive à partir des en-têtes, puis quitter
... area ... --resume-from-store PATH	Ré-exécuter les étapes de sortie depuis une structure en cache, sans refaire la voxelisation
python -m voxelizer.viz3d_cli single from-store stream	Visualiseur HTML 3-D depuis un LAZ, depuis une structure enregistrée, ou sous forme de charge utile en flux
python -m voxelizer.tileset_cli from-store from-payload	Exporter un tileset 3D Tiles 1.1 depuis une structure ou une charge utile en flux existante
python -m voxelizer.reconstruct-to-laz to-npz verify	Voxelisation inverse : structure vers LAS/LAZ, LAZ vers une structure, ou vérification d'un aller-retour
python -m voxelizer.serve_voxel_html --open	Servir le dernier visualiseur en flux en HTTP (requêtes Range prises en charge)
python -m voxelizer.download_laz / download_orthos	Télécharger des tuiles LiDAR ou orthophoto depuis l'inventaire du Grand Lyon
python -m voxelizer.gui_area	Interface Tkinter : modes fichier unique et zone, avec le dialogue préalable

Annexe C - Données d'expérience et format de sérialisation

Trois enregistrements de développement sont conservés en bref. Défaut de conservation des encodeurs (section 4.2.4) : perte initiale de 20,59 % des comptages sur structure réelle pour class smooth, ramenée à 6,33 % par la correction, le résidu étant le coût, par conception, de l'arbitrage des chevauchements décrit en annexe A.5 ; la correction du biais par comptages de retours (section 4.4.6) a relevé les heures d'ensoleillement de 0,122 h. Violation d'accès sous charge de rayons : diagnostiquée en trois temps (une allocation par voxel réelle mais innocente, puis le volume combiné des recherches dichotomiques, des milliards de clés par rayon) ; la réutilisation de la colonne courante et la lecture directe des tableaux plats ont ramené les recherches de milliers à une cinquantaine par rayon et divisé le temps par deux, résultats reproduits exactement. Défaillance mémoire du visualiseur, déterminante pour le chapitre 4 : 317 millions d'intervalles ont fait culminer le collecteur de géométrie vers 26 GB (clés en tuples Python, tri en seconde copie) sur une station de 32 GB ; la refonte en flux a suivi. Les campagnes de vérification successives (30/30, 32/32, 41/41, 14/14, 19/19) et le balayage de la section 5.2, quarante étapes de sortie sans échec, tiennent lieu de non-régression.

Le regroupement et le comblement de lacunes (v2) de la section 4.2.4 sont une seule opération dans le code (store.grouped), exécutée à des tolérances de lacune différentes sur des grilles différentes : les -29,5 % du Tableau 4 correspondent à une tolérance de 1 m sur la structure à trois tuiles à 1,0 m à l'horizontale par 0,5 m à la verticale, et les -7,01 % de la section 5.2.1 à une tolérance de deux cellules à 1,0 m au cube sur la Métropole entière. Les deux chiffres mesurent ainsi la même fusion sur des entrées différentes et ne sont pas comparables. L'octet par point repose de même sur trois bases dans ce chapitre : le Tableau 4 divise la sérialisation non compressée de l'expérience (12 octets par colonne plus

13 par intervalle) par les points d'entrée, le Tableau 5 divise le .npz compressé sur le disque, et le Tableau 6 divise la charge utile exacte (16 octets par colonne plus 13 par intervalle). Le Tableau 5, au chapitre 5, résume les huit exécutions archivées.

La tuile dense isolée 18410_51825 affiche 13,69 intervalles par colonne à la résolution la plus fine, un régime différent de celui des zones : la prudence de la section 5.2 s'applique à toute comparaison.

Choix de l'algorithme solaire (section 4.4.4) : comparatif et validation.

L'implémentation (voxelizer/solar.py) suit la formulation du NOAA Solar Calculator [41], qui est l'algorithme de position solaire à faible précision de Meeus [42]. Il est précis à environ 0,01 degré près au voisinage de J2000. Trois solutions de rechange ont été envisagées :

Trois alternatives, NREL SPA [43], PSA [44] et Michalsky [45], offraient une précision supérieure ou comparable, au prix de dix fois plus de code ou de fenêtres temporelles bornées.

NOAA/Meeus se place là où la précision cesse d'importer pour cette application, tandis que la complexité reste assez faible pour être implémentée, lue et vérifiée sans dépendance, ce qui compte parce que les contraintes de conception de la section 2.1 excluent d'ajouter des bibliothèques à la légère. Aucune bibliothèque solaire n'est installée dans l'environnement de travail (pvlib, astral, ephemeris et skyfield sont toutes absentes), si bien que l'alternative à l'implémenter était d'en ajouter une.

Validation contre des faits indépendants du code. La cohérence interne ne prouve rien. Chaque vérification se fait donc contre quelque chose de dérivable sans l'implémentation : l'élévation au midi solaire découle de la latitude et de la déclinaison par l'astronomie sphérique, et la déclinaison ne peut pas dépasser l'obliquité de l'écliptique (Tableau 14, annexe C).

Quatorze tests couvrent cela, dont les extrêmes publiés de l'équation du temps (-14,2 min en février, +16,4 min en novembre), le lever au nord-est et le coucher au nord-ouest, et le fait que la réfraction relève le soleil apparent plutôt qu'elle ne l'abaisse.

Le piège de la convergence des méridiens. Un azimuth solaire se mesure depuis le nord vrai, alors que les axes de la structure de voxels sont des axes de grille projetés. À Lyon, la convergence des méridiens est d'environ 1,3 degré, variable selon la position (1,28 à 1,41 degrés sur les zones étudiées). Fournir un azimuth vrai à un rayon aligné sur la grille fait tourner chaque ombre de cet angle et déplace la pointe d'une ombre d'environ 2,3 m à 100 m. `sun_vector_grid` effectue la conversion explicitement, en dérivant la convergence numériquement par différences finies (la même technique que celle qu'utilise l'exportateur 3D Tiles pour son géoréférencement), parce que les formes fermées invitent à des erreurs de signe et d'hémisphère qui échouent silencieusement.

Une erreur de ce genre est survenue pendant le développement et mérite d'être consignée : les deux branches de la formule d'azimut de la NOAA étaient initialement inversées, ce qui reflétait le soleil par rapport à l'axe nord-sud et faisait annoncer au lever un relèvement nord-ouest. Aucun test d'élévation ne détecte cela ; seule la vérification est/ouest l'a fait.

Dérivation des coefficients d'extinction (section 4.4.5) : détail et limitations.

La loi de transmittance de la section 4.4.3 a laissé ses coefficients non fournis ; les dériver du corpus comble ce manque. Le LiDAR mesure directement la pénétration de la canopée : une impulsion qui atteint le sol à travers une canopée est une impulsion transmise. Pour les colonnes présentant de la végétation au-dessus d'un retour de sol, la fraction p d'impulsions atteignant le sol et l'épaisseur de canopée H donnent $k = -\ln(p) / H$. H est l'épaisseur sommée des intervalles de végétation de la colonne au-dessus du sommet du sol, en voxels multipliés par la taille de cellule verticale. Les trouées internes au houppier sont par conséquent exclues, et H reste inférieure à l'étendue du houppier de sa fraction de trouées. Trois conditions sélectionnent les colonnes, appliquées comme cinq filtres successifs : au moins 2 m d'épaisseur de végétation sommée, au moins 20 retours, et la présence à la fois d'un retour de

sol et d'un retour de végétation. Appliqué à la structure à 0,5 m au cube, cela qualifie 739 923 colonnes, dont la garde de dégénérescence de la dérivation écarte ensuite sept (annexe C) ; le plancher de 2 m écarte précisément les canopées minces dont traite la limitation (b) ci-dessous. Les exclusions ont depuis été comptées une à une : sur les 3 969 467 colonnes de la structure, 3 229 544 sont écartées par les cinq filtres géométriques et aucune par les deux gardes de comptage nul. La partition est ainsi exacte et reproduit les 739 923 colonnes qualifiées, et 291 177 à 1,0 m au cube (annexe C).

L'ordre est physiquement sensé et n'a pas été imposé : l'extinction décroît de façon monotone de la végétation basse à l'étage supérieur de canopée, ce qu'un sous-étage plus dense et un houppier plus clairsemé devraient produire. Les coefficients valent, en 1/m, 0,497 pour la classe 3, 0,429 pour la classe 4, 0,326 pour la classe 5 et 0,250 pour la classe 8, contre 0,311 toutes classes confondues (Tableau 3, au chapitre 4).

(a) Il ne prédit pas les colonnes individuelles. Ajuster sur une moitié tirée au hasard et prédire l'autre moitié donne une erreur relative médiane de 57,7 % sur la fraction de retours de sol, avec une erreur absolue moyenne de 0,120 sur celle-ci : le coefficient ajusté ne prédit pas les colonnes isolées. L'écart interquartile à l'intérieur d'une même classe couvre près d'un facteur deux (classe 5 : 0,233 à 0,463). La lecture défendable est que k est une *distribution*, utilisable comme valeur typique de classe pour un calcul à l'échelle d'une zone, et non une propriété par colonne.

(b) L'estimation dépend de la résolution, ce n'est donc pas encore une grandeur physique pure. Refaire l'ajustement sur la structure à 1,0 m au cube donne des coefficients systématiquement plus bas : la classe 5 tombe de 0,326 à 0,208, la classe 3 de 0,497 à 0,163 (rapports de 1,5 à 3,1 ; la table de coefficients qu'applique le calcul d'heures d'ensoleillement, citée avec la Figure 10 et le Tableau 7, porte la classe 3 à 0,163, le même ajustement transcrit à trois décimales), et à 1,0 m la médiane de la classe 3 tombe au bas des quatre, sous celle de la classe 4 et au niveau de celle de la classe 8 à trois décimales (0,1627 contre 0,1626), la surestimation des canopées minces à son pire. Le mécanisme est direct – H est mesurée en voxels, et une canopée mince remplit un voxel entier de 1 m tout en n'occupant qu'une partie d'un voxel de 0,5 m ; les cellules grossières surestiment l'épaisseur et sous-estiment donc k . Tout coefficient cité doit porter la résolution à laquelle il a été dérivé, et les calculs d'ombrage doivent utiliser le coefficient dérivé à la résolution à laquelle ils s'exécutent.

(c) p était estimée à partir des comptages de retours, et les coefficients ci-dessus portent la correction. La structure conserve des comptages agrégés par intervalle, non l'identité des impulsions, si bien que la première dérivation prenait p comme $n_{\text{ground}} / (n_{\text{ground}} + n_{\text{vegetation}})$. Une impulsion qui pénètre la canopée y compte une fois au numérateur et deux fois au dénominateur, et une impulsion absorbée dans la canopée plusieurs fois au seul dénominateur : les deux dégonflent p et gonflent $k = -\ln(p) / H$, raison pour laquelle ces coefficients étaient publiés comme *bornes supérieures* sur l'extinction véritable. Les impulsions ont depuis été reconstituées depuis le LAZ livré par le couple (identifiant de source, temps GPS) sur l'ensemble du corpus de dérivation, 98 946 764 points en 76 743 324 impulsions, soit 1,289 retour par impulsion, et le Tableau 3, au chapitre 4, est l'ajustement refait : mêmes colonnes, même H , mêmes filtres, p prise par impulsion. Chaque coefficient baisse et aucun ne monte, de 7,4, 6,6, 3,7 et 3,0 % pour les classes 3, 4, 5 et 8 à 0,5 m et de 17,9, 13,6, 7,7 et 6,4 % à 1,0 m, la médiane toutes classes de 4,3 et 8,8 %. L'ordonnancement des classes est inchangé aux deux grilles. Toute affirmation d'ordre ci-dessus survit donc à la correction, et le coefficient par comptages dépasse celui par impulsion sur 99,978 % des colonnes à 0,5 m et 99,953 % à 1,0 m. Sa portée a depuis été fixée en refaisant la dérivation sur l'ensemble de la livraison (campagne du 2026-08-08, run E12_per_pulse_metropolis, archivée hors de cette livraison) : l'ajustement appliqué à 1,0 m est confirmé à 3,3, 1,1, 2,6 et 6,6 % près par classe sur 174,9 millions de colonnes qualifiées, et le rapport de biais par colonne est invariant d'échelle, médiane 1,078 sur la Métropole contre 1,071 ici. Les déplacements agrégés ont pourtant crû, à 5,2 et 11,1 % contre 4,3 et 8,8, pour deux raisons mesurées : au même titre que les 39,05 %, la livraison porte 34,5 % de points à retours multiples (les 19,68 % comptent les retours non premiers, une autre mesure), et la composition de la population qualifiée se déplace, faisant passer à 1,0 m la classe 3 sous la classe 8, le mécanisme de la limitation (b) à l'échelle (annexe C).

(d) Le coefficient est dérivé d'un relevé proche du nadir et appliqué tel quel à des rayons obliques.

H est une épaisseur verticale mesurée à partir d'impulsions descendant pour l'essentiel à la verticale, tandis que l'exécution d'heures d'ensoleillement de la section 4.4.6 envoie le même coefficient tout au long de l'arc journalier ; pour toute distribution d'angles foliaires non sphérique, le coefficient d'extinction dépend lui-même de l'angle zénithal du faisceau, dépendance pour laquelle Campbell et Norman [37] est cité, et aucune correction angulaire n'est appliquée.

La Figure 22, en annexe D, montre la distribution ajustée par classe. La dérivation par comptages est reproductible depuis le dépôt : script `Experiments/derive_extinction.py`, ajustement `Experiments/extinction_fit.json` ; l'ajustement par impulsion qui le remplace provient de la campagne du 2026-08-08 (run `E10_per_pulse_refit_scopeA`), archivée hors de cette livraison.

Heures d'ensoleillement direct (section 4.4.6) : règles d'origine des rayons, correction du plafond oblique et incident natif.

Pour chaque colonne et chaque position solaire au-dessus de l'horizon, un rayon est lancé depuis 1,5 m au-dessus du sommet de l'intervalle le plus bas de la colonne en direction du soleil, et la transmittance est accumulée selon $\text{sun_hours} = \text{sum}(\text{tau} * \text{dt})$. Là où une colonne possède un retour de sol, cette origine est à 1,5 m au-dessus du sol, à peu près la hauteur des yeux ; là où elle n'en a pas – typiquement sous un toit – elle est à 1,5 m au-dessus de la surface mesurée la plus basse. Ces colonnes rapportent donc des heures d'ensoleillement au niveau des toits plutôt qu'au niveau de la rue. La règle peut aussi se fixer par le bas : lorsque l'intervalle le plus bas d'une colonne est une valeur aberrante basse de classe 7 sous le terrain, l'origine l'y suit, et l'indice de sol que la structure porte déjà n'est pas consulté. Calculer cela deux fois – une fois avec tous les matériaux opaques, une fois avec les coefficients de la section 4.4.5 – isole ce qu'apporte le modèle volumétrique des matériaux. Aucune des deux cartes ne répond seule à l'objectif (iv) ; c'est la différence entre elles qui y répond.

Ces chiffres découlent de la correction du plafond oblique appliquée à la marche de transmittance. La marche reçoit désormais le plafond valable sur toute la structure, valide dans toutes les directions, à la place du plafond local de la colonne d'origine, qu'un rayon oblique quitte avant d'avoir pu rencontrer la géométrie qui se dresse plus loin sur sa route. Chaque valeur absolue d'heures d'ensoleillement baisse d'une heure et demie à deux heures, aucune des 48 000 paires colonne-modèle échantillonnées n'a augmenté, et la différence que l'expérience existe pour mesurer conserve son signe et son ordonnancement (`Experiments/sun_hours_ceilingfix/`). Ce plafond fixe aussi le prix de l'exécution. La même charge de 756 000 rayons a été reprise trois fois, avec un résultat par colonne identique élément par élément à chaque reprise : la moyenne est de 1 165,2 s pour 12 000 colonnes à 63 positions solaires, la dispersion de 2,0 % de cette moyenne, soit 1,54 ms par rayon, coût établi sous les coefficients d'origine de la dérivation. À ce taux d'origine de 1,54 ms, une position solaire sur les 967 432 colonnes de la zone coûte environ 25 minutes dans ce régime, et l'arc journalier entier environ 26 heures, plutôt que les deux minutes par position que donne la projection à rayons verticaux de la section 4.4.2 (annexe C) ; sous l'ajustement par impulsion appliqué, la même marche mesure 1,86 et 1,67 ms par rayon sur la zone végétalisée (deux répétitions), la zone bâtie coûtant environ le tiers (0,57 à 0,63 ms), une extinction plus faible laissant les rayons porter plus loin. L'arc sur la zone végétalisée est ainsi plus proche de 31 heures.

Le récit complet de cette défaillance native et de ses correctifs, à résultats prouvés identiques et temps d'exécution divisé par deux, figure plus haut dans cette annexe, parmi les récits d'incidents du chapitre 5 ; le constat matériel apparu plus tard et la mesure de ceinture et bretelles pour les exécutions à l'échelle de la Métropole sont consignés à la section 5.4.

Modes d'export LAS/LAZ, protections et étude de coût (section 4.3.7) :

La section 4.3.7 résume les trois modes d'export (`density`, `one_per_voxel`, `exact`) et leur garantie ; ce qui suit consigne le détail que le corps omet.

En mode exact, densité de points et géométrie des voxels survivent toutes deux. Il émet un point par voxel de l'intervalle, restaurant l'occupation et la classe, plus $\text{count} - (z_{\text{end}} - z_{\text{start}})$ points supplémentaires stockés dans le premier voxel de l'intervalle, restaurant le comptage.

Ce placement du remplissage a l'air arbitraire et est en réalité le seul que les données autorisent : le voxeliseur somme les comptages de points par intervalle et jamais par voxel. La position du remplissage dans l'intervalle est inobservable pour une re-voxelisation, et la structure n'a jamais détenu la distribution d'origine par voxel qui permettrait de les répartir. Les stocker à la base de l'intervalle est le choix correct le moins coûteux et, contrairement à une répartition uniforme, la convention est explicitée afin de ne pas présenter une valeur construite comme une observation.

Aux deux protections résumées en 4.3.7 (la grille voyage dans le fichier ; seules les structures brutes sont archivées) s'ajoute la vérification par fragment, seconde ligne de défense, et la suppression d'une structure source est refusée tant que cette vérification n'est pas activée. Cinquante tests répartis entre `tests/test_reconstruct_roundtrip.py` et `tests/test_archive_cli.py` assurent la conformité et chacune des protections qui l'entourent, y compris les cas négatifs : une structure assemblée à la main qui n'est pas l'encodage canonique de son propre ensemble de cellules doit échouer, car l'exactitude est une propriété de ce que le voxeliseur émet et non de toute structure bien formée.

Deux détails d'en-tête méritent d'être consignés parce que tous deux étaient des défauts réels. Le module d'écriture émet un système de coordonnées de référence, ce qui ressemble à de l'intendance et n'en est pas : le propre lecteur LAZ du paquet rejette les fichiers qui n'en ont pas, si bien qu'avant l'existence de ce paramètre, rien de ce que produisait la voie de reconstruction ne pouvait être réinjecté dans la chaîne de traitement. Et le champ de classification ne fait que cinq bits dans les formats de points LAS 0 à 5 : les codes personnalisés de l'IGN 64 à 67 débordaient à l'intérieur de la bibliothèque LAS sans lever quoi que ce soit qu'un appelant puisse lire. Les fichiers dont les classes dépassent 31 sont désormais promus en LAS 1.4 et format de points 6, où la classification occupe un octet complet, et un test épingle aussi le cas négatif, exigeant que des données ordinaires du Grand Lyon, dont la plage de classes déclarée s'étend de 0 à 22, ne changent pas de format de manière involontaire.

La quantification des coordonnées est un accident plus heureux : les échelles restent au millimètre avec le décalage placé à la coordonnée minimale, un centre de voxel pour un nuage reconstruit. Chaque coordonnée exportée diffère donc du décalage d'un multiple exact de la taille de cellule, et l'erreur d'arrondi du pire cas, d'environ un demi-millimètre, se tient face à une marge d'un quart de mètre jusqu'à la frontière de voxel la plus proche. La quantification au millimètre est donc sans perte pour ce contenu, et non simplement tolérable, ce pourquoi la conversion dans les deux sens survit à un vrai fichier LASzip et pas seulement à une expansion en mémoire.

Reste le coût. L'intuition selon laquelle un LAZ exact doit être plus gros que la structure qu'il encode – il nomme après tout chaque voxel individuellement – a tenu pour la première mesure effectuée et s'est avérée être une propriété de cette mesure plutôt que du format. Généraliser depuis une seule tuile à une seule résolution conduirait à conclure que l'archive n'économise rien ; c'est vrai pour cette tuile à cette résolution, et faux partout ailleurs. Le Tableau 10 donne l'image corrigée : trois tuiles réelles de l'inventaire du Grand Lyon à 2 842 tuiles, la plus dense de la livraison et les deux plus creuses, chacune voxelisée à trois résolutions, la taille d'archive étant exprimée comme un multiple de la structure qu'elle remplace. Chacune des neuf cases a de plus été vérifiée comme la conversion exacte au bit près. Le Tableau 10, reporté en annexe C, mesure ainsi le seul coût et non un compromis contre la fidélité.

Le rapport bouge avec la résolution parce que les deux artefacts sont caractérisés par des grandeurs différentes. La structure coûte un montant à peu près fixe par intervalle, et l'unité est ici l'octet compressé sur le disque et non l'enregistrement de 13 octets en mémoire : sur la plus grande tuile, le `.npz` se tient à 3,66, 3,60 et 4,00 octets par intervalle sur les trois résolutions. Sa taille suit de très près le nombre d'intervalles, et le nombre d'intervalles est ce qu'une grille plus fine multiplie – 1,34 million, puis 5,67 millions, puis 25,9 millions sur cette tuile, soit des facteurs de 4,2 et 4,6.

L'archive ne suit pas. Sur les deux mêmes pas de raffinement, elle ne croît que d'un facteur respectif de 2,9 et 2,8, parce que le nombre de points qu'elle doit écrire est fixé par la tuile source et ne change jamais : raffiner la grille n'ajoute pas de points, cela ne fait que convertir des doublons de remplissage en centres de voxels distincts. Les doublons ne coûtent presque rien, environ 0,02 octet chacun, puisque le prédictor du compresseur code des points consécutifs identiques à quasiment zéro, et les centres de voxels qu'ils deviennent sont eux-mêmes consécutifs et prédits à bas prix. Le coût de la structure grossit donc avec la grille alors que celui de l'archive non, et les deux courbes se croisent.

À 0,25 m à l'horizontale et 0,1 m à la verticale, l'archive était plus petite que la structure sur les trois tuiles, par des facteurs de 1,5 à 3,4. À la valeur par défaut de 0,5 m du projet, les deux sont proches de la parité dans l'ensemble – la projection pour la livraison entière est d'environ 34 GB d'archive contre environ 33 GB de fragments, ce chiffre de fragments étant la somme de 2 842 fichiers par tuile compressés séparément là où les 20,41 GB du Tableau 6 forment une seule structure fusionnée ; le surcoût de conteneur par fichier et les dictionnaires non partageables entre fragments expliquent l'écart, qui n'est pas décomposé ici – mais la moyenne masque une dispersion : l'archive coûte 17 % de plus sur la tuile la plus dense et économise sur les deux tuiles creuses.

À 1 m, le cas se mesure exécution par exécution (section 4.3.7).

Calibration de l'estimateur préalable (section 4.2.5) : ajustement et validation détaillés.

Ses deux constantes de forme sont désormais calibrées sur la mesure. La fraction d'occupation et le nombre d'intervalles par colonne sont donnés par des lois de puissance en taille de cellule horizontale, ajustées sur trente des quarante structures : dix zones de 2 km sur 2 km voxelisées à quatre grilles (annexe C). La quatrième grille est un témoin qui raffine la seule cellule verticale d'un facteur 2,5 : le nombre d'intervalles par colonne bouge de 0,05 %. Le modèle ne porte aucun terme en cellule verticale.

La validation porte sur huit cas tenus hors de l'ajustement : les six exécutions métropolitaines de la section 5.2.1, dont la grille de 2,0 m tombe hors de la plage ajustée, et la paire d'équivalence de la section 5.1. L'estimation centrale y tient entre -3,9 % et +9,3 % de la taille mesurée, et le chiffre de planification, marge de sécurité comprise, de 1,14 à 1,30 fois au-dessus de cette taille et jamais en dessous d'aucun des huit, là où les constantes antérieures sous-estimaient les huit de 22 à 46 %.

À 2,0 m, la grille où l'ancienne estimation était dangereuse, le modèle calibré prédit 19,0 GB contre 19,7 GB relevés ; aux grilles fines, il décrit la structure fusionnée résidente dans un seul processus, ce que le mode partitionné existe précisément pour éviter. Le chiffre y est un plafond et non une attente.

Disposition mémoire de la structure (section 4.2.1) : les deux paragraphes suivants consignent la refonte et sa justification chiffrée.

La façon dont les intervalles sont conservés en mémoire a été repensée une fois, et cette refonte est un choix argumenté qui mérite d'être consigné. La première implémentation stockait un dictionnaire Python associant des tuples (i_x, i_y) à des objets `Column`, chacun possédant de petits tableaux `numpy` : conforme aux usages du langage, lisible et ruineuse à l'échelle, avec des dizaines de millions d'objets Python, un surcoût de pointeurs écrasant la charge utile, et un dictionnaire dont les seules clés consommaient des gigaoctets (le chapitre 5 revient sur l'incident des 116 millions de colonnes qui a tranché la question).

La disposition de production est une structure de tableaux : six vecteurs `numpy` plats (clés de colonnes empaquetées en `uint64`, décalages par colonne en `int64`, bornes d'intervalles en `int32`, classes en `uint8`, comptages en `int32`) donnant 16 octets par colonne plus 13 octets par intervalle, soit une réduction mesurée d'un facteur 16 par rapport à la forme objet. Ce chiffre est mesuré, non estimé : 208,7 octets par instance de la dataclass `Interval` sous `tracemalloc` contre 13 octets en tableaux `numpy` parallèles, soit un rapport de 16,1 ; une estimation antérieure par `sys.getsizeof` (facteur 30 à 50) comptait deux fois le dictionnaire d'instance que CPython amortit entre les instances.

Sur la tuile 18410_51825 (60,9 millions de points), la structure à cellules de 1,0 m par 0,5 m contient 3,42 millions d'intervalles, une réduction d'un facteur 17,8 ; le rapport va de 10,7 pour le sol (plages minces réparties sur de nombreuses colonnes) à 44 pour les bâtiments (blocs pleins avec peu de ruptures de classe), et la moyenne de 13,7 intervalles par colonne, face aux moyennes métropolitaines de 1,78 à 2,63 du Tableau 6, mesure la fragmentation causée par les surfaces verticales. Le Tableau 10 compte la même tuile à des grilles isotropes, ce qui explique que ses chiffres diffèrent.

Les expériences d'encodeurs de la section 5.2 sérialisent chaque structure en une charge utile binaire compacte : par colonne, un en-tête (ix int32, iy int32, n_intervals int32 – les indices sont signés, et négatifs au voisinage d'une origine de grille) suivi des tableaux d'intervalles (z_start int32, z_end int32, class uint8, count int32, chacun répété n_intervals fois). Les trois tuiles de test étaient 18410_51755, 18410_51760 et 18415_51760 (20,9 à 27,1 millions de points ; classes 1 à 6, étage supérieur de canopée 8, et l'eau 9 pour les deux premières). La campagne livre dans le dépôt ses manifestes, sa table de métriques et les vues en plan reproduites au chapitre 5 ; les données de travail volumineuses sont archivées hors de la livraison.

C.1 Tableaux d'appui des chapitres 4 et 5

Les tableaux ci-dessous portent le détail chiffré que les chapitres 4 et 5 résument ; leur numérotation et les renvois du corps du texte sont inchangés.

Tableau 10 : Taille de l'archive LAZ exacte exprimée en multiple de la structure .npz qu'elle encode, pour trois tuiles réelles du Grand Lyon à trois résolutions. Sous 1,00, l'archive est plus petite que la structure. Les neuf cases ont été vérifiées identiques octet pour octet à la conversion (SHA-256 de la structure ré-enregistrée).

Tuile	Caractère	Points	1,0 m au cube	0,5 m au cube	0,25 m xy / 0,1 m z
18410_51825	dense (la plus grande des 2 842)	60 937 237	1,65	1,17	0,65
18385_51830	creuse	16 864	0,62	0,37	0,29
18380_51835	la plus creuse (la plus petite des 2 842)	7 676	1,14	0,82	0,50

Tableau 11 : Coût d'un rayon montant, avant optimisation (échantillons de 100 colonnes, 3 répétitions ; données Experiments/ray_benchmark.json).

Structure	Rayon partant au-dessus du sommet de la colonne	Rayon partant du sol
1,0 m au cube	6,27 ms	0,0033 ms
0,25 m xy, 0,1 m z	57,10 ms	29,35 ms

Tableau 12 : Coût d'un rayon montant, après la borne de plafond (même exécution de mesure que le Tableau 11). Les rapports suivent la précision du chronomètre du côté de CeilingDDA et sont à lire comme des ordres de grandeur, comme les temps dont ils proviennent.

Structure	Départ	Référence	CeilingDDA	Accélération
1,0 m au cube	au-dessus du sommet de colonne	6,27 ms	0,0005 ms	12 547
1,0 m au cube	depuis le sol	0,0033 ms	0,0050 ms	0,7
0,25 xy / 0,1 z	au-dessus du sommet de colonne	57,10 ms	0,0005 ms	114 207
0,25 xy / 0,1 z	depuis le sol	29,35 ms	0,1363 ms	215

Tableau 13 : Comportement sur des structures réelles. Les trois structures sont une même boîte de 1 km sur 1 km dans l'est lyonnais (bbox 1841000, 5175500 à 1842000, 5176500 ; les quatre tuiles de 500 m, 98 946 764 points) voxelisée à trois grilles, si bien que les lignes ne diffèrent que par la résolution. 600 colonnes échantillonnées par structure, rayons verticaux vers le ciel, coefficient appliqué aux classes 3, 4, 5 et 8, rayons bornés par le plafond par colonne de la section 4.4.2 ; données Experiments/transmittance_probe.json.

Structure	k_veg	τ moyen	Fraction avec $\tau > 0,5$	Trajet moyen en canopée (m)	ms/rayon
1,0 m au cube	opaque	0,0000	0,0000	0,034	0,004
1,0 m au cube	0,1	0,0293	0,0333	0,459	0,007
1,0 m au cube	0,3	0,0137	0,0083	0,459	0,006
1,0 m au cube	0,8	0,0045	0,0017	0,449	0,006
0,5 m au cube	opaque	0,4400	0,4400	0,117	0,029
0,5 m au cube	0,1	0,5768	0,5967	1,049	0,039
0,5 m au cube	0,3	0,5193	0,5117	1,049	0,037
0,5 m au cube	0,8	0,4752	0,4600	1,049	0,038
0,25 xy / 0,1 z	opaque	0,6500	0,6500	0,023	0,143
0,25 xy / 0,1 z	0,1	0,8145	0,8350	0,592	0,189
0,25 xy / 0,1 z	0,3	0,7693	0,7717	0,592	0,188
0,25 xy / 0,1 z	0,8	0,7271	0,7250	0,592	0,195

Cas contrôlés, comblement de lacunes avec une tolérance d'un mètre, avant la correction :

Cas	Comptages d'entrée	Sortie
même classe, lacune d'un voxel	20 + 30 = 50	30
même classe, contiguës	20 + 30 = 50	30
même classe, superposées	20 + 30 = 50	30
classes différentes, lacune	20 + 30 = 50	50 (correct)

Tableau 14 : Géométrie solaire à Lyon (45,7647 N, 4,8386 E) confrontée à la théorie. La colonne prédite prend la déclinaison pour exactement 0 ou exactement plus ou moins 23,44 degrés à la date du calendrier, ce qu'elle n'est pas à l'instant du midi solaire local, si bien que les résidus de 0,04 degré à l'équinoxe et au solstice de décembre sont des propriétés de la valeur de référence plutôt que de l'implémentation.

Date	Élévation prédite à midi	Calculée	Durée du jour calculée
Solstice de juin	$90 - 45,76 + 23,44 = 67,68^\circ$	$67,67^\circ$	15,67 h
Équinoxe	$90 - 45,76 = 44,24^\circ$	$44,20^\circ$	12,08 h
Solstice de décembre	$90 - 45,76 - 23,44 = 20,80^\circ$	$20,84^\circ$	8,58 h

C.2 Mesures complémentaires

Les mesures ci-dessous ont été conduites après la rédaction des chapitres 4 et 5, pour fermer les points que ceux-ci laissaient ouverts. Chacune est vérifiée par des identités entières exactes plutôt que par des comparaisons à tolérance ; les journaux et les empreintes SHA-256 sont conservés avec les sorties, hors de la livraison.

Structures et grilles. Dix zones de 2 km sur 2 km ont été tirées par une graine enregistrée parmi les 2 287 blocs dont les seize tuiles sont disponibles localement, sous une séparation minimale de 3 km, puis voxelisées à quatre grilles : 1,0 m, 0,5 m et 0,25 m au cube, et 0,25 m à l'horizontale par 0,1 m à la verticale. Les quarante structures ont abouti, chacune du premier coup, en 4 h 24 min d'exécution continue, pour 3 546 676 808 points par grille. Deux cents contrôles exacts passent : seize fragments présents sans tuile manquante, somme des comptages de points des fragments égale au total fusionné, aucune colonne hors du réseau de sa zone sur 1 388 297 158 colonnes, aucun intervalle mal formé ni comptage nul sur 2 124 475 488 intervalles, et statistiques écrites depuis les fragments en accord avec la structure fusionnée.

Le nombre d'intervalles par colonne est fixé par la seule cellule horizontale. Il vaut 2,116 à 1,0 m, 1,776 à 0,5 m et 1,480 à 0,25 m, et 1,481 à la quatrième grille, qui raffine la cellule verticale d'un facteur 2,5 à cellule horizontale égale. Les colonnes et les intervalles par grille valent 38 045 963 et 80 495 729 à 1,0 m au cube, 151 496 569 et 269 071 024 à 0,5 m, 599 377 313 et 887 229 468 à 0,25 m, et 599 377 313 et 887 679 267 à la quatrième grille. Le jeu de colonnes y est d'ailleurs identique octet pour octet à celui de 0,25 m au cube pour les dix zones, et le nombre total d'intervalles ne monte que de 0,051 %. La fraction d'occupation vaut 0,951, 0,947, 0,937 et 0,937, contre la valeur fixe de 0,6 que l'ancien estimateur supposait, avec une amplitude par zone de 0,690 à 1,000.

Densité de points et structure verticale ne vont pas ensemble. La densité des dix zones va de 38,8 à 129,2 points par mètre carré, soit un facteur 3,3, tandis que le nombre d'intervalles par colonne à 0,5 m va de 1,461 à 2,046 ; la zone la plus dense est parmi les plus simples verticalement. Un chiffre du

chapitre 5 tiré d'une seule zone porte donc un facteur trois non énoncé en densité et un facteur 1,4 en structure verticale, ce qui est l'argument du tirage à dix zones.

Sélection des colonnes d'extinction. Le décompte des exclusions de la section 4.4.5 se répartit ainsi à 0,5 m, sur 3 969 467 colonnes : 1 571 325 sans retour de sol (39,59 %), 1 357 755 sans végétation (34,20 %), 41 217 sans canopée au-dessus du sol (1,04 %), 233 222 sous le plancher de 2 m (5,88 %), 26 025 sous le seuil de 20 retours (0,66 %), et aucune par les deux gardes de comptage nul ; 3 229 544 écartées, 739 923 qualifiées, soit 18,64 %. À 1,0 m, sur 999 170 colonnes, 707 993 sont écartées et 291 177 qualifiées, soit 29,14 %. L'ajustement lui-même porte sur 739 916 colonnes à 0,5 m et 291 053 à 1,0 m : pour les autres, la fraction de sol par impulsion n'est pas strictement comprise entre 0 et 1 et la garde de dégénérescence que la dérivation applique déjà les écarte. Le plancher de 2 m écarte proportionnellement 2,4 fois plus de colonnes à 0,5 m qu'à 1,0 m, si bien que l'ajustement à 1,0 m ne porte pas sur un sous-échantillon des colonnes de 0,5 m mais sur une population autrement constituée : c'est le mécanisme de la limitation (b) de la section 4.4.5.

Recensement par impulsion. Les impulsions ont été reconstituées depuis le LAZ par le couple (identifiant de source, temps GPS), d'abord sur les boîtes B04 et B10 à 1,0 m au cube, puis sur l'ensemble du corpus de dérivation. Sur les deux boîtes, soit 347 946 et 60 782 colonnes qualifiées, l'identité tient : le nombre de points lus égale, à l'unité près, celui de la structure, 65 513 414 et 45 753 292, et les retours de sol comme les retours de végétation par colonne égalent ceux de la structure sans aucune colonne divergente ; le rapport du coefficient par comptages au coefficient par impulsion y vaut 1,050 à la médiane sur B04 et 1,098 sur B10, 1,222 et 1,413 au neuvième décile. Sur le corpus de dérivation, 98 946 764 points se groupent en 76 743 324 impulsions, soit 1,289 retour par impulsion, avec 49 groupes en collision, un taux de $6,4e-7$ qui les écarte de l'ajustement. L'estimateur par comptages y est d'abord reproduit champ pour champ, 41 champs à chaque grille et aucun écart, si bien que l'ajustement refait corrige un objet qu'il reproduit d'abord. Le rapport médian par colonne y vaut 1,0300 à 0,5 m et 1,0710 à 1,0 m, cette seconde valeur tombant entre les deux valeurs des boîtes et la première en deçà des deux, et le coefficient par comptages dépasse celui par impulsion sur 99,978 % et 99,953 % des colonnes. Le biais reste le plus fort pour la classe 3 et le plus faible pour la classe 8. Enfin, 39,05 % des points du bloc de dérivation sont à retours multiples contre 34,5 % sur la livraison mesurée au même titre, soit environ 1,13 fois, ce qui explique une part de l'ampleur du déplacement. Les 19,68 % cités par ailleurs comptent les retours non premiers sur la livraison, une autre mesure, qui ne se compare à aucune des deux.

Angles de balayage près des arêtes. Une passe par point sur les quatre tuiles de la mesure d'équivalence, 52 705 942 points et 211 221 colonnes de bâtiment, joint l'angle de balayage absolu à la distance de Chebyshev à la colonne de bâtiment la plus proche. Les retours de sol situés d'une à six cellules d'un bâtiment ont un angle absolu moyen de 12,87 degrés contre 13,17 degrés au-delà de quarante cellules : l'écart est du signe attendu et négligeable, et 74 % des retours de sol immédiatement voisins d'un bâtiment arrivent à plus de 5 degrés du nadir, si bien que le chiffre de plus ou moins 5 degrés que portaient les notes de conception n'est pas soutenu. La largeur suspecte qui en découle vaut 1,00 m à la médiane, 2,46 m au neuvième décile et 3,67 m au quatre-vingt-dix-neuvième, et elle croît avec la hauteur du bâtiment, que ce relevé porte à 4,50 m à la médiane et à 14,00 m au plus haut.

Estimateur préalable. Les six exécutions métropolitaines de la section 5.2.1 ont été comparées à leur pic de mémoire résidente enregistré, par deux analyseurs ne partageant aucun code et s'accordant exactement sur les six relevés. L'ancien modèle sous-estimait le pic de 35 % à 2,0 m et le surestimait d'un facteur 3,7 à 0,5 m, deux erreurs de sens opposé : la fraction d'occupation supposée trop basse pousse vers le bas, la résidence de la structure entière dans un seul processus pousse vers le haut. La calibration exposée à la section 4.2.5 en découle.

Ordre des post-traitements. Les quatre comparaisons croisées d'ordre existent désormais toutes les quatre. Les points sont conservés exactement dans les quatre, les structures ne coïncident dans aucune, et la passe d'absorption élargit le désaccord partout : de 55 391 à 61 769 colonnes divergentes sur la

première zone à 0,5 m, de 45 263 à 47 869 à 0,25 m (xy) / 0,1 m (z), de 48 312 à 59 660 sur la seconde zone à 0,5 m et de 35 595 à 39 735 à 0,25 m (xy) / 0,1 m (z), soit de +5,8 % à +23,5 %.

C.3 Comparaison aux approches 2-D et 2.5-D

La comparaison de la section 4.4.7 a porté sur dix carrés de 1 km de l'inventaire du Grand Lyon, stratifiés par fraction bâtie de 0,2 à 25,9 %, à 1,0 m et 0,5 m au cube : vingt structures et 694 428 978 points. Les modèles ont été vérifiés sur quatre scènes de synthèse de 40 sur 40, trois en accord et le diagnostic de canopée au-dessus du sol, chaque modèle calculé deux fois avec des résultats identiques. Sur un plan horizontal, sur des blocs pleins et sur un anneau fermé élevé dont 80,0 % des colonnes sont sous la journée pleine, 0 colonne sur 1 600 diffère, et sur le plan les deux modèles rendent la forme fermée 15,75 heures – 63 positions solaires à un quart d'heure – pour chaque colonne. La quatrième scène, une canopée sur du sol, est celle où ils doivent diverger, et elle sépare les deux mécanismes : des 400 colonnes portant une canopée au-dessus du sol, 395 diffèrent et le modèle S lit la journée pleine sur chacune d'elles, tandis que des 1 200 colonnes de sol à corps unique, 828 diffèrent et le modèle S est plus bas de 1,42 heure en moyenne, parce que remplir l'air mesuré sous une couronne voisine bloque les rayons rasants qui passent dessous dans la structure.

Heures d'ensoleillement direct par modèle, sur les colonnes appariées de la zone végétalisée B04 et de la zone bâtie B10. Chaque colonne de 0,5 m est l'une des quatre qui couvrent une clé de 1,0 m tirée, de sorte que les deux grilles sont lues sur le même sol ; 3 000 colonnes par zone et par grille, solstice d'été, échantillonnage à 15 minutes. La dernière ligne donne la part de colonnes sur lesquelles les deux modèles rendent exactement la même valeur.

modèle	B04, 1,0 m	B04, 0,5 m	B10, 1,0 m	B10, 0,5 m
structure en colonnes, opaque	4,4408 h	5,6997 h	10,4015 h	11,4364 h
structure en colonnes, sensible aux classes	6,2593 h	6,7399 h	11,1239 h	11,8814 h
modèle de surface (S)	9,7322 h	9,4548 h	12,7352 h	12,7349 h
contrôle échantillonné au terrain (S2)	4,1293 h	5,3190 h	10,2286 h	11,2146 h
modèle S moins la structure opaque	+5,2914 h	+3,7550 h	+2,3337 h	+1,2985 h
modèle S2 moins la structure opaque	-0,3115 h	-0,3808 h	-0,1729 h	-0,2218 h
colonnes où le modèle S égale la structure opaque	35,1 %	43,4 %	71,3 %	75,8 %

Les autres requêtes sur les dix zones. Le volume de canopée est le volume mesuré par la structure rapporté au volume qu'implique le champ de hauteurs, l'empreinte multipliée par la hauteur de l'objet ; les colonnes à lacune portent deux corps occupés ou plus séparés par de l'air mesuré, qu'un champ de hauteurs ne peut pas représenter ; la classe supérieure donne l'accord entre l'intervalle le plus haut de la structure et la classe du point brut le plus haut de la même cellule. Quatre lectures complètent le tableau : la structure mesure de 0,65 à 0,88 du volume de canopée qu'implique le champ de hauteurs à 1,0 m, si bien que l'empreinte multipliée par la hauteur surestime de 13 à 54 % ; presque tout désaccord de classe supérieure est une paire de classes voisines à l'intérieur d'un même voxel ; la hauteur de base de couronne se lit de 1,0 à 1,7 m plus haut à 0,5 m, de sorte qu'une question de dégagement pour la plantation posée à une grille de 1 m hérite d'environ un mètre et demi de biais ; et la même passe à retrouvé exactement le nombre de points bruts des dix zones, soit une vérification de conservation de bout en bout sur 694 428 978 points.

zone	bâti	végétation	volume de canopée, 1,0 m	0,5 m	colonnes à lacune, 1,0 m	0,5 m	classe sup., 1,0 m	0,5 m
B01	0,2 %	22,3 %	0,8675	0,4936	9,26 %	13,47 %	94,76 %	97,99 %
B02	0,7 %	44,9 %	0,8823	0,5188	16,70 %	30,78 %	86,61 %	95,03 %
B03	2,4 %	32,6 %	0,7586	0,4518	18,18 %	22,69 %	95,63 %	98,31 %
B04	3,7 %	58,6 %	0,6512	0,3457	40,04 %	40,48 %	94,92 %	97,94 %
B05	6,7 %	49,4 %	0,7935	0,4609	30,21 %	38,76 %	95,01 %	97,78 %
B06	8,3 %	23,5 %	0,7821	0,4368	17,59 %	18,22 %	97,33 %	98,89 %
B07	11,9 %	37,3 %	0,7432	0,3955	26,18 %	31,31 %	94,24 %	97,91 %
B08	13,9 %	12,2 %	0,7906	0,4492	11,70 %	15,76 %	97,21 %	98,86 %
B09	19,8 %	22,6 %	0,7498	0,4201	19,20 %	23,83 %	96,41 %	98,61 %
B10	25,9 %	7,9 %	0,7734	0,3869	8,53 %	10,95 %	98,32 %	99,42 %

Coût, mesuré en vis-à-vis sur un même kilomètre carré (x de 1 845 000 à 1 846 000, y de 5 175 000 à 5 176 000, EPSG:3946). Les acquisitions diffèrent – le référentiel raster est la livraison 2018, 12 156 432 points, tandis que notre corpus du même carré est la livraison 2023, 8,46 fois plus dense – de sorte que le vis-à-vis fait tourner notre voxelisation sur le fichier de 2018, ce qui rend les jeux de points identiques.

axe	mesure	ce que cela chiffre
construction, notre voxelisation sur la tuile 2018 (12 156 432 points)	2,50 s à 1,0 m, 3,58 s à 0,5 m	notre implémentation
construction, la boucle de référence point par point, mêmes points	18,65 s	l'implémentation de référence
construction, à l'identique sur le comblement de trous, sortie identique	0,54 s contre 0,478 s pour la nôtre	l'algorithme
construction, notre voxelisation sur notre corpus 2023 du même carré (102 896 384 points)	117,4 s à 1,0 m, 99,0 s à 0,5 m	notre implémentation sur des données plus denses
stockage, les quatre GeoTIFF LZW	7 310 972 o	la représentation raster
stockage, notre structure à 1,0 m	6 474 827 o, 0,886 fois les rasters	la représentation volumétrique
stockage, notre structure à 0,5 m	20 807 936 o, 2,846 fois les rasters	la représentation volumétrique
requête, une lecture scalaire par cellule	raster 0,231 us, structure 5,064 us, 21,9 fois plus lente	les deux représentations
requête, les questions verticales	le raster ne peut pas y répondre	les deux représentations

Deux de ces lignes chiffrent des implémentations plutôt que des représentations. La construction de référence est une boucle Python sur chaque point, et deux concessions ont joué en sa faveur : un affichage de progression absent de l'environnement a été remplacé par une itération simple, ce qui rend la boucle plus rapide que dans sa version livrée, et ses quatre écritures raster sont exclues de son total alors que le nôtre inclut l'écriture de la structure. Sur le fragment que les deux chaînes exécutent jusqu'à une sortie identique, le comblement de trous, elles sont à un facteur 1,1 l'une de l'autre. Le résultat de stockage a un mécanisme derrière lui : la structure porte 999 736 colonnes et 1 955 696 intervalles pour

ce kilomètre carré, soit un peu moins de deux plages par colonne, de sorte que l'encodage par plages sur la verticale fait un travail réel sur une surface qui est le plus souvent un seul corps plein par cellule.

Annexe D - Figures supplémentaires

Les Figures 18 à 23, citées aux chapitres 4 et 5, sont regroupées ici : les trois voies d'export de la section 4.3.4, la mesure de coût de rayon de la section 4.4, la distribution d'extinction de la section 4.4.5, et le graphe complet des imports de la section 4.1.

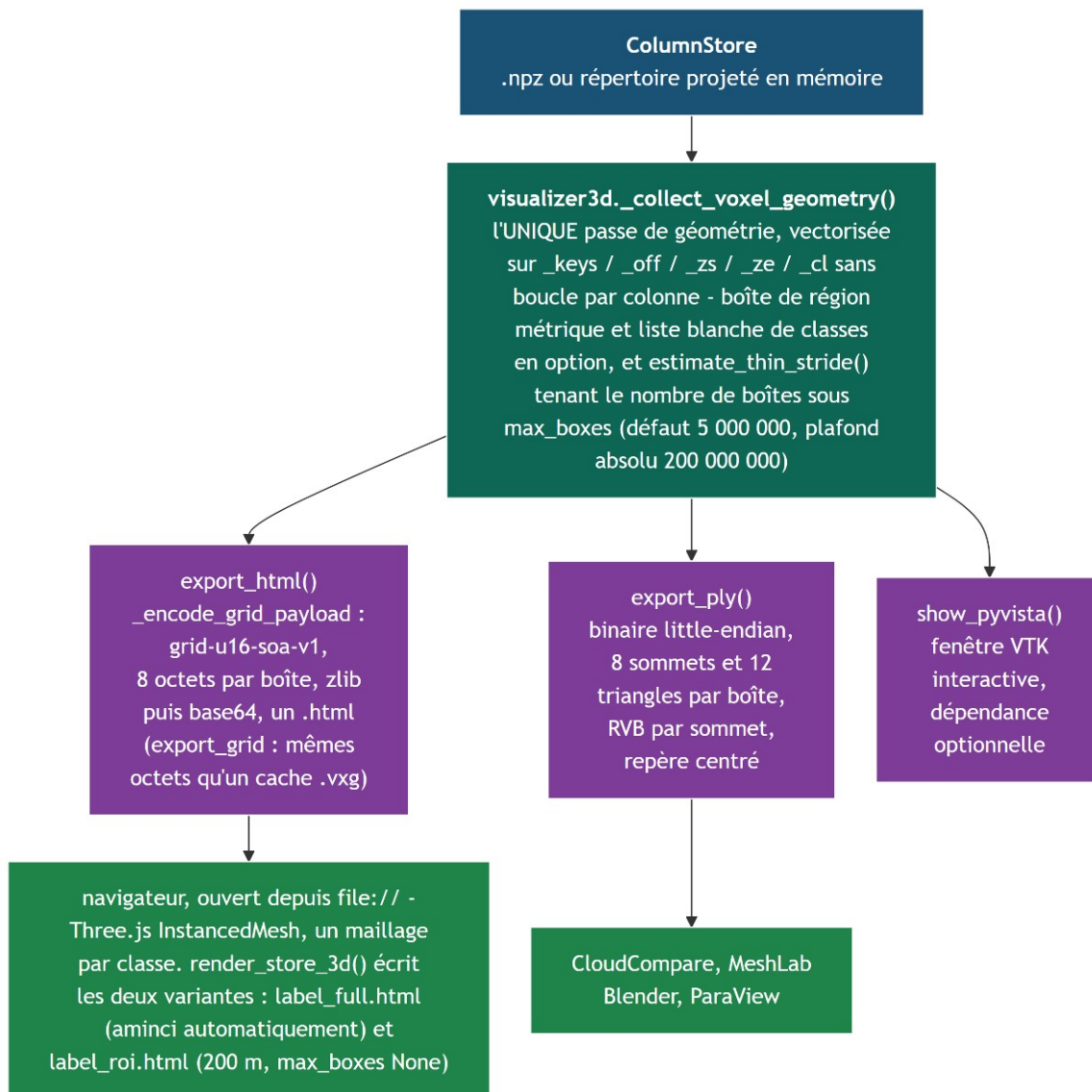


Figure 18 : Les voies d'export internes au processus : `export_html`, `export_ply` et `show_pyvista` sont des encodeurs terminaux au-dessus d'un unique appel à `visualizer3d._collect_voxel_geometry`. Le budget d'amincissement et le filtre de région s'appliquent ainsi identiquement aux trois. Le fichier HTML porte sa charge utile de voxels en ligne et ne charge depuis un CDN que ses deux modules `Three.js`.

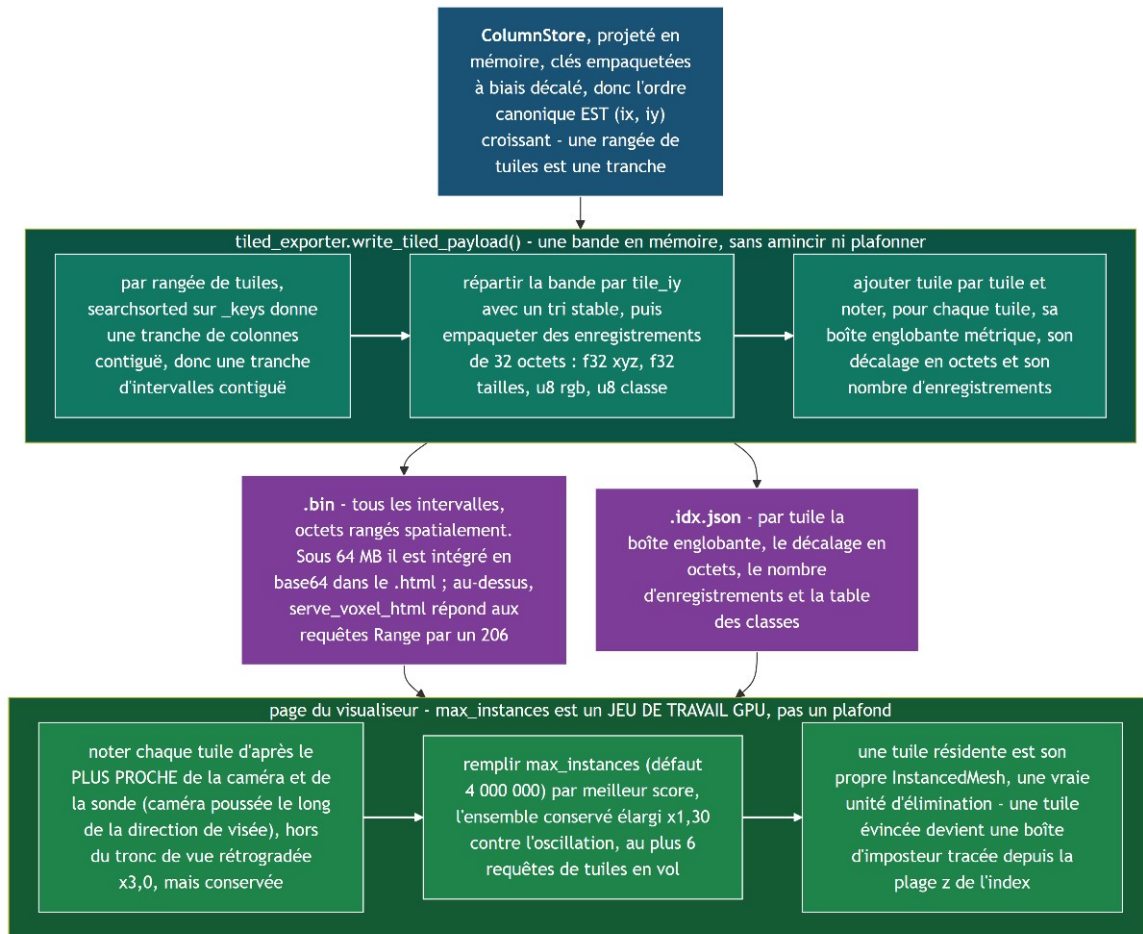


Figure 19 : La voie d'export en flux : `write_tiled_payload` parcourt la structure une rangée de tuiles à la fois. Le pic de mémoire est celui d'une bande et non de la charge utile, et le visualiseur traite `max_instances` comme un ensemble de travail GPU – les tuiles proches de la caméra restent résidentes en pleine finesse, les autres s'effondrent en silhouettes simplifiées.

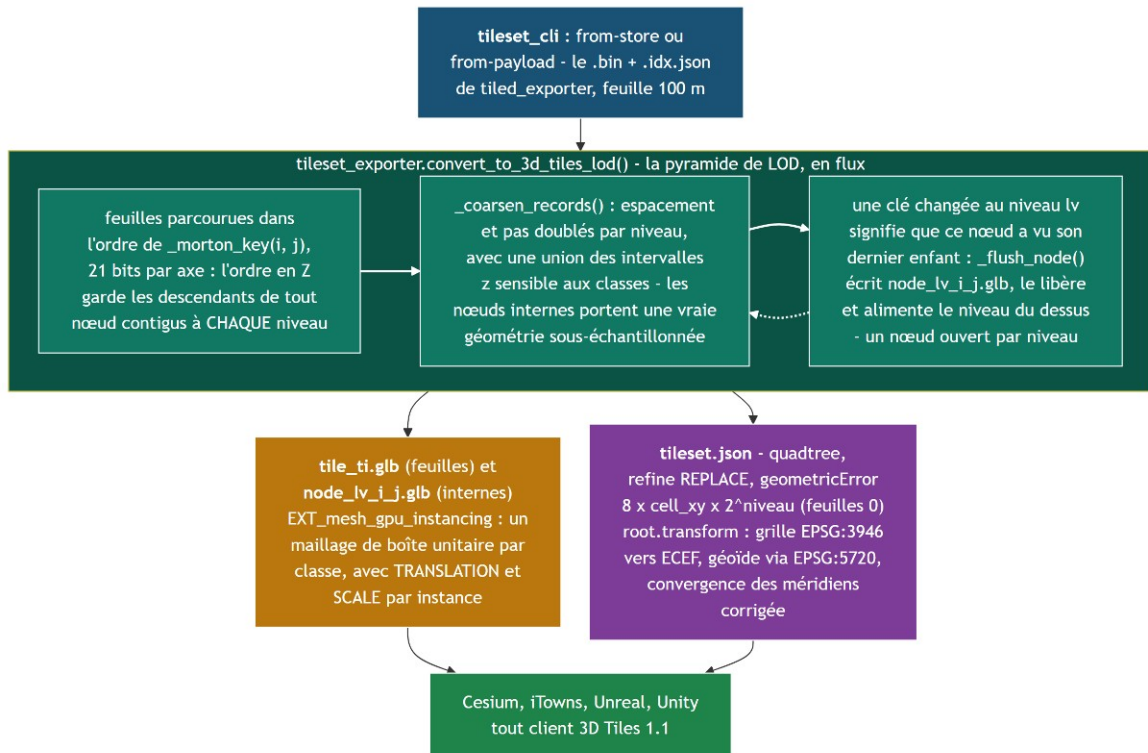


Figure 20 : La voie d'export 3D Tiles, qui consomme la charge utile en flux plutôt que la structure : le parcours des feuilles dans l'ordre de Morton permet à `convert_to_3d_tiles_lod` d'écrire chaque nœud interne dès l'arrivée de son dernier enfant, d'où un pic de mémoire qui suit la profondeur de l'arbre et non l'étendue de la zone.

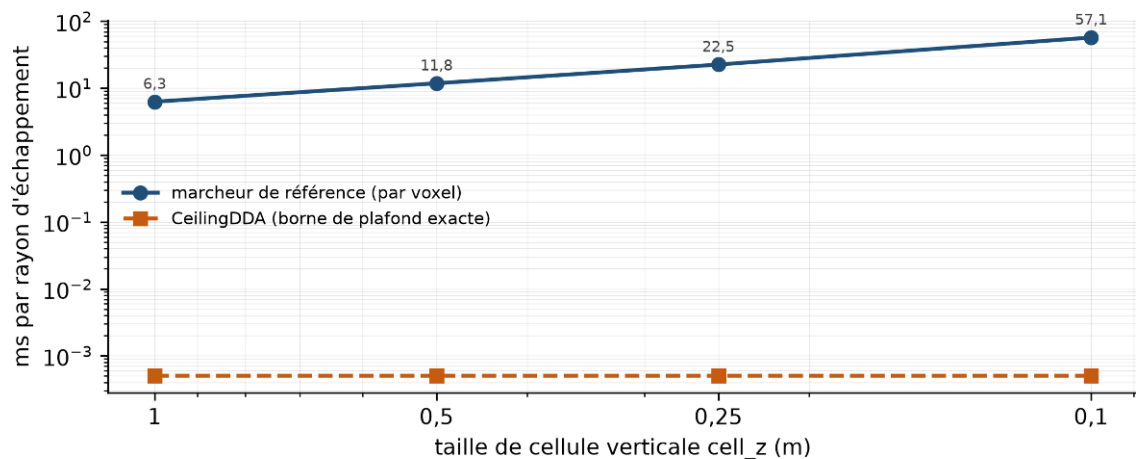


Figure 21 : Coût d'un rayon d'échappement montant en fonction de la taille de cellule verticale, marcheur de référence contre marcheur borné par le plafond (échelle logarithmique ; temps relevés sur une seule machine, ordres de grandeur ; données `Experiments/ray_benchmark.json`).

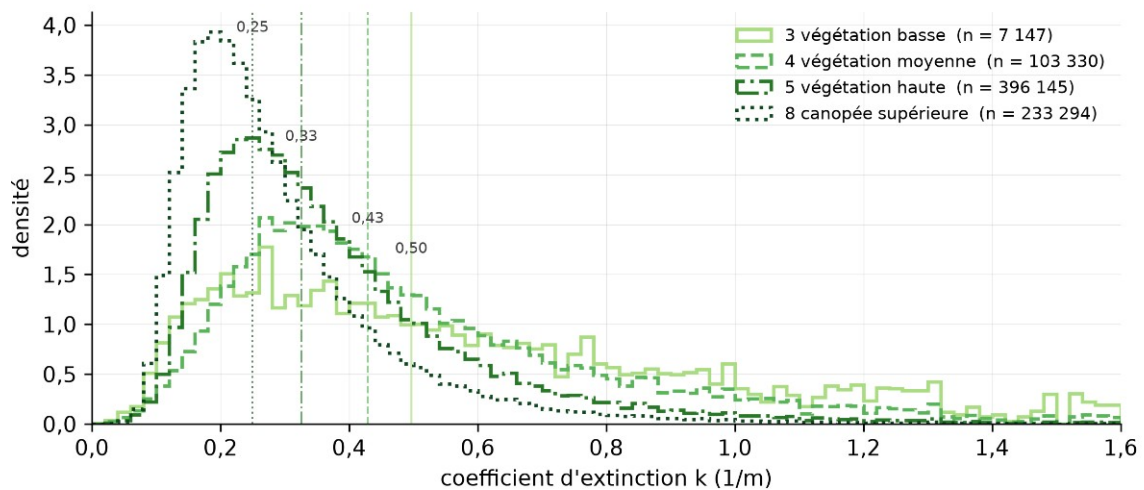


Figure 22 : Distribution du coefficient d'extinction par classe de végétation dominante sur la structure à 0,5 m, médianes de classe marquées (739 916 colonnes après la garde de dégénérescence ; la dérivation de la section 4.4.5).

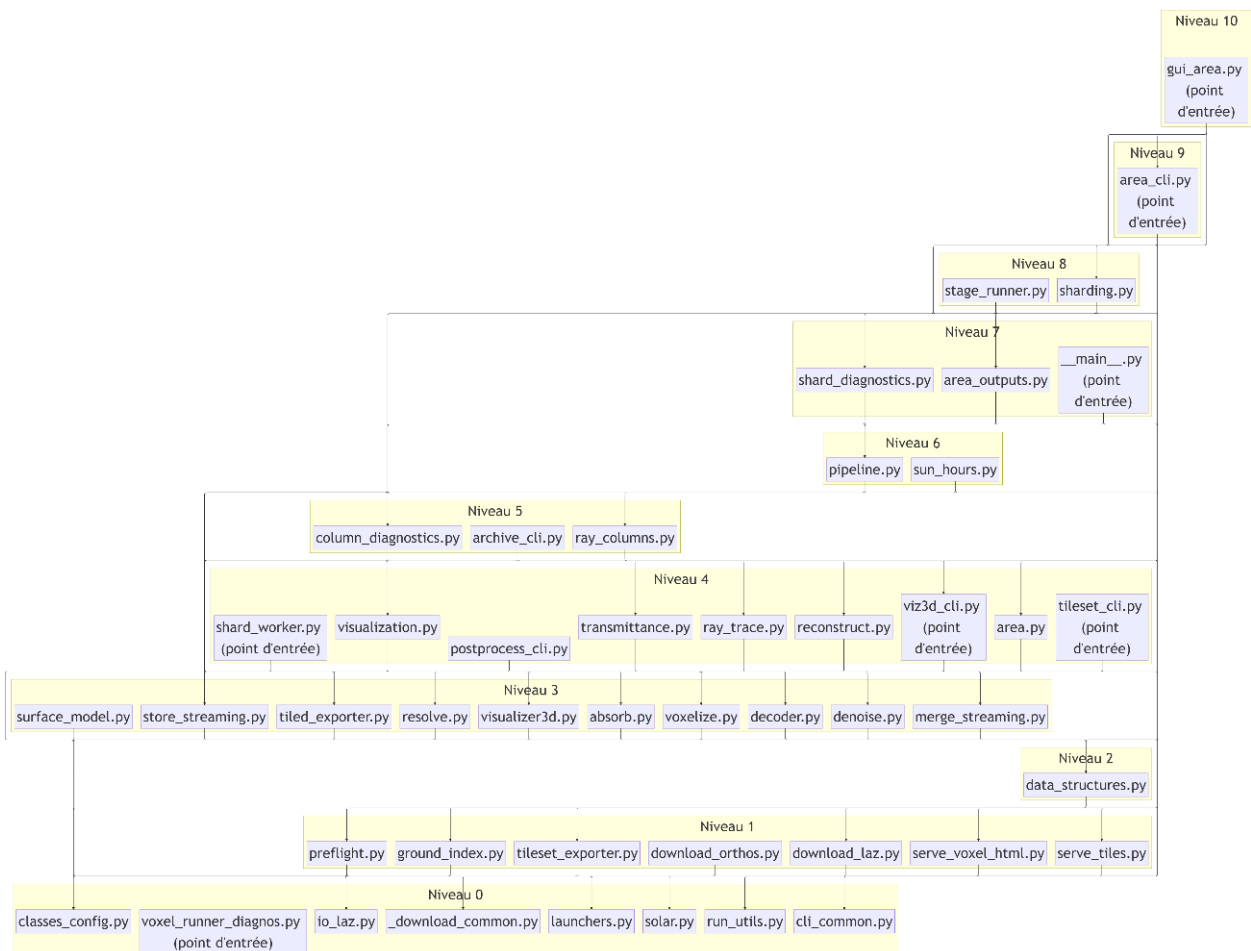


Figure 23 : Le graphe complet des imports de modules du paquet voxelizer – les quarante-sept modules qui s'importent les uns les autres, sur onze niveaux de dépendance ; l'initialiseur du paquet n'est pas dessiné. Chaque flèche va d'un module vers un module qu'il importe. Les points d'entrée se tiennent en haut et les feuilles du niveau 0 en bas.

Annexe E - Codes de classification LiDAR HD de l'IGN

Les codes de classification portent la sémantique de l'entrée. Les tuiles livrées sont en LAS 1.2, format d'enregistrement de points numéro 1 (LAS 1.4 numérote les formats d'enregistrement de points de 0 à 10 [7] ; le numéro 1 est le format de base augmenté d'un champ de temps GPS [8]), vérifié sur l'ensemble des 2 842 fichiers, et les codes de classification réellement présents vont de 1 à 9 seulement : non classé (1), sol (2), végétation répartie selon la hauteur au-dessus du sol en basse (3, sous 50 cm), moyenne (4, de 50 cm à 1,5 m) et haute (5, au-dessus de 1,5 m), bâtiment (6), bruit de points bas (7), un étage supérieur de canopée (8) et eau (9).

Une clarification s'impose, car il est facile de se tromper : la table de codes de ce projet est taillée pour LAS 1.4 ; elle déclare les codes ASPRS standards 0 à 22 (sauf 12, que la table omet) plus la plage personnalisée de l'IGN 64 à 67 (« sursol pérenne », artefact, point virtuel, bâti probable non confirmé) – vingt-six en tout. C'est une propriété du logiciel, non des données. Ces codes personnalisés ne peuvent exister que dans les formats de points étendus 6 à 10 de LAS 1.4 ; dans le format de points numéro 1, le champ de classification fait cinq bits, valeur maximale 31 [8] : les codes 64 à 67 ne peuvent donc pas physiquement apparaître dans ce corpus, et aucun n'y a été observé.

Le tableau ci-dessous liste les codes de classification auxquels ce rapport se réfère, avec les noms et remarques employés partout, tirés des vingt-six que déclare la table de codes du projet ; les dix omis n'apparaissent nulle part dans ce corpus. Les codes 0 à 22 appartiennent à l'espace standard de l'ASPRS ; les codes 64 à 67 sont les extensions documentées de l'IGN dans la plage utilisateur. La table plus large est conservée pour que le même logiciel ingère une livraison LAS 1.4 sans modification.

Code	Nom	Remarque
0	created_never_classified	Valeur par défaut de la saisie brute ; ne devrait pas survivre au contrôle qualité
1	unclassified	Traité mais non attribué (véhicules, mobilier urbain, bruit)
2	ground	Surface de terrain nu, naturelle ou artificielle ; ancre l'indice de sol
3	low_vegetation	Végétation sous 50 cm (découpage en hauteur de l'IGN)
4	medium_vegetation	Végétation entre 50 cm et 1,5 m
5	high_vegetation	Végétation au-dessus de 1,5 m : arbres et canopée
6	building	Toits, façades, cheminées, balcons
7	low_point_noise	Points aberrants bas et retours parasites sous la surface
8	high_vegetation_upper	Model Key-point (mass point) dans la table de classes des formats de points 0-5, Reserved seulement dans la table étendue de LAS 1.4 pour les formats 6-10 ; étage supérieur de canopée de fait dans les livraisons du Grand Lyon – 3,32 milliards de points, 6,4 %, dans 2 691 des 2 842 tuiles
9	water	Rivières, lacs, étangs
11	road_surface	Surfaces de chaussée (déclarées ; non présentes dans les tuiles de cette livraison)
17	bridge_deck	Tablier de pont uniquement ; piles et parapets se retrouvent en non classés (déclaré ; absent de cette livraison)
64	perennial_above_ground	IGN : structures durables telles que pylônes, lignes électriques, éoliennes, antennes
65	artefact	IGN : points inexpliqués ou parasites
66	virtual_point	IGN : points synthétiques insérés sous les ponts pour la modélisation du terrain
67	misc_building_like	IGN : objets de type bâti non confirmés par les bases de données de référence

Annexe F - Glossaire

3D Tiles : un standard de l'OGC pour la diffusion en flux de grands jeux de données géospatiales 3-D ; la cible d'export pour l'interopérabilité de ce travail.

ASPRS : American Society for Photogrammetry and Remote Sensing ; éditeur du format LAS et de ses codes de classification standards.

CC46 (RGF93/CC46, EPSG:3946) : la zone de projection conique conforme couvrant Lyon, dans le système géodésique français RGF93 ; le système de coordonnées de référence de toutes les tuiles d'entrée.

ColumnStore : la structure de données centrale de ce projet ; un ensemble creux de colonnes, chacune portant des intervalles sémantiques compressés par plages, disposés en tableaux numpy plats.

CRS : Coordinate Reference System, système de coordonnées de référence ; le repère qui donne aux coordonnées une position sur la Terre.

DDA : Digital Differential Analyzer, analyseur différentiel numérique ; la famille d'algorithmes de parcours de grille utilisée par le marcheur de rayons pour visiter les colonnes le long d'un rayon.

IGN : Institut national de l'information géographique et forestière ; l'agence cartographique nationale française et productrice du LiDAR HD.

Indice de Jaccard [46] : recouvrement de deux ensembles (intersection sur union) ; utilisé par classe pour mesurer quelle part de l'occupation brute un encodeur préserve.

Indice de sol : indice de voxel, par colonne, de la surface du terrain, pris sur l'intervalle classé sol le plus haut ; il ancre le décodeur d'espaces vides.

Intervalle : une plage verticale maximale de voxels de même classe dans une colonne, stockée sous la forme (z_start, z_end, class, count) avec z_end exclusif.

Lacune / vide : une lacune est l'espace vertical entre deux intervalles occupés d'une même colonne, que le comblement des lacunes (variante v2) et le regroupement d'intervalles peuvent refermer ; le vide désigne l'état d'un voxel inoccupé tel que le décodeur le classe (air mesuré, intérieur opaque ou sous-sol). Le rapport maintient les deux termes distincts.

LAS / LAZ : le format d'échange de nuages de points de l'ASPRS et sa variante compressée LASzip ; le format d'entrée de ce travail.

LiDAR HD : le relevé laser aérien national à haute densité de la France par l'IGN, livré sous forme de nuages de points classifiés.

Marcheur de rayons : l'algorithme de parcours qui avance voxel par voxel le long d'un rayon (famille DDA) ; distinct du lanceur de rayons, la couche de requêtes qui l'emploie.

Nadir : la direction de visée verticale vers le bas d'un capteur aéroporté ; l'hypothèse de faisceau nadiral traite chaque colonne comme si elle avait été balayée verticalement.

NODATA : valeur sentinelle marquant une colonne sans retour de sol ; ces colonnes prennent par défaut un intérieur opaque dans le décodeur.

$O(1)$, $O(n)$: notation de coût asymptotique, qui décrit la croissance du coût d'une opération avec la taille n de son entrée ; $O(1)$ est un coût constant, indépendant de la taille, $O(n)$ un coût qui lui est proportionnel.

Octree : une structure arborescente qui subdivise récursivement l'espace 3-D en huit octants ; la représentation classique des voxels creux, exclue ici par la note de cadrage du projet.

OpenVDB : la structure de données volumétrique creuse open source développée chez DreamWorks, standard de fait pour les données volumétriques dans les effets visuels ; NanoVDB en est le pendant allégé pour les GPU et les flux out-of-core.

Orthophoto : une photographie aérienne géoréférencée et corrigée des distorsions ; utilisée dans ce travail uniquement comme fond d'affichage, jamais comme entrée de calcul.

Out-of-core : traiter des données plus grandes que la mémoire en diffusant des morceaux bornés ; la propriété de la chaîne de zone partitionnée.

Partitionnement (sharding) : découper une exécution de zone en structures indépendantes par tuile afin que le pic de mémoire reste borné par une seule tuile.

RLE : Run-Length Encoding, codage par plages ; compresser une séquence en stockant chaque plage de valeurs identiques une seule fois avec sa longueur.

SBRT : Stack-Based Representation of Terrains ; le modèle de terrain en piles de matériaux stratifiées de la ligne de travaux Arches, ancêtre conceptuel de la structure en colonnes.

Score F1 : la moyenne harmonique de la précision et du rappel, utilisée comme résumé en un seul chiffre de la qualité d'une classification ; 1 correspond à un accord parfait.

Structure de tableaux : une disposition mémoire tenant un tableau plat par champ au lieu d'un objet par enregistrement ; la disposition du ColumnStore.

Taux de désaccord sur la classe supérieure : le pourcentage de colonnes dont la classe la plus haute diffère entre la structure brute et la structure encodée. Il compte à quelle fréquence la surface supérieure encodée est en désaccord avec la surface brute ; c'est un taux, non une distance.

Voxel : élément de volume ; l'analogue 3-D d'un pixel, la cellule atomique d'une grille volumétrique régulière.